The Super Friendly CCNA Study Guide

Learn Networking Like You're 12 — But Go Really, Really Deep

What is this? This is a big, friendly book that teaches you everything you need to pass the **CCNA** exam (Cisco Certified Network Associate, exam code **200-301**). It explains hard computer stuff in easy words, with lots of pictures made out of text, real examples, and step-by-step commands you can practice.

How to use it: Read it top to bottom the first time. Don't skip the "Story Time" and "Try It" boxes — that's where the learning sticks. Later, use the Table of Contents to jump around and review.

About the depth sections: some sections go deeper than the CCNA exam strictly requires — the mechanism underneath, not just the behaviour. They're marked, so you can skip them on a tight timeline and come back later. They're also where CCNP begins; **Appendix B** maps them onto what comes next.

This guide is one of five: the **Study Guide** (this book) teaches the concepts, the **Practice Question Bank** tests them with 152 exam-style questions grouped by domain, the **Subnetting Drill Sheet** builds the one skill you need to be fast at, the **Flashcard Deck** (208 cards, for Anki or Quizlet) drills the facts into instant recall, and the **Lab Workbook** (17 hands-on labs for EVE-NG/GNS3) turns what you read into what you can type. Read a chapter here, then answer that domain's questions and build its lab — the appendix at the back maps every official exam topic to the section that covers it.

Table of Contents

1. What Is a Network? (The Big Picture)
2. How Computers Talk: Models & Layers
3. The Physical Stuff: Cables, Signals & Ports
4. Numbers Computers Use: Binary & Hex
5. MAC Addresses & Ethernet
6. Switches: The Traffic Cops of the LAN
7. VLANs: Splitting One Switch Into Many
8. Trunks, VTP & Inter-VLAN Routing
9. Spanning Tree Protocol (STP): Stopping Loops
10. EtherChannel: Bundling Cables
11. IP Addresses: The Home Address of Every Device
12. Subnetting: Cutting the Network Cake
13. IPv6: The New, Giant Address System
14. Routers & Routing: Finding the Path

15. Static Routing
16. Dynamic Routing & OSPF
17. DHCP, DNS, NAT & Other Helpers
18. Network Security Basics
19. Access Control Lists (ACLs)
20. Wireless Networking (Wi-Fi)
21. Network Management & Monitoring
22. Automation & Programmability
23. Troubleshooting Toolbox
24. Exam Tips & Study Plan
25. Glossary
26. Appendix: Exam Blueprint Coverage Map
27. Appendix B: Beyond CCNA — The Road to CCNP

Chapter 1: What Is a Network? (The Big Picture)

1.1 The Simple Idea

A **network** is just **two or more devices connected so they can share stuff**. That "stuff" can be messages, pictures, videos, games, or files.

Think about talking to your friend with two paper cups and a string:

You hold one cup and talk into it; your **friend** holds the other and listens. The **string** stretched between them carries the sound.

- The **cups** are like your computers (they turn your voice into something the string can carry, and back again).
- The **string** is like the **cable** or **Wi-Fi** that carries the message.
- Your **voice** is the **data**.

A computer network is the same idea, just with way more cups, way more strings, and super fast messages.

1.2 Why Do We Even Need Networks?

Before networks, if you wanted a file from another computer, you had to copy it to a floppy disk and walk it over. People jokingly called this **"sneakernet"** (because you used your sneakers to walk!). Networks let us:

- **Share files** (documents, photos).
- **Share devices** (one printer for the whole office).

- **Talk** (email, video calls, chat).
- **Reach the internet** (the biggest network of all).

1.3 Types of Networks (by Size)

Name	Stands For	Size	Everyday Example
PAN	Personal Area Network	Tiny (a few feet)	Your phone connected to wireless earbuds
LAN	Local Area Network	One building or home	Your school's computer lab
WLAN	Wireless LAN	One building, no wires	Wi-Fi at a coffee shop
CAN	Campus Area Network	A few nearby buildings	A university campus
MAN	Metropolitan Area Network	A whole city	A city's traffic camera network
WAN	Wide Area Network	Country or worldwide	The **Internet** itself

Story Time: Imagine your house is a **LAN**. Your whole neighborhood connected together is like a **MAN**. All the neighborhoods, cities, and countries connected is the **WAN** (the Internet). Each is just a bigger circle around the last one.

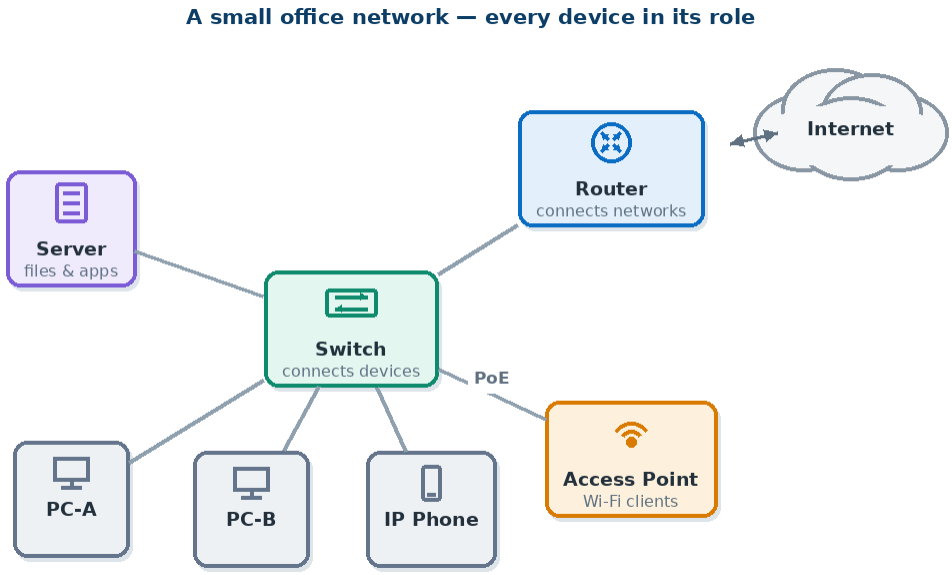
Why do we bother naming networks by size? It's not just trivia — the size changes everything about how you build it:

- **Who owns the wires?** In a **LAN**, you own all the cables and switches, so bandwidth is basically free and huge (1–10 Gbps is normal). In a **WAN**, you cross land you don't own, so you **rent** the connection from a provider (an ISP), it costs money every month, and it's usually much slower than your LAN. That's why the LAN/WAN line matters: one side is "free and fast," the other is "paid and slower."
- **Distance changes the technology.** Short distances can use cheap copper cable. Long distances need fiber or leased provider links. So knowing the scale tells you which media and devices you'll even be allowed to use.
- **Who fixes it?** In a LAN, your team fixes problems. In a WAN, you often have to call the provider. Different scale = different responsibility.

Type	Reaches	Example
PAN (Personal)	A few metres — around one person	Phone ↔ earbuds
LAN (Local)	One building or site	An office, school or home
MAN (Metropolitan)	A city	A university's campuses linked across town
WAN (Wide)	Countries, the globe	The internet itself

Each one **contains** the smaller: your PAN sits inside a LAN, which reaches the world through a WAN.

1.4 The Main Network Devices (Meet the Characters)



You will meet these devices over and over. Let's introduce them like characters in a story.

Device	Job (Simple)	Job (Real)
Hub	The loudspeaker that shouts to everyone	Repeats signals to all ports (dumb, old)
Switch	The smart mail sorter inside a building	Sends data only to the right device using MAC addresses
Router	The GPS that connects different cities	Connects different networks and picks paths using IP addresses
Access Point (AP)	The Wi-Fi broadcaster	Lets wireless devices join the wired network
Firewall	The security guard at the door	Allows or blocks traffic based on rules
Server	The helper that stores/serves things	Provides files, websites, email, etc.
Client	The person asking for things	Your laptop or phone that requests services

Key difference to remember:

- A **switch** connects devices **inside the same network** (a LAN).
- A **router** connects **different networks together** (LAN to LAN, or LAN to the Internet).

But WHY do we even need two different devices? Why not connect the whole world with switches? This is one of the most important "why"s in all of networking, so let's really get it:

A switch **floods broadcasts to everyone** (remember, that's its job). If the entire internet were one giant switched network, then every "who has this address?" broadcast from every device on Earth would be sent to **every other device on Earth**. The whole internet would instantly drown

in broadcast traffic and collapse. It literally could not work.

Routers solve this because **routers do NOT forward broadcasts**. A router breaks the world into separate networks and only passes along the specific traffic that needs to cross between them. So:

- **Switches** keep a local group fast and simple (but can't scale to the world).
- **Routers** create boundaries so networks stay a manageable size, and provide the smart "which way do I send this?" decisions needed to reach far-away networks.

That's the real reason the two devices exist: switches for speed inside a small area, routers for scale and boundaries between areas. You'll see this theme (switch = local, router = between networks) over and over.

Picture two separate LANs. In **LAN A**, a handful of PCs plug into a switch; in **LAN B**, the same. Each switch then has one uplink to a **shared router**.

The switches move traffic **within** their own LAN. The router is the only device joining the two — every packet crossing from A to B passes through it.

> **Coming up:** these devices don't get scattered randomly — section **1.7** shows the standard shapes networks are built in, and section **3.7** explains how many of them get their electricity from the network cable itself.

1.5 Clients, Servers & Peers

- **Client-Server model:** One computer (the **server**) waits and gives out services; other computers (**clients**) ask for them. Example: A web **server** holds YouTube's videos; your phone is the **client** asking to watch.
- **Peer-to-Peer (P2P):** Every computer is both a client AND a server at the same time. Example: Sharing files directly between two laptops.

Why pick one over the other? Each shines in a different situation:

- **Client-Server** gives you **one place to control everything** — one server to secure, back up, and update. That's why businesses use it: if all the company files live on one server, you protect and back up that one machine instead of 500 laptops. The downside is why it can be risky too: if that one server dies, **everyone** loses the service (a "single point of failure").
- **Peer-to-Peer** needs **no central server**, so it's cheap and easy for small setups (two laptops sharing a folder) — that's why home file-sharing and some video-call apps use it. The downside is it's hard to secure and manage once you have many devices, because there's no single place in charge.

The lesson: "should this be client-server or P2P?" is really a question of control and reliability vs. simplicity and cost.

1.6 How Data Moves: Packets

Big messages are **cut into small pieces** called **packets** before traveling. Why? Because small pieces are easier to send, and if one gets lost you only resend that little piece — not the whole thing.

Story Time: Imagine mailing a giant puzzle. Instead of one huge box (heavy, easy to lose everything), you send each puzzle piece in its own envelope. Each envelope has the **address** (where it's going) and a **number** (so the puzzle can be rebuilt in order). If envelope #7 gets lost, you only resend #7.

Let's go deeper — WHY is chopping data into packets so clever? There are three big reasons, and they explain how the whole internet manages to work:

1. **Sharing the road (this is the big one).** If one computer sent its entire huge file as one unbroken stream, it would **hog the wire** and everyone else would have to wait. By cutting files into packets, many computers can **take turns** sending packets on the same wire — their packets interleave, so everyone makes progress at once. It's like taking turns talking instead of one person giving a two-hour speech. This idea is called "packet switching," and it's why the internet can serve billions of people over shared lines.
2. **Cheap error recovery.** If a packet gets damaged or lost, you only re-send that one small packet, not the whole file. Re-sending piece #7 is cheap; re-sending a 4 GB movie because one bit flipped would be awful.
3. **Flexible paths.** Different packets can even take **different routes** to the destination and be reassembled at the end. If one path gets congested or breaks, later packets can go another way. This is why the internet is so resilient — there's no single fragile path.

A big file doesn't travel as one lump. It's chopped into numbered **packets** — P1, P2, P3, P4 — and each one carries four things:

Every packet carries	Why it's needed
TO address	So the network knows where to deliver it
FROM address	So the far end knows who to reply to
ORDER number	So the pieces can be reassembled in sequence
DATA	The actual slice of your file

They travel independently — possibly by different routes — and are rebuilt at the other end.

1.7 How Networks Are Shaped: Design Architectures

We've met the devices. But **how do you arrange them** when you're building a network for a real company? You don't just plug everything into everything — networks follow **standard shapes** (architectures) that engineers reuse because they're predictable, easy to grow, and easy to troubleshoot.

The 3-Tier (Hierarchical) Design

The classic campus design splits the network into **three layers**, each with one job:

Layer	Job	Typical gear
Access	Where **end devices plug in** (PCs, phones, APs). Port security, VLANs, PoE.	Layer 2 switches
Distribution	The **middle manager**: routing between VLANs, ACLs/policy, aggregates access switches.	Layer 3 switches
Core	The **highway**: moves huge amounts of traffic between distribution blocks as fast as possible.	High-speed L3 switches

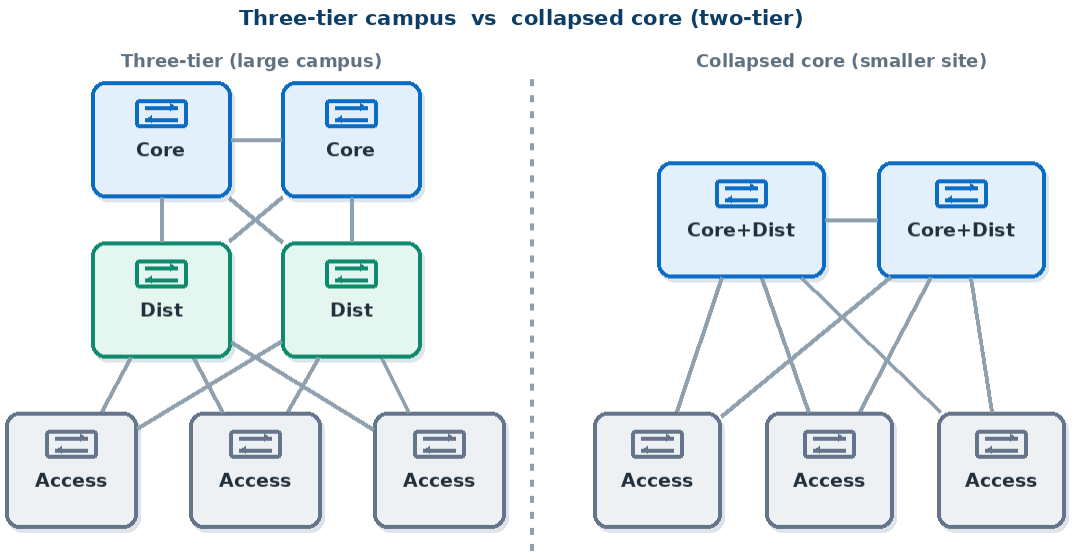
Why split it into layers at all — why not just one big flat switch network? Because of **scale and blast radius**. In a flat network, every switch connects to every other switch in a tangle: adding one more switch means touching many devices, one loop can take down everything, and troubleshooting means checking everywhere. Layers fix this by giving each device **one clear job and one clear place**:

- **Growth becomes copy-paste.** Need a new building? Add an access/distribution "block" and connect it to the core. You don't redesign anything else.
- **Failures stay contained.** A problem at an access switch affects that closet, not the company.
- **Troubleshooting has a map.** "Users on floor 3 can't reach the server" immediately tells you which block to look at.
- **Policy has a home.** Filtering happens at distribution — one predictable place — instead of scattered randomly.

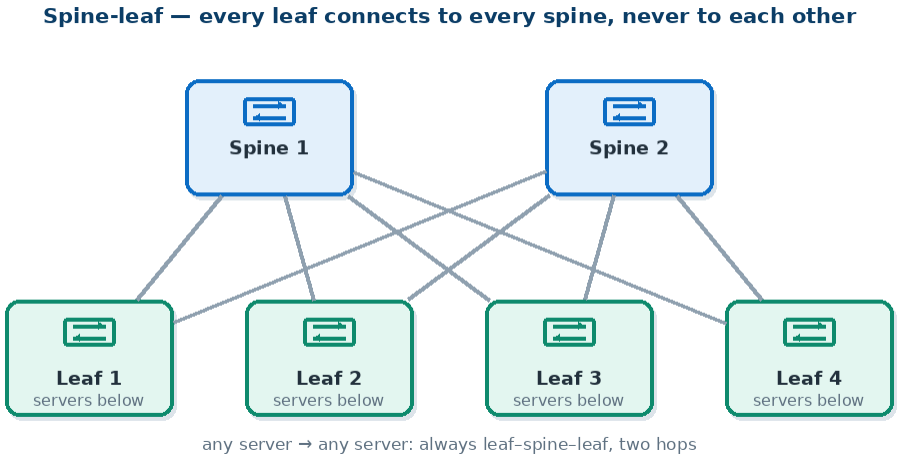
The Collapsed Core (2-Tier) — for smaller networks, the core and distribution layers are **merged into one layer**:

In a collapsed core, the access switches connect straight up into a single combined core/distribution layer — two tiers instead of three.

Why collapse them? Because a separate core only pays off when you have **enough distribution blocks to justify it**. In a single-building company, a dedicated core layer would just be extra hardware forwarding traffic between two switches — expensive, and more devices to manage for no benefit. The rule of thumb: 3-tier for large multi-building campuses, 2-tier (collapsed core) for smaller sites.



Spine-Leaf (The Data Center Shape)



Data centers use a different shape. Every **leaf** connects to **every spine** — and leaves never connect to each other:

The shape is simple to describe: a row of **spine** switches, a row of **leaf** switches, and **every leaf connected to every spine**. Leaves never connect to each other, and spines never connect to each other. Servers hang off the leaves.

Why does the data center need its own shape? Because traffic patterns changed. Old networks were mostly **north-south** (user → server → internet), which the 3-tier design handles well. Modern data centers are mostly **east-west** — servers talking to other servers (a web app querying a database, virtual machines syncing). In a 3-tier design, two servers on different access switches must travel up to distribution or core and back down — and the path length depends on where they happen to sit.

Spine-leaf makes every server **exactly the same distance from every other server**: leaf → spine → leaf. Always two hops. That's called **predictable latency**, and it's the whole point. Need more capacity? Add a spine — every leaf instantly gets more bandwidth. Need more servers? Add

a leaf. Same idea as the hierarchy above — one job per layer — but tuned for server-to-server traffic instead of user-to-internet traffic.

1.8 SOHO, On-Premises & Cloud

Not every network is a campus. The exam expects you to recognize a few common **deployment styles**.

SOHO (Small Office / Home Office) is your house or a small shop. The defining feature: **one box does everything**. Your home "router" is secretly a router plus a switch plus a wireless access point plus a firewall plus a DHCP server plus a NAT device — all in one plastic case.

Why combine them at home but separate them at work? Cost and scale. A home has a handful of devices and no IT staff, so a single cheap all-in-one is perfect. A company has thousands of devices, needs each function to scale independently (more switch ports without buying more firewalls), and needs redundancy — so those functions get split into dedicated, replaceable devices. Same functions, different packaging.

On-premises means the servers live **in your building**, on hardware you bought and maintain. **Cloud** means they live in someone else's data center (AWS, Azure) and you rent them.

Aspect	On-Premises	Cloud
Hardware	You buy and own it	Provider owns it
Cost shape	Big up-front purchase	Pay monthly for what you use
Scaling	Buy and install more (weeks)	Click a button (minutes)
Maintenance	Your team	Provider
Control	Total	Limited to what the provider exposes

Many companies run **hybrid** — some systems on-premises (sensitive data, legacy apps), some in the cloud (websites, backups). Why hybrid? Because the trade-offs above land differently per application, so the sensible answer is rarely "all one way."

1.9 Virtualization: One Machine Pretending to Be Many

Here's something that seems like magic: a **single physical server** can run **twenty separate "computers"** at once, each with its own operating system, IP address, and reboot button.

Virtual Machines & Hypervisors

A **virtual machine (VM)** is a complete pretend computer — pretend CPU, pretend disk, pretend network card — running as software on a real one. The program that creates and manages VMs is the **hypervisor**.

The stack, from the bottom up:

Level	What's there
Physical server	Real CPU, RAM and network card
Hypervisor	Divides that real hardware into virtual slices
VM 1 · VM 2 · VM 3	Each with its **own complete OS** — Windows, Linux, whatever

Why bother? Why not just use the physical server directly? Because a physical server running one application typically uses **5–15% of its power** — the rest is wasted, but you still pay for the box, the rack space, the electricity, and the cooling. Virtualization lets one strong server do the work of many weak ones. And you gain things physical servers can't do: **snapshots** (save a machine's exact state before a risky change and roll back in seconds), **moving a running server** to different hardware without downtime, and **creating a new server in a minute** instead of ordering hardware. It turns servers from things you buy into things you create.

Containers

A **container** is a lighter-weight idea. Instead of virtualizing the whole computer, containers **share the host's operating system** and package just the application plus what it needs to run.

The container stack, bottom up:

Level	What's there
Physical server	Real hardware
One shared operating system	A single OS, used by everything above
Container engine (e.g. Docker)	Packages and isolates each app
App A · App B · App C	The apps — **no OS inside each one**

Compare with the VM stack above: the guest operating systems are simply gone.

Aspect	Virtual Machine	Container
Contains	Full OS + app	Just the app + its dependencies
Size	Gigabytes	Megabytes
Start time	Minutes	Seconds
Isolation	Stronger (separate OS)	Lighter (shared OS kernel)

Why does the shared OS matter so much? Because the operating system is the heavy part. Ten VMs mean ten copies of Windows or Linux booting, patching, and eating RAM. Ten containers share one. That's why containers start in seconds and you can pack far more onto one server — the trade-off being **weaker isolation**, since they all trust the same kernel. VMs when you need strong separation or different operating systems; containers when you want many small, fast, identical app instances.

VRFs — Virtualization Inside a Router

VRF (Virtual Routing and Forwarding) does for a **router** what a hypervisor does for a server: it splits one physical router into **several independent virtual routers**, each with its **own separate routing table**.

Why would you want that? Because sometimes two networks must stay completely separate even though they share hardware. Picture a building shared by two companies, or a hospital keeping medical devices apart from guest Wi-Fi. Without VRFs, all their routes live in one routing table — so if both use `192.168.1.0/24`, they collide, and worse, traffic could leak between them.

With VRFs, each gets its own routing table that **cannot see the other**. They can even use identical, overlapping IP ranges without conflict, because those routes live in different tables entirely.

The neat way to remember it: **VLANs separate at Layer 2, VRFs separate at Layer 3.** A VLAN splits one switch into many broadcast domains; a VRF splits one router into many routing tables. Same instinct — one physical box, several isolated logical ones.

Chapter 2: How Computers Talk — Models & Layers

2.1 Why Do We Need "Models"?

When something is complicated, we break it into **layers** so each layer has one job. This makes it easier to understand, build, and fix.

Story Time: Think of sending a birthday present through the mail:

1. **You** pick the gift (the actual thing you want to send).
2. **You** wrap it in a box.
3. **You** put an address label on it.
4. The **post office** decides the route.
5. The **truck/plane** physically carries it.

Each step is a "layer." The gift doesn't care how the truck works, and the truck doesn't care what's in the box. Networks work the same way!

But WHY is this layering so powerful? Here's the real payoff: because each layer only talks to the layers right above and below it, you can **swap out one layer without touching the others**. This is the whole reason networking can improve over time:

- When Wi-Fi replaced cables, only **Layer 1** (physical) changed. Your web browser (Layer 7), TCP (Layer 4), and IP (Layer 3) didn't need a single change — they don't know or care whether the bits travel over copper, fiber, or radio.

- When websites moved from HTTP to secure HTTPS, only the top layers changed. The routers in the middle kept working exactly the same.

That's the magic of layers: each one hides its messy details from the others, so we can upgrade any piece independently. Imagine if changing your Wi-Fi meant rewriting every website — the internet could never evolve. Layering is what lets thousands of companies build networking pieces that all snap together.

And WHY does this help YOU troubleshoot? Because when something breaks, you can check **one layer at a time** instead of panicking about everything at once. No cable/signal? That's Layer 1. Wrong VLAN? Layer 2. Bad IP or gateway? Layer 3. Can't reach the app but ping works? Layer 4+. The layers become a checklist. (We use exactly this in the Troubleshooting chapter.)

2.2 The OSI Model — 7 Layers

The **OSI model** (Open Systems Interconnection) is a **7-layer** map of how data travels. It's a teaching tool — real networks use TCP/IP (next section) — but the CCNA exam LOVES the OSI model, so learn it well.

Layers are numbered from the **bottom (Layer 1)** to the **top (Layer 7)**:

#	Layer	Its job	You'd recognize
7	Application	What you actually see and use	Web browser, email
6	Presentation	Translate, compress, encrypt	JPEG, TLS
5	Session	Start, keep and end conversations	Holding a call open
4	Transport	Reliable delivery, port numbers	TCP, UDP
3	Network	Addressing and routing between networks	IP, routers
2	Data Link	Local delivery on one link	Switches, Ethernet, MAC
1	Physical	Wires, signals, light, radio	Cables, fiber, Wi-Fi

A Trick to Remember the 7 Layers

From Layer 7 down to 1: **"All People Seem To Need Data Processing"**

- **A**pplication
- **P**resentation
- **S**ession
- **T**ransport
- **N**etwork

- **D**ata Link
- **P**hysical

From Layer 1 up to 7: **"Please Do Not Throw Sausage Pizza Away"**

- **P**hysical
- **D**ata Link
- **N**etwork
- **T**ransport
- **S**ession
- **P**resentation
- **A**pplication

What Each Layer Does (Deep but Simple)

Layer 1 — Physical: The actual **wires, radio waves, and electricity/light**. It turns 1s and 0s into signals you can send (a flash of light, a pulse of electricity, a radio wave). No thinking here — just moving bits. Examples: copper cables, fiber optic, connectors, voltage.

Layer 2 — Data Link: Delivers data **between two devices on the same local network**. It uses **MAC addresses** (a device's built-in hardware ID). This layer packs data into **frames**. Switches live here. It also checks for errors in each frame.

Layer 3 — Network: Delivers data **across different networks** using **IP addresses**. This is where **routers** decide the best path. Data here is called a **packet**.

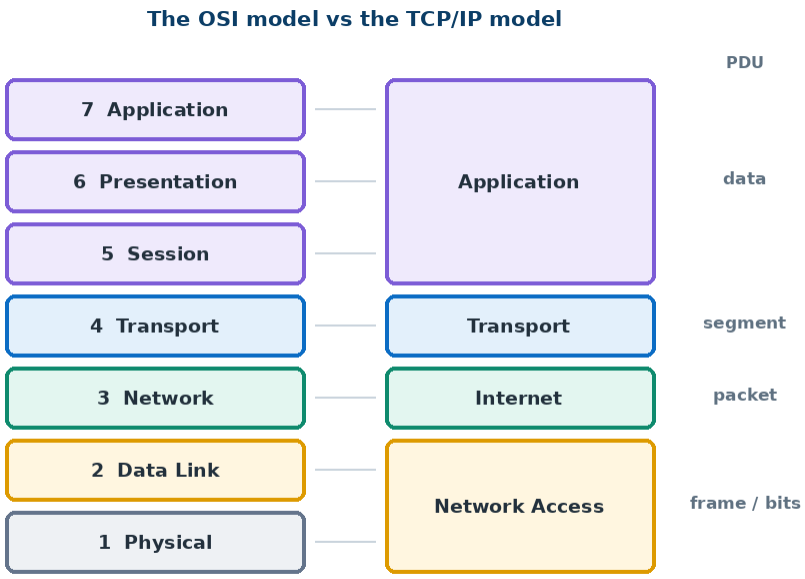
Layer 4 — Transport: Makes sure data arrives **correctly and in order**. It uses **TCP** (careful and reliable) or **UDP** (fast but not guaranteed). It also uses **port numbers** to know which app the data belongs to. Data here is called a **segment** (TCP) or **datagram** (UDP).

Layer 5 — Session: Opens, manages, and closes the **conversation** between two apps. Think of it as keeping the phone line open while you talk.

Layer 6 — Presentation: Translates data into a format both sides understand. Handles **encryption** (scrambling for safety) and **compression** (shrinking to save space). Example: turning a photo into JPEG.

Layer 7 — Application: The layer **closest to you**, the human. It's the services apps use: web (HTTP/HTTPS), email (SMTP), file transfer (FTP). Note: this is NOT the app itself (like Chrome) — it's the networking rules the app uses.

2.3 The TCP/IP Model — The Real-World Version

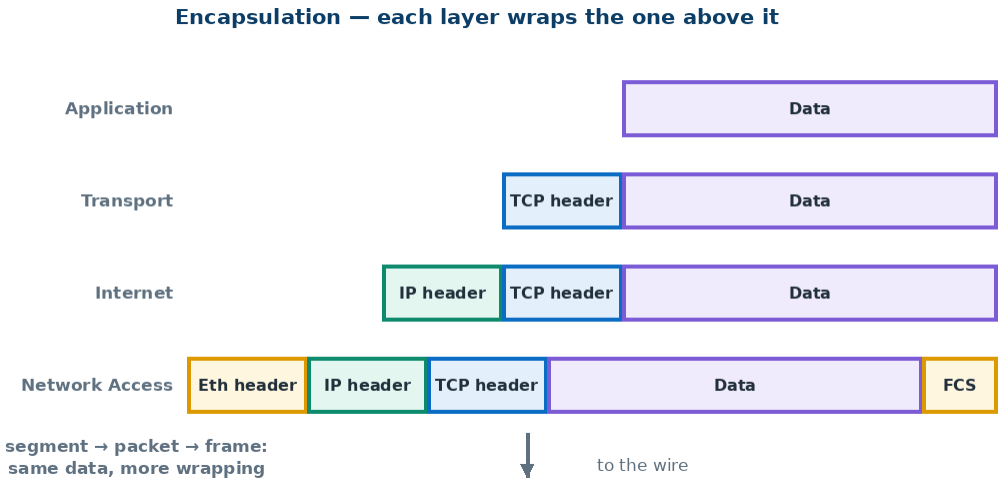


The **TCP/IP model** is what the Internet actually uses. It has **4 layers** (sometimes shown as 5). It's basically the OSI model squished together.

TCP/IP layer	Covers which OSI layers
Application	7 Application + 6 Presentation + 5 Session
Transport	4 Transport
Internet	3 Network
Network Access	2 Data Link + 1 Physical

TCP/IP simply **merges** layers OSI keeps separate — the top three become one, and the bottom two become one.

2.4 Encapsulation — Wrapping Data in Layers



As data goes **down** the layers (from your app to the wire), each layer **adds its own wrapper** (called a **header**). This is called **encapsulation**. It's like putting a letter inside envelope inside a box inside a shipping container.

Going down the stack	What gets added	The result is called
Layers 7–5	—	**Data**
Layer 4	TCP header	**Segment**
Layer 3	IP header	**Packet**
Layer 2	MAC header **and** a trailer	**Frame**
Layer 1	— (turned into signals)	**Bits**

Each layer wraps what the one above handed it — and the receiving device unwraps them in reverse.

At the other end, the receiving computer does the **opposite** — it peels off each wrapper. This is called **de-encapsulation**.

Why wrap data in all these layers? Why not just send the raw data? Because each header carries the exact information one specific device needs to do its job — and nothing more:

- The **switch** in the middle only reads the **Layer 2 (MAC) header** to decide the next local hop. It doesn't need to understand your data — just the "local address."
- The **router** only reads the **Layer 3 (IP) header** to pick the path across networks. It ignores your actual data too.
- The **receiving computer** reads the **Layer 4 (port) header** to know which app gets the data (web browser? email?).

So each wrapper is like an **address label for a different worker** along the journey. The switch reads its label, the router reads its label, the destination reads its label. That's why we add headers in order going down, and strip them in reverse going up — each device unwraps only as far as it needs to, does its job, and passes it on. It's an assembly line where every station reads only its own instructions.

A key detail (exam favorite): as a frame passes through switches and routers, the **MAC addresses (Layer 2) change at every hop**, but the **IP addresses (Layer 3) stay the same** from start to finish. Why? Because MAC = "the next local step" (changes each hop, like passing a note hand-to-hand across a room), while IP = "the final destination" (never changes, like the address written on the envelope). Hold onto this — it clicks fully once you reach routing in Chapter 14.

The Names of Data at Each Layer (PDUs)

PDU means **Protocol Data Unit** — the fancy name for "the chunk of data" at each layer. Memorize these:

Layer	Data is called
Transport (4)	**Segment** (TCP) / **Datagram** (UDP)
Network (3)	**Packet**
Data Link (2)	**Frame**
Physical (1)	**Bits**

Memory trick: "**Some People Fear Birthdays**" → **S**egment, **P**acket, **F**rame, **B**its (from Layer 4 down to 1).

2.5 TCP vs. UDP — The Two Delivery Styles

This is SUPER important for the exam. Both live at Layer 4 (Transport).

TCP (Transmission Control Protocol) — The Careful Delivery

TCP is like sending a package with **signature required** and **tracking**. It makes sure everything arrives, in order, with nothing missing.

- **Reliable:** Confirms every piece arrived.
- **Ordered:** Puts pieces back in the right order.
- **Connection-based:** Sets up a connection first (the "3-way handshake").
- **Slower** because of all the checking.
- **Used for:** Websites (HTTP/HTTPS), email, file downloads — anything where every bit matters.

The 3-Way Handshake (how TCP starts a conversation):

The three-way handshake, in order:

1. **A → B: SYN** — "Hi, can we talk?"
2. **B → A: SYN-ACK** — "Sure. Can you hear me?"
3. **A → B: ACK** — "Yes. Let's go."

Only after that third message does either side send real data.

Remember: SYN → SYN-ACK → ACK. Like knocking, getting a "who is it?", then answering.

UDP (User Datagram Protocol) — The Fast Delivery

UDP is like **shouting a message** — fast, but you don't check if the person heard every word.

- **Not reliable:** No confirmation.
- **No ordering:** Pieces might arrive out of order.
- **Connectionless:** Just sends, no handshake.
- **Faster** because no checking.
- **Used for:** Video calls, live streaming, online games, DNS — anything where **speed matters more** than perfect delivery. (If one frame of video is lost, who cares? You want it fast!)

TCP vs UDP Side by Side

Feature	TCP	UDP
Reliable?	Yes	No
In order?	Yes	No
Handshake?	Yes (SYN/SYN-ACK/ACK)	No
Speed	Slower	Faster
Good for	Web, email, files	Video, voice, games, DNS

2.6 Port Numbers — Apartment Numbers for Apps

An IP address gets you to the right **building** (computer). A **port number** gets you to the right **apartment** (app/service) inside it.

Story Time: Your friend lives in a big apartment building. The **street address** (IP) gets the mail to the building. The **apartment number** (port) makes sure it reaches your friend and not the neighbor.

Ports you MUST memorize for the exam:

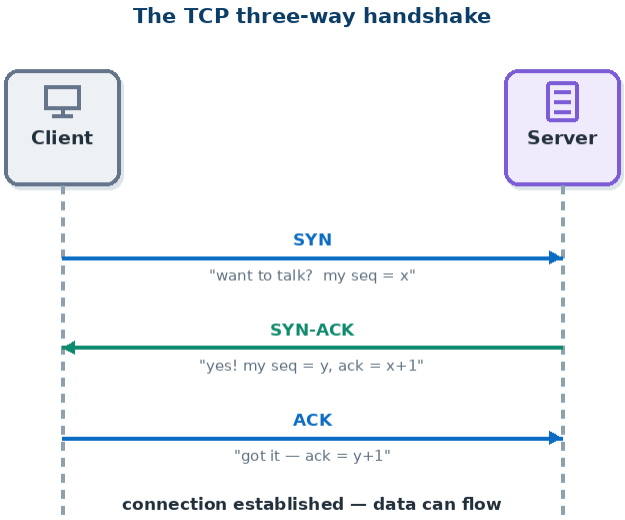
Port	Protocol	What It Does	TCP/UDP
20, 21	FTP	Transfer files	TCP
22	SSH	Secure remote login	TCP
23	Telnet	Remote login (NOT secure)	TCP
25	SMTP	Send email	TCP
53	DNS	Turn names into IP addresses	UDP (and TCP)
67, 68	DHCP	Hand out IP addresses	UDP
69	TFTP	Simple file transfer	UDP
80	HTTP	Web pages (not secure)	TCP
110	POP3	Receive email	TCP

Port	Protocol	What It Does	TCP/UDP
143	IMAP	Receive email (better)	TCP
161, 162	SNMP	Manage network devices	UDP
443	HTTPS	Secure web pages	TCP
514	Syslog	Send log messages	UDP

Memory trick for secure vs not: HTTP is **80**, add secure "S" and it becomes HTTPS **443**. SSH (**22**) is the secure version of Telnet (**23**) — they're next-door neighbors, but only 22 locks the door.

> **Ports also let the network treat traffic differently.** Recognizing "this is voice, that is a file download" is the first step of **QoS**, which decides who goes first when a link is full — see section **17.7**.

2.7 TCP In Depth — How Reliability Actually Works



> **Depth section.** The exam asks what TCP does; this explains how. Skip it if you're on a tight sprint — but this is the layer of understanding CCNP assumes you already have.

Section 2.5 said TCP is "reliable." That word hides four separate mechanisms, and knowing which one is doing what is the difference between memorizing and understanding.

What's Actually in the Header

Field	Size	What it's for
Source / Destination Port	16 bits each	Which application at each end
Sequence Number	32 bits	Where this data sits in the byte stream
Acknowledgment Number	32 bits	The next byte expected from the other side
Data Offset	4 bits	Where the header stops and data starts
Flags — `URG ACK PSH RST SYN FIN`	6 bits	Connection control (SYN opens, FIN closes, RST aborts)
Window Size	16 bits	How much more the receiver can accept right now
Checksum	16 bits	Corruption check
Options	variable	MSS, window scale, selective ACK

Three fields do the heavy lifting — and the rest of this section is really just those three at work: **Sequence Number**, **Acknowledgment Number**, and **Window Size**.

Sequence & Acknowledgment — Counting Bytes, Not Packets

This is the detail most people get wrong: TCP sequence numbers count **bytes**, not segments.

Sender has 3000 bytes to send, MSS = 1000:

Segment sent	Bytes it carries
`Seq=1`	1–1000
`Seq=1001`	1001–2000
`Seq=2001`	2001–3000

The receiver replies with a single `Ack=3001` — "I have everything up to byte 3000; send me byte 3001 next."

Why count bytes instead of packets? Because it makes the acknowledgment self-describing. `Ack=3001` means "every byte below 3001 arrived" — one number confirms an arbitrary amount of data, and it stays correct even if the network splits or re-sizes segments along the way. Note the ack is the **next byte expected**, not the last byte received: it's a request, not a receipt.

What if a middle segment is lost?

Segment sent	Bytes	What happened
`Seq=1`	1–1000	arrives
`Seq=1001`	1001–2000	**LOST**
`Seq=2001`	2001–3000	arrives

The receiver now answers `Ack=1001`, then `Ack=1001` again, then again — the same number three times, because that's the truth: it still needs byte 1001.

The receiver keeps saying `Ack=1001` because that's the truth — it can't acknowledge past the hole. Three **duplicate ACKs** tell the sender "one specific segment is missing, but later ones are arriving, so the path is alive." The sender resends just that segment — **fast retransmit** — without waiting for a timeout. That's why the ack is "next expected" rather than "last received": the repetition itself becomes the signal.

Flow Control vs Congestion Control — Two Different Problems

These get confused constantly, and the exam likes the distinction.

Aspect	Flow control	Congestion control
Protects	The **receiver**	The **network**
Question	"Can you keep up?"	"Can the path keep up?"
Mechanism	Window Size field	Sender's congestion window
Signal	Receiver advertises a window	Packet loss / duplicate ACKs

Flow control is the receiver saying how much buffer it has left: `Window=64240` means "send up to 64,240 unacknowledged bytes." A slow receiver shrinks it; a full one advertises **zero** and the sender pauses. This stops a fast server from drowning a slow phone.

Congestion control is the sender's own guess about the network, which nobody advertises. It starts small (**slow start**, doubling each round trip), grows until loss appears, then backs off sharply and climbs gently. **Why treat loss as the congestion signal?** Because a full router queue drops packets — so loss is the only feedback the network gives for free. Flow control is told to you; congestion control has to be inferred.

The actual send rate is the **smaller** of the two windows. Either can be the bottleneck.

MSS, MTU and Why 1460 Shows Up Everywhere

MTU is the largest frame payload a link carries — **1500 bytes** on Ethernet. **MSS** is the largest TCP data chunk, negotiated in the handshake:

```
1500 (MTU) − 20 (IP header) − 20 (TCP header) = 1460 (MSS)
```

Why negotiate this instead of just sending and letting routers fragment? Because fragmentation is expensive and fragile: the destination must hold fragments and reassemble, and losing **one** fragment destroys the whole original packet — turning a 1% loss rate into something far worse. Agreeing on a size that fits end to end avoids the problem entirely. This is also why a tunnel (GRE, IPsec, PPPoE) causes mysterious breakage: its extra headers shrink the usable MTU, and traffic that ignores the new limit gets dropped rather than delivered.

Closing Down

Opening takes three messages; closing normally takes **four**, because each direction closes independently:

1. **A → B: FIN** — "I'm done sending."
2. **B → A: ACK**
3. **B → A: FIN** — "So am I."
4. **A → B: ACK**

Why can't both sides just hang up at once? Because TCP is **full duplex** — two independent byte streams. One side finishing its upload doesn't mean the other finished its download, so a **half-close** is legitimate: the server can keep sending after the client says FIN. (A **RST** is the abrupt alternative — "this connection is invalid, stop now" — which is what you get connecting to a closed port.)

Chapter 3: The Physical Stuff — Cables, Signals & Ports

3.1 Why Physical Matters

Everything digital eventually rides on something physical: a **copper wire**, a **glass fiber**, or a **radio wave**. If the cable is broken, nothing else matters. This is **Layer 1**.

3.2 Copper Cables (Ethernet / Twisted Pair)

The most common LAN cable is **twisted-pair copper**, ending in an **RJ-45** connector (looks like a fat phone plug). Inside are **8 tiny wires**, twisted into **4 pairs**.

An **RJ-45** connector has **8 pins**, numbered 1–8 across the front, with a small plastic clip that holds it in the socket. Inside the cable are **4 twisted pairs** — 8 wires total, twisted in pairs to cancel out interference.

Why twisted? Twisting the wires cancels out electrical noise (interference), so the signal stays clean. Cool trick!

Categories of Copper Cable

Category	Max Speed	Common Use
Cat 5	100 Mbps	Old networks
Cat 5e	1 Gbps	Common homes
Cat 6	1–10 Gbps (short)	Modern offices
Cat 6a	10 Gbps	High-speed networks
Cat 7 / 8	10–40 Gbps	Data centers

> **Mbps** = Megabits per second. **Gbps** = Gigabits per second (1000 Mbps). Bigger = faster.

Straight-Through vs. Crossover Cables

This is a classic CCNA topic, and once you understand the **"why"** behind it, you'll never forget it. Let's build it up slowly.

First, the Big Secret: Devices Talk on Different Wires

Inside that Ethernet cable, some wires are used to **transmit** (send) and others to **receive** (listen). On older 10/100 Mbps Ethernet, only 2 of the 4 pairs are used:

- **Pins 1 and 2** = **Transmit** (Tx) — the "mouth"
- **Pins 3 and 6** = **Receive** (Rx) — the "ears"

Here's the key rule: For two devices to talk, one device's **mouth (Tx)** must connect to the other device's **ears (Rx)**. If both devices talk on the same wires and listen on the same wires, it's like two people both shouting into pins 1-2 and both listening on pins 3-6 — nobody hears anybody!

Devices **transmit** on one pair and **listen** on another:

Device	Sends on	Listens on
PC / router	pins **1, 2**	pins **3, 6**
Switch	pins **3, 6**	pins **1, 2**

That opposition is the whole trick. A PC's transmit pins line up with a switch's listen pins, so a plain straight-through cable works. Connect two of the same kind of device, though, and transmit meets transmit — both talking, neither listening — which is exactly what a crossover cable fixes.

Why "Same" vs "Different" Devices Matters

Different device types are wired **oppositely** on purpose, so they naturally line up:

- A **PC** transmits on pins 1-2 and receives on 3-6.
- A **switch** does the **opposite**: it receives on 1-2 and transmits on 3-6.

So when you connect a PC to a switch with a **straight-through cable** (pin 1→1, 2→2, 3→3, 6→6), the PC's Tx (1,2) lands right on the switch's Rx (1,2). They line up perfectly with no crossing needed!

But when you connect **two of the same** device (two PCs, or two switches), they both transmit on the same pins. A straight-through cable would connect Tx→Tx and Rx→Rx (the "BAD" picture above). So you need a **crossover cable** that physically swaps the pairs (1→3, 2→6) to make Tx meet Rx.

The Two Cable Types

- **Straight-Through cable:** Each wire goes to the **same pin** on both ends (pin 1 → pin 1, pin 2 → pin 2, and so on). Used to connect **DIFFERENT** kinds of devices, because they're already wired oppositely.
- PC ↔ Switch
- Router ↔ Switch
- Wireless Access Point ↔ Switch
- Switch ↔ Hub
- **Crossover cable:** The transmit and receive pairs are **swapped** inside the cable (pin 1 → pin 3, pin 2 → pin 6). Used to connect **SIMILAR** kinds of devices, because they'd otherwise both talk on the same wires.
- Switch ↔ Switch
- PC ↔ PC
- Router ↔ Router
- PC ↔ Router (both are "smart" end-devices — routers and PCs are wired the same way, so they count as "similar")
- Switch ↔ Hub (sometimes; both are "network" devices)

> **Wait — why is PC ↔ Router a crossover if they're different-looking devices?** Because what matters is how they're **wired**, not what they look like. Both PCs and routers are "end devices" that transmit on pins 1-2. Switches and hubs are "network devices" that transmit on pins 3-6. **Same category = crossover; different category = straight-through.**

A Simple Way to Remember Which Cable

Sort every device into one of two teams by how it's wired:

Team A — sends on 1, 2	Team B — sends on 3, 6
PC / Laptop	Switch
Router	Hub
Server	Wireless AP (acts like a switch port)
Firewall	—

- **Same team** (A-A or B-B) → use a **CROSSOVER** cable.

- **Different teams** (A-B) → use a **STRAIGHT-THROUGH** cable.

Quick check: Switch ↔ Switch = both Team B = **crossover**. PC ↔ Switch = Team A + Team B = **straight-through**. Router ↔ PC = both Team A = **crossover**. Easy!

The Actual Wire Pinout (T568B)

Ethernet cables follow a color-order standard. The most common is **T568B**. A straight-through cable uses **T568B on both ends**. A crossover uses **T568B on one end and T568A on the other** (which swaps the green and orange pairs = swaps Tx and Rx).

Pin	Straight-through (T568B)	Crossover (T568A far end)
1	White/Orange	White/Green — crosses to 3
2	Orange	Green — crosses to 6
3	White/Green	White/Orange — crosses to 1
4	Blue	Blue
5	White/Blue	White/Blue
6	Green	Orange — crosses to 2
7	White/Brown	White/Brown
8	Brown	Brown

So the pin-to-pin connections come out as:

Pin	Straight-through	Crossover
Pin 1 →	1	**3**
Pin 2 →	2	**6**
Pin 3 →	3	**1**
Pin 6 →	6	**2**

Straight-through keeps every pin in place; crossover swaps the transmit and receive pairs.

A Third Cable: The Rollover (Console) Cable

There's one more cable you must know for the exam: the **rollover cable** (also called a **console cable**). It is **NOT** for network traffic at all — it's for **configuring** a Cisco device directly from your laptop's console.

- The wires are **completely reversed** end-to-end: pin 1 → pin 8, pin 2 → pin 7, and so on (it "rolls over").
- It's usually **light blue** (Cisco's classic color).
- One end plugs into the device's **console port**; the other connects to your laptop (often via a USB-to-serial adapter).

A **rollover** cable reverses the order completely: pin **1 ↔ 8**, **2 ↔ 7**, **3 ↔ 6**, **4 ↔ 5**. It carries no Ethernet at all — it's for console access to a device's command line.

Three cables, three jobs — memorize this table:

Cable	Wiring	Used For
Straight-through	Same pins both ends	Connecting **different** devices (PC-Switch, Router-Switch)
Crossover	Tx/Rx pairs swapped	Connecting **similar** devices (Switch-Switch, PC-PC, PC-Router)
Rollover	All pins reversed	**Console** access to configure a device

Gigabit Ethernet: All Four Pairs

On **Gigabit** Ethernet (1000 Mbps) and faster, all **4 pairs** are used, and each pair can transmit AND receive at the same time. The straight-through vs. crossover rules still apply for the exam, but modern gear almost always uses **Auto-MDIX** to sort it out automatically (see below).

Memory trick: Same devices = **cross** them. Different devices = keep them **straight**. (Think: two friends who are alike need to "cross" paths to meet; different friends walk straight to each other.)

> **Good news:** Most modern devices have **Auto-MDIX** (Automatic Medium-Dependent Interface Crossover), a feature that automatically detects whether it needs a crossover and **fixes it electronically** — so either cable type works. Because of this, you rarely need a real crossover cable today. **But the CCNA exam still tests the classic rules**, and Auto-MDIX only works when the port's speed/duplex is set to auto-negotiate, so know both the rules AND that Auto-MDIX exists.

3.3 Fiber Optic Cables — Light Instead of Electricity

Fiber uses **pulses of light** through thin glass strands instead of electricity through copper. Light is super fast and can travel very far without weakening.

Type	Core Size	Light Source	Distance	Use
Multimode (MMF)	Bigger core	LED / cheap laser	Shorter (up to ~550m)	Inside buildings
Single-mode (SMF)	Tiny core	Precise laser	Very long (many km)	Between cities

Copper carries **electricity**, so it can pick up electrical noise from nearby motors, lights and cables. **Fiber** carries **light**, which is immune to electrical interference — one reason it runs faster and very much further.

Why choose fiber?

- Immune to electrical interference.
- Much longer distances.
- Higher speeds.
- More secure (harder to tap).
- Downsides: more expensive and more fragile than copper.

3.4 Wireless (Radio Waves)

Wi-Fi uses **radio waves** (invisible signals through the air) instead of cables. We cover this in depth in Chapter 20, but for now: no wire, more freedom, but more interference and usually a bit slower/less reliable than a good cable.

3.5 Network Speeds & Duplex

Duplex = how data flows in two directions.

- **Half-duplex:** Can send OR receive, but not at the same time (like a walkie-talkie — you say "over" and take turns). Old hubs used this.
- **Full-duplex:** Can send AND receive at the same time (like a phone call — both people talk at once). Modern switches use this.

Mode	How it works	Result
Half-duplex	Send **or** receive — one at a time, taking turns	Collisions possible
Full-duplex	Send **and** receive simultaneously	No collisions; the modern default

Collision: In half-duplex, if two devices talk at once, their signals crash — that's a **collision**. Full-duplex has no collisions.

3.6 Common Ports & Interfaces on Devices

- **Ethernet ports (RJ-45):** For copper cables — the most common.
- **SFP/SFP+ slots:** Small slots where you plug fiber or copper modules — flexible!
- **Console port:** A special port used to **configure** a Cisco device directly with a laptop (usually rollover cable or USB). This is your first way to talk to a brand-new switch.
- **USB ports:** For files, or console access on newer gear.

3.7 Power over Ethernet (PoE) — Data and Electricity on One Cable

Here's a genuinely useful trick: an Ethernet cable can carry **electrical power** alongside the data. That's **Power over Ethernet (PoE)**, and it's why the phone on an office desk and the access point on the ceiling have **only one cable** going to them — no power brick, no wall outlet.

One cable carries **both** the data and the electricity. The switch feeding the power and the device drawing it have names worth knowing:

Term	Stands for	Which is
PSE	Power Sourcing Equipment	The switch (or injector) supplying power
PD	Powered Device	The AP, phone or camera drawing it

Why is this such a big deal? Think about where these devices live: an access point on a **ceiling**, a security camera on an **outside wall**, a phone in the middle of an open-plan floor. Running an electrical outlet to each of those spots means an electrician, conduit, and real money — often more than the device costs. PoE means the network cable you were already going to run delivers power too. And there's a bonus: because the power comes from the switch in the wiring closet, plugging that **switch** into a UPS keeps every phone and camera alive during a power cut — one battery protects them all.

The Standards (Know the Power Levels)

Standard	Common name	Power at the switch port	Typically powers
802.3af	PoE	~15.4 W	IP phones, basic APs
802.3at	PoE+	~30 W	Modern APs, PTZ cameras
802.3bt	PoE++ / UPoE	~60–100 W	Video screens, laptops, lighting

Note that the **device receives slightly less** than the port supplies — some power is lost as heat in the cable run. That's why an 802.3af port is listed as 15.4 W at the switch but only about 12.95 W at the device.

Does PoE risk frying a device that isn't built for it? No — and the reason is worth knowing. Before sending real power, the switch performs a **detection** step: it applies a small voltage and looks for a specific electrical signature that powered devices are built to present. No signature, no power — so your laptop plugged into a PoE port receives **only data**, exactly as normal. After detection, the switch **classifies** the device to learn how much power it actually wants, and grants only that. Negotiate first, then deliver — the same "check before you commit" pattern you'll see all over networking.

The gotcha to watch for: a switch has a **total power budget** shared by all its ports. A switch advertising 370 W of PoE cannot run 48 ports at 30 W each (that would need 1,440 W). Fill a switch with hungry devices and later ports simply get **denied power** — they'll look dead while their link light works fine. Check with:

```
SW# show power inline                  ! per-port draw + remaining budget
SW# show power inline gi1/0/5          ! detail for one port
```

Chapter 4: Numbers Computers Use — Binary & Hex

4.1 Computers Only Understand On and Off

Deep down, a computer is millions of tiny switches. Each switch is either **ON** or **OFF**. We write:

- **ON = 1**
- **OFF = 0**

A single 1 or 0 is called a **bit** (short for **b**inary dig**it**). It's the smallest piece of computer info.

"But WHY do I, a future network engineer, have to learn binary? Can't the computer do the math?" Great question — here's the honest reason: **subnetting.** When you split networks into smaller pieces (Chapter 12, one of the most-tested CCNA skills), the split happens in the middle of the binary numbers, not on the neat decimal boundaries you see. An IP address like `192.168.1.130` looks like tidy decimal numbers, but the switch and router see it as **32 on/off switches**, and the network "line" can fall between switch #25 and #26 — a spot that's invisible in decimal but obvious in binary. **If you can't see the binary underneath, subnetting feels like random magic. Once you can, it becomes simple counting.** That's why we start here: binary is the secret decoder ring for the whole addressing half of the exam.

4.2 Bits and Bytes

- **1 bit** = one 1 or 0.
- **8 bits** = **1 byte**.

Networks and computers group bits into bytes all the time. An IP address like `192.168.1.1` is made of **4 bytes** (32 bits total). More on that soon!

One **byte** is **8 bits**, and each position is worth double the one to its right:

Position	Worth	Bit
1st (leftmost)	128	**1**
2nd	64	0
3rd	32	**1**
4th	16	**1**
5th	8	0
6th	4	0
7th	2	**1**
8th (rightmost)	1	0

Add up the positions holding a 1: 128 + 32 + 16 + 2 = **178**.

4.3 How Binary Counting Works (The Place Values)

In our normal (decimal) numbers, each spot is worth 10× more as you go left: 1, 10, 100, 1000...

In **binary**, each spot is worth **2×** more as you go left. For one byte (8 bits), the spots are worth:

Position	1st	2nd	3rd	4th	5th	6th	7th	8th
Worth	128	64	32	16	8	4	2	1

To find a byte's value, **add up the spots that have a 1**.

Example 1: Convert binary `11000000` to decimal

Place	Bit
128	**1**
64	**1**
32	0
16	0
8	0
4	0
2	0
1	0

Only the 128 and 64 positions hold a 1, so: 128 + 64 = **192**

So `11000000` = **192**. (This is the first byte of `192.168.x.x`!)

Example 2: Convert binary `10101000` to decimal

Place	Bit
128	**1**
64	0
32	**1**
16	0
8	**1**
4	0
2	0
1	0

128 + 32 + 8 = **168**

So `10101000` = **168**.

Example 3: Convert decimal `200` to binary

Ask: "Does 128 fit into 200?" Yes → write 1, subtract → 200-128 = 72. "Does 64 fit into 72?" Yes → write 1, subtract → 72-64 = 8. "Does 32 fit into 8?" No → 0. "16?" No → 0. "8?" Yes → 1, subtract → 0. The rest are 0.

Work down from the biggest place value that fits:

Place	Fits in what's left?	Bit
128	yes (200 − 128 = 72)	**1**
64	yes (72 − 64 = 8)	**1**
32	no	0
16	no	0
8	yes (8 − 8 = 0)	**1**
4	no	0
2	no	0
1	no	0

So 200 = **11001000**

So `200` = **11001000**.

> **Try It:** Convert `11111111` to decimal. (Add all spots: 128+64+32+16+8+4+2+1 = **255**.) This is the biggest one byte can hold! That's why IP numbers only go up to 255.

4.4 Hexadecimal (Hex) — Shortcut for Big Binary

Binary is long and easy to mess up. **Hexadecimal** ("hex") is a shortcut. Hex uses **16 symbols**: 0-9 and then A-F.

Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9

Decimal	Binary	Hex
10	1010	**A**
11	1011	**B**
12	1100	**C**
13	1101	**D**
14	1110	**E**
15	1111	**F**

The magic: 4 bits = 1 hex digit. So one byte (8 bits) = exactly **2 hex digits**. This makes hex perfect for shortening long binary.

Example: Binary `11110000` to Hex

Split into two groups of 4: `1111` and `0000`.

- `1111` = 15 = **F**
- `0000` = 0 = **0**

So `11110000` = **F0** in hex. Way shorter!

Where you'll see hex: MAC addresses (like `00:1A:2B:3C:4D:5E`) and IPv6 addresses are written in hex. Now you know why!

But WHY use hex instead of just showing the binary or decimal? Because hex is the "Goldilocks" choice — just right for humans reading computer numbers:

- **Binary is too long and error-prone.** A MAC address in binary is 48 ones and zeros in a row: `000000000000110100010101 1...`. Miscount one digit and you're wrong. No human can read that reliably.
- **Decimal doesn't line up with bits.** Decimal digits don't map cleanly to groups of bits, so converting is annoying and hides the pattern.
- **Hex fits perfectly.** Because **4 bits = exactly 1 hex digit**, hex is a clean, lossless shorthand for binary. You can glance at a hex digit and instantly know its 4 bits, and vice-versa. It's short like decimal but lines up with the bits like binary. That's the whole reason MAC and IPv6 addresses (which are big piles of bits) are written in hex — it keeps them **short AND bit-accurate**.

Chapter 5: MAC Addresses & Ethernet

5.1 What Is a MAC Address?

A **MAC address** (Media Access Control) is a **permanent ID number** burned into every network device at the factory. Think of it as the device's **fingerprint** — it (usually) never changes.

- It's **48 bits** long, written as **12 hex digits**.

- Usually shown in pairs: `00:1A:2B:3C:4D:5E` (or with dashes, or Cisco style `001a.2b3c.4d5e`).

A MAC address splits into two halves:

Half	Example	What it means
First 24 bits	`00:1A:2B`	The **OUI** — which manufacturer made it
Last 24 bits	`3C:4D:5E`	A serial the maker assigns to that individual card

- The **first half (OUI)** tells you the **manufacturer** (Cisco, Apple, Intel...).
- The **second half** is a unique serial the manufacturer assigns.

MAC vs IP — the big difference:

- **MAC address** = permanent, physical, works only on the **local** network (Layer 2). Like your **name**.
- **IP address** = changeable, logical, works **across** networks (Layer 3). Like your **home address** (changes if you move).

The question everyone asks: "WHY do we need BOTH? Isn't one address enough?" This is one of the most important ideas in all of networking, so let's nail it with the name-vs-address analogy:

- Your **name** never changes and identifies you personally — but a name is **useless for delivering mail across the country**. You can't write "deliver this to Sarah" on a package and expect the postal system to find her among millions of Sarahs. A name doesn't tell anyone where you are.
- Your **home address** does tell everyone where to deliver — but it **changes when you move**, and it describes a location, not a person.

MAC and IP work exactly the same way, and we need both for two different jobs:

- **IP address = the location.** It's structured so routers can figure out the general direction to send data ("this address is in the 192.168.1 neighborhood, send it that way"). Routing across the world only works because IP addresses describe where a device is on the network. But an IP can change (connect to a different Wi-Fi, you get a new IP).
- **MAC address = the specific device, locally.** Once data finally arrives at the right local network, the MAC address identifies exactly which machine on that network gets it. It's permanent and unique so there's no confusion about who on the local wire.

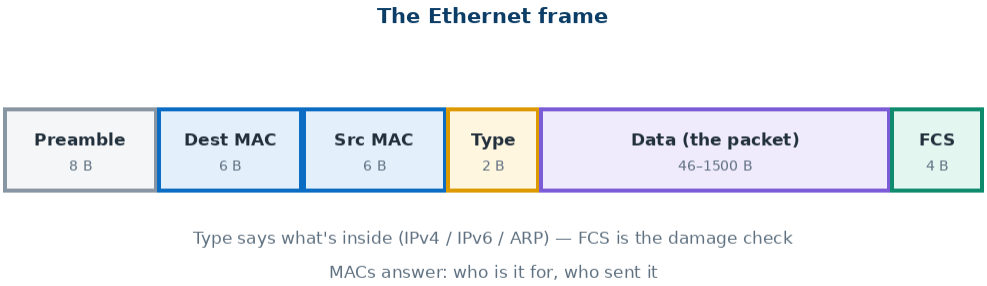
The killer reason you can't drop either one: Imagine trying to route with MACs only. MAC addresses have **no structure** — they're random factory serial numbers. There's no "neighborhood" in a MAC, so a router would need a list of every device on Earth to know where to send things. Impossible. IP's structured, location-based design is what makes worldwide routing possible. But IP alone can't pin down the final device on the local wire — that's the MAC's job.

So they're a team: IP gets your data to the right neighborhood (across many networks, hop by hop), and MAC delivers it to the exact house (on the final local network). You'll see in Chapter 14 that as data travels, the **IP stays the same the whole trip** (the final destination), while the **MAC changes at every hop** (the next local handoff). Two addresses, two jobs, working together.

5.2 What Is Ethernet?

Ethernet is the most common set of rules (a **protocol**) for wired LANs. It defines how devices format data into **frames** and share the wire. Almost every wired network you touch uses Ethernet.

5.3 The Ethernet Frame (What a Frame Looks Like)



When data travels on a LAN, it's wrapped in an **Ethernet frame**. Here's the layout:

Field	Size	What it's for
Destination MAC	6 bytes	Who the frame is **for** — read first, so a switch can forward immediately
Source MAC	6 bytes	Who **sent** it — this is what a switch learns from
Type	2 bytes	What's inside (e.g. IPv4, IPv6, ARP)
Data	46–1500 bytes	The packet being carried
FCS	4 bytes	Corruption check, added at the **end** so it can cover everything before it

- **Destination MAC:** Who is this frame for?
- **Source MAC:** Who sent it?
- **Type/Length:** What kind of data is inside (e.g., IPv4)?
- **Data:** The actual payload (an IP packet lives here).
- **FCS (Frame Check Sequence):** An error-check. If the math doesn't add up, the frame is damaged and gets thrown away.

5.4 Three Ways to Address a Frame: Unicast, Broadcast, Multicast

Type	Meaning	Real Life
Unicast	One-to-**one**	Whispering to one friend
Broadcast	One-to-**everyone**	Yelling to the whole class
Multicast	One-to-**a group**	Talking to just your study group

- **Broadcast MAC address** is all F's: `FF:FF:FF:FF:FF:FF`. Every device listens to it.

Type	Goes to	In the example
Unicast	Exactly one device	Only B receives it
Broadcast	Every device on the segment	B, C and D all receive it
Multicast	Only devices that joined the group	B and D receive it; C does not

5.5 How Devices Learn About Collisions (CSMA/CD)

Old networks (with hubs) shared one wire, so two devices could talk at once and **collide**. Ethernet used a polite system called **CSMA/CD**:

- **CS** (Carrier Sense): "Listen first — is anyone talking?"
- **MA** (Multiple Access): "We all share the same wire."
- **CD** (Collision Detection): "If we crash, stop, wait a random time, and try again."

Story Time: It's like a group of polite kids in a room. Before you speak, you listen. If two of you start at the same time, you both stop, wait a random number of seconds, and try again. The random wait means you probably won't collide twice in a row.

Why the RANDOM wait — why not a fixed wait? This is a clever detail. If both kids waited the same fixed amount (say exactly 1 second), they'd both start talking again at the same moment — and collide again, forever! By each waiting a **random** amount, they almost certainly pick different times, so one goes first and the other hears them and holds off. Randomness is what breaks the tie. (The real rule, called "backoff," even makes the random wait get bigger after repeated collisions, to calm things down when the wire is busy.)

Why does this whole system exist — and why is it mostly gone now? CSMA/CD only mattered because old **hubs** forced everyone to share one wire (one collision domain), so collisions were unavoidable and you needed rules to recover from them. Modern **switches** give every device its **own dedicated wire** and run **full-duplex** (send and receive at once), so two devices are never fighting for the same wire — **collisions literally can't happen**. That's why CSMA/CD is now basically history: we fixed the root cause (the shared wire) instead of just managing the symptom (collisions). The exam still asks about it because it explains why switches were such a huge upgrade over hubs.

> **Modern note:** Today's **switches** use **full-duplex**, so collisions basically don't happen anymore. CSMA/CD is mostly history, but the exam still mentions it.

Chapter 6: Switches — The Traffic Cops of the LAN

6.1 The Switch's Superpower: The MAC Address Table

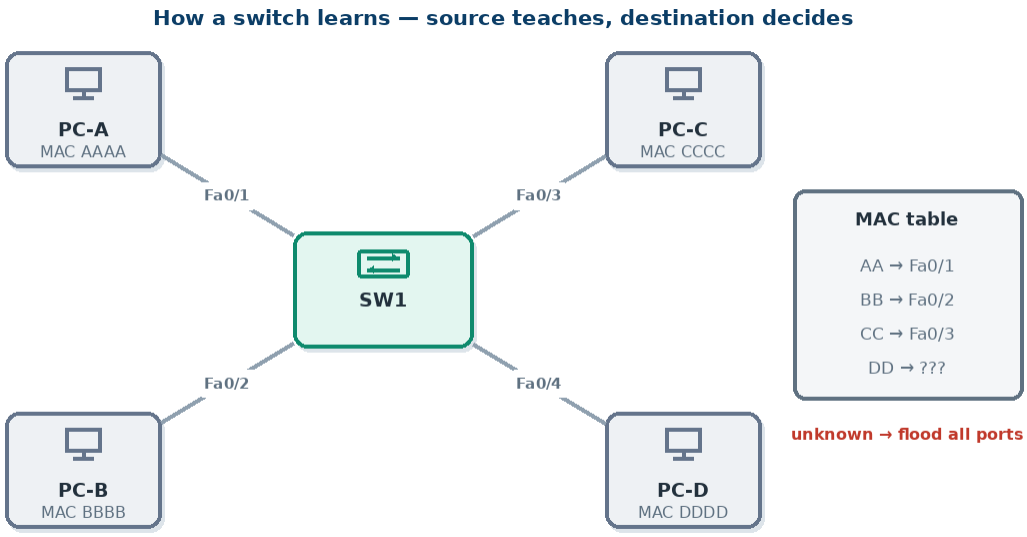
A **switch** connects many devices in a LAN. Its genius is that it learns **which device is on which port** and only sends data where it needs to go (unlike a dumb hub that shouts to everyone).

It keeps a list called the **MAC address table** (or CAM table):

A switch's MAC address table is simply a list of which address lives on which port:

MAC Address	Port
`00:11:…:AA`	Fa0/1
`00:22:…:BB`	Fa0/2
`00:33:…:CC`	Fa0/3

6.2 How a Switch Learns (Step by Step)



Story Time: A new switch is like a new mail sorter who doesn't know anyone yet. It learns by watching return addresses.

1. **Learning:** When a frame comes in, the switch looks at the **Source MAC** and remembers "Aha, that device is on this port."
2. **Flooding (when unsure):** If the switch doesn't know where the **Destination MAC** is, it sends the frame out **all ports** (except the one it came in on) — just in case. This is called **flooding**.
3. **Forwarding:** Once it knows the destination's port, it sends the frame **only** to that one port.
4. **Filtering:** If the destination is on the **same port** the frame came from, the switch drops it (no need to send it back).
5. **Aging:** If a device is quiet for a while (default 300 seconds), the switch forgets it to keep the table fresh.

Step 1. PC-A (on port 1) sends a frame to PC-C, but the switch has never heard of PC-C. So it **floods** the frame out ports 2, 3 and 4 — and at the same time **learns** that PC-A is on port 1, from the frame's source address.

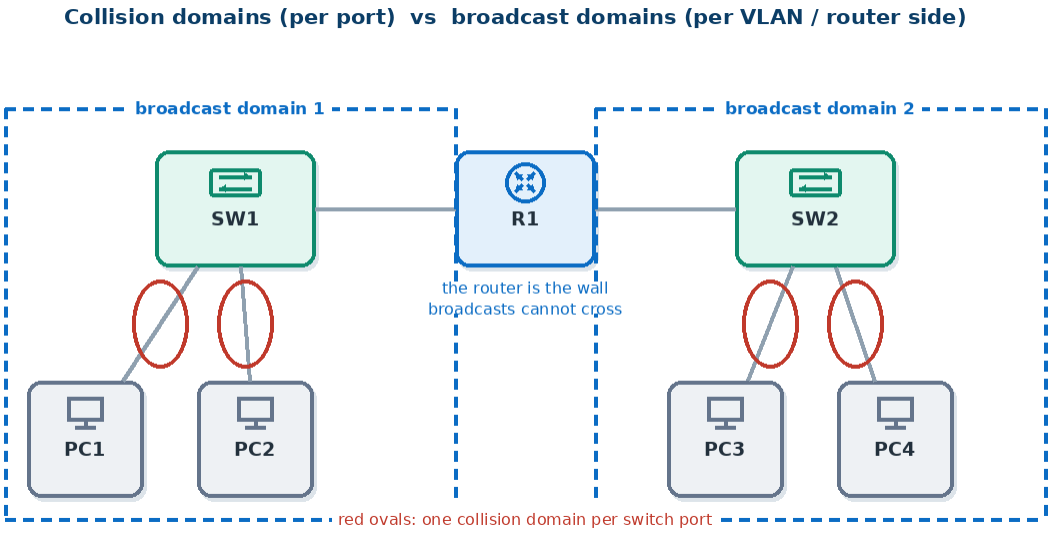
Step 2. PC-C (on port 3) replies to PC-A. The switch already knows PC-A is on port 1, so it sends the reply **only** to port 1 — and **learns** that PC-C is on port 3.

The table now holds both. From here on, traffic between them goes straight to the right port with no flooding.

Let's understand WHY the switch behaves this way — every step has a smart reason:

- **Why learn from the SOURCE, not the destination?** Because the source is a **fact the switch can trust**: a frame physically arrived on port 1 carrying source MAC AA, so AA must be reachable through port 1 — no guessing. The destination, on the other hand, is just a request ("please deliver to CC") that the switch may not know yet. You learn from what already happened, not from what's being asked.
- **Why FLOOD an unknown destination instead of dropping it?** Because dropping it would break communication for a device the switch simply hasn't heard from yet. The switch reasons: "I don't know where CC is, but CC probably exists — let me send this out every port so it reaches CC wherever it is." Flooding guarantees delivery even before learning is complete. And it's self-correcting: the moment CC replies, the switch **learns CC's port from that reply** and never has to flood to CC again. So flooding is a one-time "I'll shout until I learn," not a permanent waste.
- **Why FILTER (drop) a frame whose destination is on the same port it came in on?** Because that device is already on that wire — it doesn't need the switch to echo the frame back the way it came. Sending it back would be pointless noise.
- **Why AGE OUT (forget) quiet devices after ~300 seconds?** Two reasons. First, the MAC table has **limited memory** — you can't remember every device forever. Second, devices **move** (unplugged and plugged into a different port, or a laptop roams). If the switch remembered the old port forever, it would keep sending frames to the wrong place after a device moved. Forgetting stale entries keeps the map **fresh and correct**. If the device is still active, it'll send another frame and get re-learned instantly — so nothing is lost.

6.3 Collision Domains vs. Broadcast Domains



Two important ideas:

- **Collision Domain:** An area where two frames could collide. **Each switch port is its own collision domain** (that's why switches are great — tiny collision domains = no collisions).
- **Broadcast Domain:** An area where a broadcast reaches everyone. **All ports on a switch are one broadcast domain** by default. A **router** (or a VLAN) breaks up broadcast domains.

```
HUB:   one big collision domain (everyone can crash)
SWITCH: each port = own collision domain
        but all ports = one broadcast domain
ROUTER: each interface = own broadcast domain
```

Memory trick: Switches break up collision domains. Routers break up broadcast domains.

Why do these two "domains" even matter in real life? Because they're really two different questions about how big a problem can spread:

- A **collision domain** asks "how many devices are fighting over one wire?" Bigger collision domains = more crashes = slower network. Switches shrink each collision domain down to a **single wire** (one device), which is why replacing hubs with switches was such a massive speed boost — nobody fights over the wire anymore.
- A **broadcast domain** asks "how far does a 'HEY EVERYONE!' shout travel?" Every device in the domain must stop and process every broadcast. Too many devices in one broadcast domain = everyone drowning in broadcast noise (exactly the problem VLANs and routers solve, from Chapter 1 and Chapter 7). This is why we care: keeping broadcast domains a reasonable size is the difference between a snappy network and a sluggish one. Switches can't shrink broadcast domains on their own — only **routers** (or **VLANs**, which are just broadcast domains in software) can. That single fact drives a huge amount of network design.

6.4 Meeting the Cisco IOS (The Switch's Brain)

Cisco devices run an operating system called **IOS** (Internetwork Operating System). You type commands to configure them. Let's learn the modes:

```
Switch>              ← User EXEC mode (look, but limited)
Switch#              ← Privileged EXEC mode (full view, "enable")
Switch(config)#      ← Global Config mode (change settings)
Switch(config-if)#   ← Interface Config (change one port)
```

How to move between modes:

```
Switch> enable                 ← go from User to Privileged (# prompt)
Switch# configure terminal     ← go into Global Config
Switch(config)# interface fa0/1  ← go into one interface
Switch(config-if)# exit        ← go back one level
Switch(config)# end            ← jump all the way back to Privileged
```

6.5 Your First Switch Configuration (Try It!)

Here's a friendly starter config with comments explaining each line:

```
Switch> enable
Switch# configure terminal
Switch(config)# hostname SW1                  ! name the switch "SW1"
SW1(config)# enable secret MyPass123          ! password to enter privileged mode
SW1(config)# line console 0                   ! configure the console port
SW1(config-line)# password ConsolePass        ! set a console password
SW1(config-line)# login                       ! require the password
SW1(config-line)# exit
SW1(config)# service password-encryption      ! scramble passwords in the config
SW1(config)# banner motd #Authorized Users Only!#  ! warning message
SW1(config)# exit
SW1# copy running-config startup-config       ! SAVE! (or it's lost on reboot)
```

> **Super important:** `running-config` is the **live** config in memory (lost on reboot). `startup-config` is **saved** to storage. Always `copy running-config startup-config` (or `write memory`) to save your work!

6.6 Setting Up Remote Management (SSH)

To manage a switch over the network (not just the console cable), set up **SSH** (secure) — never Telnet (unsecure). A switch needs an IP address on a **management VLAN** for this:

```
SW1(config)# ip domain-name mycompany.com
SW1(config)# crypto key generate rsa          ! creates encryption keys (choose 1024+)
SW1(config)# username admin secret AdminPass ! a login account
SW1(config)# interface vlan 1
SW1(config-if)# ip address 192.168.1.10 255.255.255.0  ! management IP
SW1(config-if)# no shutdown                   ! turn the interface ON
SW1(config-if)# exit
SW1(config)# ip default-gateway 192.168.1.1  ! how to reach other networks
SW1(config)# line vty 0 15                    ! the 16 virtual "remote login" lines
SW1(config-line)# transport input ssh         ! allow SSH only (block Telnet)
SW1(config-line)# login local                 ! use the username/password above
SW1(config-line)# exit
```

6.7 Helpful "Show" Commands (Your Eyes Into the Switch)

Command	What it shows
`show running-config`	The current live configuration
`show mac address-table`	What MACs the switch learned & where
`show interfaces status`	Which ports are up/down, speed, VLAN
`show ip interface brief`	Quick list of interfaces & IPs
`show version`	IOS version, uptime, hardware info
`show vlan brief`	List of VLANs and their ports

6.8 Port Security — Locking Down a Port

Port security stops strangers from plugging into your network. It limits **which** and **how many** MAC addresses can use a port.

```
SW1(config)# interface fa0/1
SW1(config-if)# switchport mode access           ! make it a normal device port
SW1(config-if)# switchport port-security         ! turn on port security
SW1(config-if)# switchport port-security maximum 1   ! allow only 1 device
SW1(config-if)# switchport port-security mac-address sticky  ! remember the first MAC seen
SW1(config-if)# switchport port-security violation shutdown  ! if broken, shut the port
```

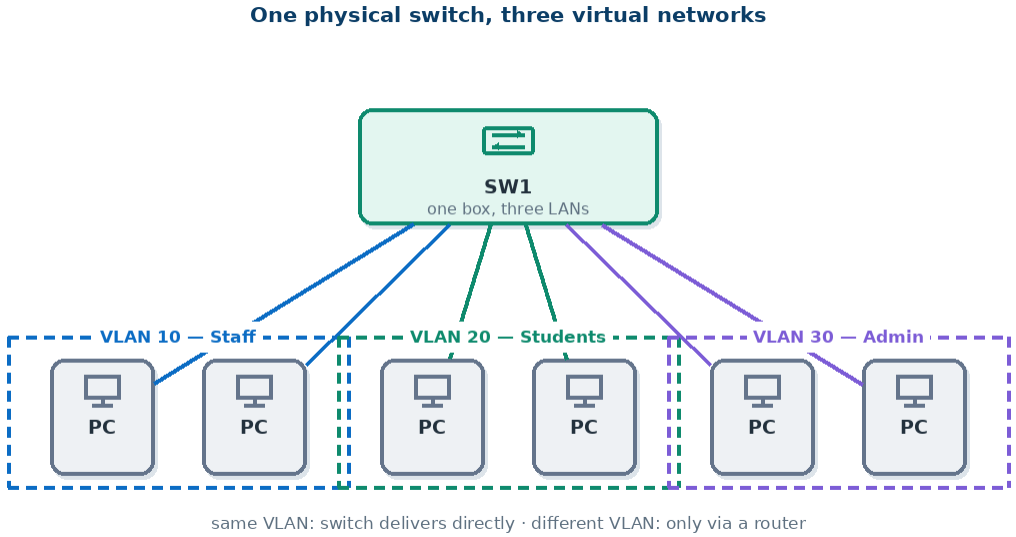
Violation modes (what happens if a rule is broken):

- **protect:** Silently drop the bad traffic. No alert.
- **restrict:** Drop the bad traffic AND log/alert.
- **shutdown:** Turn the port off completely (default). Needs an admin to turn it back on.

Story Time: Port security is like a locker that only opens for the first backpack you put in it. If a different backpack tries, the locker can ignore it (protect), ignore-and-tattle (restrict), or slam shut and refuse everyone (shutdown).

Chapter 7: VLANs — Splitting One Switch Into Many

7.1 The Problem VLANs Solve (Let's Really Understand This)



To understand why VLANs exist, we first have to feel the **pain of not having them**. So let's build up the problem slowly, step by step.

Step 1: Remember what a switch does by default

A plain switch puts **every port in one big network**. Any device can talk to any other device. More importantly — and this is the key — **a broadcast from one device reaches EVERY other device** on that switch.

Remember broadcasts? They're the "HEY EVERYONE!" messages (destination MAC `FF:FF:FF:FF:FF:FF`). Devices send them constantly for normal things:

- **ARP** ("Who has IP 192.168.1.5? Tell me your MAC!")
- **DHCP** ("Any DHCP server out there? I need an address!")
- Windows network discovery, printer announcements, and more.

A switch **must flood every broadcast out of every port**. It has no choice — that's the rule. The area a broadcast reaches is called a **broadcast domain**.

Step 2: Now picture a real school with one flat network

Imagine a school with **200 devices** all plugged into switches, all in one broadcast domain: student laptops, teacher PCs, the principal's computer, printers, security cameras, and the payroll server.

Problem A — Broadcast noise (performance): Every one of those 200 devices sends broadcasts. Every broadcast goes to **all 200 devices**. Each device's CPU has to stop and inspect every broadcast, even ones meant for nobody it cares about. With hundreds of devices, this becomes a constant storm of interruptions that **wastes bandwidth and slows everything down**.

In one flat network with no VLANs, a single PC asking "who has 192.168.1.5?" sends a broadcast that reaches **all 200 devices** — laptops, printers, cameras, the payroll server, everything. Every one of them has to stop and process a question that almost none of them can answer.

Problem B — No security separation: Because everyone is on the same network, a **student laptop can try to reach the payroll server** directly. There's no wall between them. If a student's laptop gets a virus, it can spread to the principal's computer and the cameras. That's dangerous.

Problem C — Location locks you in: In a flat network, "which network you're on" is decided by "which switch you physically plug into." If the principal moves to a classroom across the building, they'd be on the classroom's network — mixed in with students. To fix it, someone would have to **run new cables**. Ugh.

Step 3: The old (bad) fix, and why VLANs are better

Before VLANs, the only way to separate groups was to **buy a separate physical switch for each group** — one switch for teachers, one for students, one for admin — and keep them totally unplugged from each other. That's expensive, wasteful (a 48-port switch with only 6 teachers uses 6 ports), and inflexible.

VLANs solve all three problems at once by letting you create **multiple separate networks inside a single physical switch** — invisible walls, in software.

Setup	Broadcast domains	Who shares one
Without VLANs	**1**	Teachers, students and admin all together
With VLANs	**3**	VLAN 10 Teachers · VLAN 20 Students · VLAN 30 Admin

And that's on the **same physical switch** — no extra hardware involved.

The single most important sentence about VLANs:

> **A VLAN = a broadcast domain.** When you make 3 VLANs, you've made 3 separate broadcast domains — as if you had 3 separate physical switches, even though it's one box.

Now a broadcast from a student only reaches **other students** (VLAN 20), never the teachers or the payroll server. Problem A (noise) shrinks, Problem B (security) is solved by the wall, and Problem C (location) is solved because you can just **assign a port to a VLAN with a command** instead of running cables.

7.2 Why VLANs Are Awesome (Each Benefit, With the Reasoning)

Now that you feel the problem, here's why each benefit actually works:

- **Security (a real wall):** Devices in different VLANs are on different networks (different subnets). A device **physically cannot** send a frame straight into another VLAN — the switch won't do it. The only way between VLANs is through a **router**, and at the router you can add rules (ACLs) to allow or block specific traffic. So VLANs turn "everyone can reach everything" into "you decide exactly who can reach what." Example: students (VLAN 20) simply have no path to the payroll server (VLAN 30) unless you deliberately create and permit one.
- **Smaller broadcast domains (speed):** Fewer devices per VLAN = fewer broadcasts per device = less wasted CPU and bandwidth. Example: splitting 200 devices into four 50-device VLANs means each broadcast bothers 50 devices instead of 200.
- **Organization by role, not location:** You group devices by **what they are** (all cameras together, all VoIP phones together), no matter where they're plugged in. Example: a camera in the gym and a camera in the office can both be in VLAN 50, even though they're on opposite sides of the building.
- **Flexibility (no re-cabling):** Moving someone to a different network is one command (`switchport access vlan X`), not a cabling job. Example: the principal moves rooms; you just set their new port to VLAN 30 and they're back on the admin network instantly.
- **Cost savings:** One physical switch does the job of several. No need to buy separate hardware per group.

Big idea (and a cliffhanger): Each VLAN is its **own broadcast domain / its own subnet**. That's great for separation — but sometimes teachers legitimately need to reach a shared server on another VLAN. Moving data **between** VLANs requires a **router** (or a Layer 3 switch). That's called **inter-VLAN routing**, and we cover it in Chapter 8. For now, just hold this thought: VLANs separate; routers reconnect — on your terms.

7.3 Access Ports vs. Trunk Ports — The Deep "Why"

This is the concept students most often memorize without understanding. Let's make sure **you truly get it**, because once you see the problem, the answer becomes obvious.

The core question: how does a switch remember which VLAN a frame belongs to?

Here's a subtle thing. A normal Ethernet frame has **no field that says "I'm in VLAN 20."** VLANs are a switch invention — regular frames know nothing about them. So the switch keeps that information **in its own memory**: "the frame that came in on port Fa0/2 belongs to VLAN 20, because I configured Fa0/2 as a VLAN 20 port."

As long as everything stays **inside one switch**, this works perfectly. The switch knows every port's VLAN, so it keeps VLANs separate on its own.

But what happens when we have TWO switches?

The scenario that creates the whole problem

Imagine two switches, SW1 and SW2, in different parts of a building. You have VLAN 10 (Teachers) and VLAN 20 (Students) on **both** switches — because teachers and students sit in both areas.

Now, Teacher-A is plugged into SW1, and Teacher-B is plugged into SW2. They're both in VLAN 10, so they should be able to talk. Their frame has to travel across the **one cable that connects SW1 to SW2**.

Here's the trap. When SW1 sends Teacher-A's frame across that cable to SW2, SW2 receives a **plain Ethernet frame with no VLAN information in it**. SW2 thinks: "A frame arrived on my uplink port... but which VLAN is it? Teachers? Students? I have no idea!"

Picture SW1 and SW2, each holding both a Teacher (VLAN 10) and a Student (VLAN 20), joined by **one cable**. A frame leaves SW1 — but the frame itself carries **no label saying which VLAN it belongs to**. SW2 receives it and has no way to answer the only question that matters: which VLAN is this?

If that one cable were an **access port** (belongs to a single VLAN), it could only carry **one** VLAN's traffic. So you'd need a **separate cable for every VLAN** between the switches:

The naive fix is one cable per VLAN: a VLAN 10 cable, a VLAN 20 cable, a VLAN 30 cable, each plugged into access ports at both ends. It works — and it scales terribly. Twenty VLANs would mean **twenty cables** and twenty burned ports on each switch.

That obviously doesn't scale. We need **one cable to carry many VLANs** — but then we're back to the problem: how does the receiving switch know which VLAN each frame belongs to?

The solution: tagging, and that's what a TRUNK is

The answer is beautifully simple: **before sending a frame across the shared cable, the switch attaches a little label (a "tag") that says which VLAN it belongs to.** The receiving switch reads the tag, learns the VLAN, removes the tag, and delivers the frame correctly.

A **cable/port that carries multiple VLANs by tagging frames** is called a **TRUNK**. A port that carries **just one VLAN with no tags** (for a normal end device) is called an **ACCESS** port. That's the whole distinction:

The real fix is **one trunk cable** carrying VLAN 10 and VLAN 20, with every frame **tagged** with its VLAN number so SW2 knows exactly where it belongs.

Port	Role	Carries	Tagged?
Fa0/1, Fa0/2	**Access**	One VLAN each	No — for PCs
Fa0/24	**Trunk**	Many VLANs	Yes — switch-to-switch

So: WHY access, WHY trunk? (The rule you'll never forget)

- Use an **ACCESS port** for a link to a **normal end device** (PC, printer, phone, camera, server). Why? Because that device belongs to exactly **one** VLAN and knows nothing about VLAN tags. The switch keeps the VLAN info to itself; the device just sends and receives plain frames, blissfully unaware VLANs even exist. If you sent an ordinary PC a tagged frame, it usually wouldn't understand it.
- Use a **TRUNK port** for a link between **two network devices that both need to carry many VLANs** — switch-to-switch, switch-to-router (for inter-VLAN routing), or switch-to-access-point. Why? Because a single cable must carry traffic for **multiple** VLANs, and tagging is the only way for the other end to tell them apart.

One-sentence memory hook:

> **Access = to a device (one VLAN, no tag). Trunk = between switches (many VLANs, tagged).**

Ask yourself: "Does this link need to carry more than one VLAN?" If **yes → trunk**. If **no (it's an end device) → access**. That single question answers it every time.

A concrete walk-through (follow the frame!)

Let's trace Teacher-A (VLAN 10 on SW1) sending a frame to Teacher-B (VLAN 10 on SW2):

1. Teacher-A's PC sends a **plain frame** into SW1's port Fa0/1 (an **access** port in VLAN 10). The PC has no idea about VLANs.
2. SW1 knows "Fa0/1 = VLAN 10," so internally it treats the frame as VLAN 10.
3. SW1 needs to send it to SW2 across the **trunk** (Fa0/24). Because the trunk carries multiple VLANs, SW1 **adds a tag: "VLAN 10"** to the frame.
4. The tagged frame crosses the cable. SW2 receives it on **its** trunk port, reads the tag, and thinks "Ah, VLAN 10."
5. SW2 **removes the tag** and looks for VLAN 10 devices. Teacher-B is on Fa0/5 (an **access** port in VLAN 10), so SW2 sends the now-plain frame out Fa0/5.
6. Teacher-B's PC receives a normal, untagged frame — it never knew a tag was ever involved.

Notice the pattern: **tags exist only on the trunk, in transit between switches.** They're added when entering the trunk and stripped before reaching the end device. Access ports never see tags; trunks always use them.

7.4 Configuring VLANs (Try It!)

```
SW1(config)# vlan 10                  ! create VLAN 10
SW1(config-vlan)# name Teachers       ! give it a friendly name
SW1(config-vlan)# vlan 20
SW1(config-vlan)# name Students
SW1(config-vlan)# exit

SW1(config)# interface fa0/1          ! the port a teacher's PC uses
SW1(config-if)# switchport mode access      ! this is an access port (one VLAN)
SW1(config-if)# switchport access vlan 10   ! put it in VLAN 10
SW1(config-if)# exit

SW1(config)# interface range fa0/2 - 12   ! do many ports at once
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 20  ! students
```

Why `switchport mode access` AND `switchport access vlan 10`? The first line says "this port carries exactly one VLAN and will never tag frames" (its role). The second says "and that one VLAN is number 10" (which VLAN). Role first, then assignment.

Check your work:

```
SW1# show vlan brief
VLAN Name       Status    Ports
---- ---------- --------- -------------------------------
1    default    active    Fa0/13, Fa0/14, ...
10   Teachers   active    Fa0/1
20   Students   active    Fa0/2, Fa0/3, ... Fa0/12
```

7.5 The Default & Special VLANs (And Why They Matter)

- **VLAN 1 — the default:** Every port starts in VLAN 1 out of the box. That's convenient, but it's a security weakness: attackers know VLAN 1 is the default and often target it. **Best practice: don't use VLAN 1 for real user traffic or management.** Move important things to a purpose-made VLAN. Why: reducing predictability reduces risk.
- **Native VLAN — the "untagged" exception on a trunk:** Here's a subtle but important idea. On a trunk, almost every frame gets a tag. But 802.1Q leaves **one** VLAN **untagged** — the **native VLAN** (default VLAN 1). Any frame that arrives on a trunk **without** a tag is assumed to belong to the native VLAN.

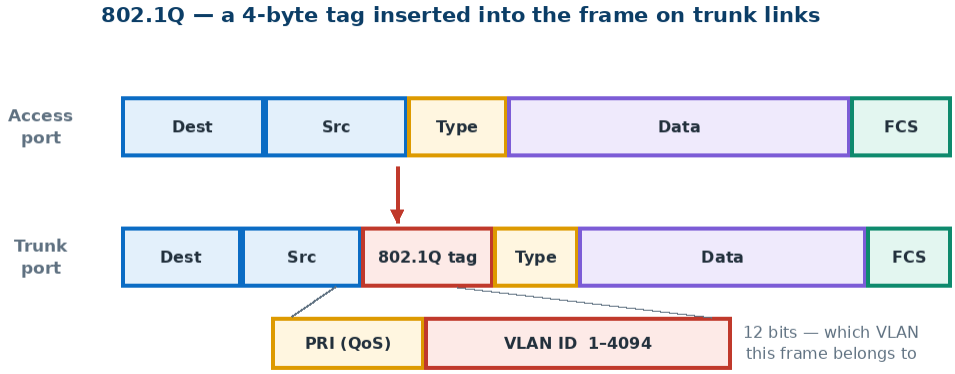
Why does an untagged option even exist? Historically, for compatibility with older devices that didn't understand tags (like a legacy hub or a switch that predates 802.1Q). It gives the trunk a sensible default: "if there's no label, put it here."

Why is it a security topic? If the two ends of a trunk **disagree** on the native VLAN, frames can "leak" from one VLAN into another (a **VLAN-hopping** attack called double-tagging). **Best practice: set the native VLAN to an unused VLAN (e.g., 99) and make sure both ends match.** Why: it removes the mismatch attackers exploit and keeps user traffic off the untagged path.

- **Management VLAN — keep admins separate:** The VLAN you use to remotely reach and configure your switches (SSH to the switch's IP). Putting it in its own dedicated VLAN (not VLAN 1, not a user VLAN) means normal users can't even see the management traffic, let alone attack it. Why: separation of "who runs the network" from "who uses the network."
- **VLANs 1002–1005:** Reserved for old Token Ring/FDDI tech. You'll never use them today — just know they're reserved so the numbers aren't a surprise on the exam.

Chapter 8: Trunks, VTP & Inter-VLAN Routing

8.1 Trunking with 802.1Q (Dot1Q) — How the Tag Actually Works



In Chapter 7 you learned why trunks exist: a single cable must carry many VLANs, so we **tag** each frame with its VLAN number. Now let's see how that tag works, because the exam tests the details.

The tagging standard is **802.1Q** (say "dot-one-Q"). When a frame goes onto a trunk, the switch inserts a **4-byte tag** right into the middle of the Ethernet frame — squeezed in between the Source MAC and the Type field:

Frame	Fields, in order
Normal frame	Dest MAC → Src MAC → Type → Data → FCS
Tagged frame	Dest MAC → Src MAC → **802.1Q tag (4 bytes)** → Type → Data → FCS

The tag is **inserted after the source MAC**, not bolted on the front — the addresses stay exactly where hardware expects to find them, so a switch reads them at full speed and only then notices the tag.

What's inside that 4-byte tag? The important part for the exam is the **VLAN ID** — a number from **1 to 4094** that says which VLAN the frame belongs to. (The tag also holds a priority field for QoS, but the VLAN ID is the star.)

Why does the switch re-do the checksum? The frame ends with an **FCS** (error-check). Since the switch changed the frame by adding a tag, it must **recalculate the FCS** so the frame still passes the error check on the other side. Good to know it happens automatically.

The full journey of a tag (tie it together): 1. A plain frame enters an **access** port → the switch knows its VLAN internally, no tag yet. 2. The frame needs to cross a **trunk** → the switch **adds** the 802.1Q tag with the VLAN ID. 3. The other switch's trunk **reads** the tag to learn the VLAN. 4. Before the frame reaches the destination **access** port, the switch **removes** the tag. 5. The end device gets a normal, untagged frame. The tag lived only on the trunk.

Configuring a Trunk (and why each line matters)

```
SW1(config)# interface fa0/24
SW1(config-if)# switchport mode trunk            ! this port carries many VLANs (tagged)
SW1(config-if)# switchport trunk native vlan 99  ! untagged traffic = VLAN 99 (not 1, for security)
SW1(config-if)# switchport trunk allowed vlan 10,20,30  ! only let these VLANs cross
```

- `switchport mode trunk` — sets the port's role to trunk, so it will tag/expect-tags instead of belonging to a single VLAN.
- `switchport trunk native vlan 99` — moves the untagged "native" VLAN off the default VLAN 1 to a safe, unused VLAN. Why: prevents native-VLAN-mismatch VLAN-hopping attacks. Both ends must match.
- `switchport trunk allowed vlan 10,20,30` — a whitelist. Why bother? A trunk carries **all** VLANs by default, which wastes bandwidth (broadcasts for VLANs nobody on the far switch uses) and widens the attack surface. Restricting the trunk to only the VLANs that actually need to cross is called **VLAN pruning** — it's cleaner, faster, and safer.

Check it:

```
SW1# show interfaces trunk
Port    Mode   Encapsulation  Status    Native vlan
Fa0/24  on     802.1q         trunking  99
```

8.2 DTP (Dynamic Trunking Protocol) — And Why to Disable It

Cisco switches can **automatically negotiate** whether a link becomes a trunk, using **DTP**. It's convenient, but understanding it matters for security.

Modes:

- **trunk (on):** Always a trunk.
- **access:** Always an access port.
- **dynamic auto:** Will become a trunk only if the other side asks.
- **dynamic desirable:** Actively asks the other side to form a trunk.

Why is auto-negotiation risky? Because an attacker can plug in a laptop that pretends to be a switch and says "let's form a trunk!" If your port is in a dynamic mode, it agrees — and now the attacker's laptop receives **all VLANs**, defeating your separation. This is a **VLAN-hopping** attack.

> **Security best practice:** On end-device ports, hard-set `switchport mode access` and add `switchport nonegotiate` (which turns DTP off). On real switch-to-switch links, hard-set `switchport mode trunk` and also `switchport nonegotiate`. Why: never leave "should this be a trunk?" up to negotiation an attacker could exploit.

8.3 VTP (VLAN Trunking Protocol) — Convenience vs. Danger

VTP lets you create a VLAN on **one** switch and have it **automatically copy** to all other switches in the same VTP domain. In a network with 50 switches, that's a huge time-saver — no need to type `vlan 10` on all 50.

VTP modes:

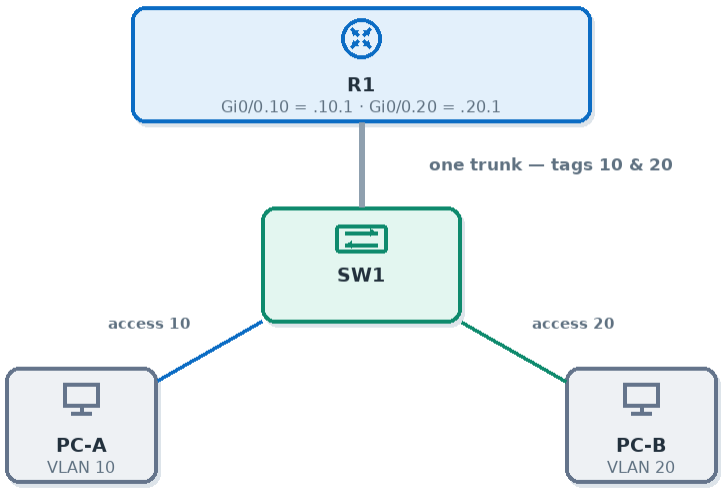
- **Server:** Can create/edit/delete VLANs; shares changes to others.
- **Client:** Receives VLAN info; can't edit VLANs itself.
- **Transparent:** Ignores VTP for its own VLANs (keeps them local) but forwards VTP messages along.

Why VTP can be dangerous (the famous horror story): VTP uses a **revision number** that increases every time you change VLANs. Switches trust whichever message has the **highest revision number**. If you take an **old switch** that has a high revision number and a nearly-empty VLAN list, and plug it into your network, it can **overwrite every other switch's VLANs with its own (empty) list** — instantly wiping VLANs network-wide and taking everyone offline.

> **Best practice:** Before adding any used switch to your network, **reset its VTP revision number to 0** (set it to VTP transparent mode and back, or erase config). Many engineers avoid VTP entirely, or use the safer VTP version 3, precisely because of this risk. Why: a convenience feature should never be able to delete your whole VLAN design by accident.

8.4 Inter-VLAN Routing — Why VLANs Need a Router to Talk

Router-on-a-stick — one trunk, one subinterface per VLAN



A → B: up the trunk tagged 10, routed, back down tagged 20

We spent Chapter 7 building **walls** between VLANs. But walls are inconvenient when teachers legitimately need to reach a shared file server on a different VLAN. So how do we let some traffic through, on our terms?

Why can't a switch just do it? Because VLANs are **different subnets** (e.g., VLAN 10 = 192.168.10.0/24, VLAN 20 = 192.168.20.0/24). A plain switch works at **Layer 2** (MAC addresses) and only moves frames **within** the same VLAN/subnet. The moment traffic must go from one subnet to another, that's a **Layer 3 (routing) decision** — and routing is a **router's** job (or a Layer 3 switch's job).

Story Time: VLANs are like separate countries with closed borders (great for order). A **router** is the international airport + customs — the only official crossing point, where you can also check passports (ACLs) and decide who's allowed through. Without the airport, the countries simply can't reach each other.

There are three ways to provide that routing:

Method 1: Old Way — One Router Cable Per VLAN

Run a separate physical link from the router to the switch for **each** VLAN. It works, but it burns a router port and a switch port per VLAN — the exact "one cable per VLAN" waste we solved with trunks. **Don't do this for more than a couple of VLANs.**

Method 2: Router-on-a-Stick (ROAS) — One Trunk, Virtual Sub-Ports

Use **one** trunk link between the switch and router. The router splits that single physical interface into **subinterfaces** — one virtual sub-port per VLAN — and gives each one an IP address that becomes that VLAN's **default gateway**.

Why subinterfaces? Because the trunk delivers **tagged** frames for many VLANs down one wire. The router needs a separate "identity" (and gateway IP) for each VLAN, so it creates a virtual interface per VLAN and matches each to a tag with `encapsulation dot1q`.

The router keeps **one physical link** to the switch — the "stick" — configured as a trunk, and splits it into **subinterfaces**, one per VLAN:

Subinterface	VLAN	Acts as gateway
`Gi0/0.10`	10	192.168.10.1
`Gi0/0.20`	20	192.168.20.1

The switch port at the other end is a trunk carrying both VLANs.

Router config for ROAS:

```
R1(config)# interface gi0/0.10          ! subinterface for VLAN 10
R1(config-subif)# encapsulation dot1q 10   ! "frames tagged VLAN 10 belong to me"
R1(config-subif)# ip address 192.168.10.1 255.255.255.0  ! gateway for V10
R1(config-subif)# exit
R1(config)# interface gi0/0.20
R1(config-subif)# encapsulation dot1q 20
R1(config-subif)# ip address 192.168.20.1 255.255.255.0
```

How a cross-VLAN trip works here: A teacher (VLAN 10) wants the student server (VLAN 20). Because it's a different subnet, the teacher's PC sends the frame to its **default gateway** (192.168.10.1) up the trunk (tagged VLAN 10). The router receives it on subinterface Gi0/0.10, **routes** it to the 192.168.20.0 network, re-sends it tagged as VLAN 20 back down the trunk, and

the switch delivers it to the server. The router was the border crossing.

Method 3: Layer 3 Switch (SVI) — The Modern, Fast Way

A **Layer 3 switch** can route between VLANs **by itself**, using **SVIs** (Switch Virtual Interfaces) — a virtual gateway interface for each VLAN, right inside the switch. No external router, no trunk-to-a-router bottleneck.

Why is this preferred today? ROAS sends all inter-VLAN traffic up and back down a single trunk cable, which can become a bottleneck. A Layer 3 switch routes internally at hardware speed, so it's much faster — the standard choice in real networks.

```
L3SW(config)# ip routing                  ! turn on Layer 3 routing (off by default)
L3SW(config)# interface vlan 10
L3SW(config-if)# ip address 192.168.10.1 255.255.255.0   ! SVI = gateway for VLAN 10
L3SW(config-if)# no shutdown
L3SW(config)# interface vlan 20
L3SW(config-if)# ip address 192.168.20.1 255.255.255.0
L3SW(config-if)# no shutdown
```

Each PC uses its VLAN's `.1` address as its **default gateway** to reach other VLANs. Why the `.1`? It's just convention — the gateway is usually the first usable address in the subnet, so it's easy to remember and standardize.

Chapter 9: Spanning Tree Protocol (STP) — Stopping Loops

9.1 The Scary Problem: Loops

For safety, we connect switches with **extra cables** (redundancy) so if one breaks, another works. But this creates a **loop** — and loops are a disaster in Layer 2!

Story Time: Imagine two mirrors facing each other. A single reflection bounces forever, multiplying endlessly. A broadcast frame in a switch loop does the same — it circles forever, multiplying, until the network **melts down** (a "broadcast storm").

Wire SW1, SW2 and SW3 together in a triangle — each connected to both of the others — and you've built a **loop**. A broadcast entering that triangle circles it endlessly, because nothing in a frame tells a switch to stop forwarding it.

Here's the WHY that makes STP make sense — and it's a fact most beginners miss: Why do loops destroy a Layer 2 network but not the Internet, which is full of loops? The answer is a single missing feature: **Ethernet frames have no "expiration date."**

When you learn about IP packets (Layer 3) in Chapter 14, you'll meet a field called **TTL (Time To Live)** — a little counter that drops by 1 at each router. When it hits 0, the packet is thrown away. TTL is a **built-in self-destruct** that guarantees a lost packet can't circle forever. **Layer 2 Ethernet frames have NOTHING like this.** There is no TTL in a frame. So if a frame ever enters

a loop, **nothing will ever stop it** — it circles endlessly.

Now combine that with two switch behaviors you already know:

1. Switches **flood broadcasts and unknown frames out every port**. In a loop, a single broadcast comes back around, and the switch floods it again... and again... forever. Worse, at a loop junction the frame gets **duplicated** down multiple paths, so one broadcast becomes two, then four, then eight — a doubling avalanche. That's the **broadcast storm**, and it saturates every link in seconds.
2. Because the same source MAC keeps arriving on different ports as the frame loops, the switch's **MAC table thrashes** — constantly rewriting "AA is on port 1... no, port 2... no, port 3!" This is called **MAC flapping**, and it corrupts the switch's forwarding logic on top of the storm.

So a Layer 2 loop isn't a small slowdown — it's a **total, instant meltdown** of the whole broadcast domain. That's why we can't just "leave the backup cable plugged in and hope." We need something to actively guarantee there's never a loop — while still keeping that backup cable ready. Enter STP.

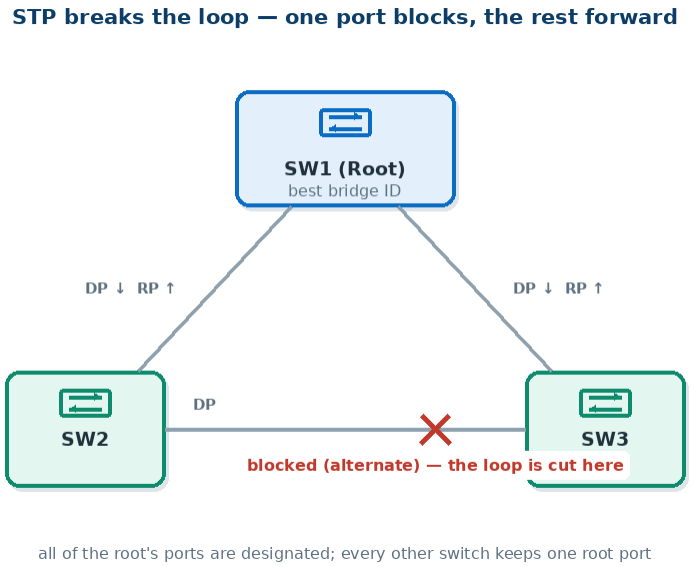
9.2 The Hero: Spanning Tree Protocol

STP (802.1D) automatically finds loops and **blocks** one path so there's only ONE way through — no loops. If the active path breaks, STP **unblocks** the backup path. Best of both worlds: redundancy WITHOUT loops.

STP leaves the triangle physically wired but **blocks one port**, so there's only one active path and no loop. If the working link later breaks, STP unblocks that port and everyone stays connected — the redundancy is still there, just held in reserve.

Why "block a port" instead of "unplug the cable"? Because a blocked port is a **backup on standby**, not a removed one. It still listens for STP messages, so it knows the instant the main path fails — and then it springs into action and starts forwarding. You get the safety of no-loops and the resilience of a spare path, automatically. Unplugging the cable would give you no loop but also no backup. Blocking gives you both.

9.3 How STP Chooses (The Election)



STP holds "elections" using special messages called **BPDUs** (Bridge Protocol Data Units).

1. **Pick a Root Bridge:** The "boss" switch. It's the one with the **lowest Bridge ID** (Bridge ID = priority + MAC address). Lower priority wins; if tied, lower MAC wins.
2. **Each other switch finds its Root Port:** The port with the **lowest cost** path to the root.
3. **Each segment picks a Designated Port:** The best port to forward on that link.
4. **Any leftover ports get Blocked:** To break the loop.

Why elect a "root bridge" at all? Why does STP need a boss? Because to decide which port to block, every switch needs to agree on a **single reference point** to measure distance from. Think of it like a group of hikers who all need to find the shortest way home: if they each pick a different "home," their maps disagree and they'll make conflicting choices — someone might block the wrong link and cut the network in half. By all agreeing on **one** root bridge, every switch measures "how do I best reach the root?" against the same landmark, so their independent decisions fit together into one loop-free tree. **The root is the shared point of reference that makes the whole thing consistent.** (That's also why it's called a spanning tree — a tree has no loops, and every branch traces back to one root.)

Why does "lowest Bridge ID" win, and why is it priority + MAC? STP needs a tie-proof way to pick exactly one winner. The **MAC address** is guaranteed unique, so it can always break a tie — there's never a stalemate. But you don't want the root chosen by random factory MAC luck (the oldest switch, often the weakest, tends to have the lowest MAC!). So STP puts an admin-controllable **priority** in front of the MAC. Priority is checked first, so by lowering the priority on your best switch, **you** decide the root — and the MAC is just the automatic backup tiebreaker. (This is exactly why section 9.7 tells you to set the root manually.)

Why pick ports by "lowest cost to root"? Because cost is based on **link speed** — a fast link has a low cost, a slow link a high cost (see the table). Choosing the lowest-cost path means STP keeps your **fastest** route active and blocks the slower, redundant one. So the network isn't just loop-free — it's loop-free on the best available path. Makes sense: if you have a 1 Gbps and a 100 Mbps route to the same place, you want the gigabit one working and the slow one as backup.

STP Path Costs (memorize these!)

Link Speed	STP Cost
10 Mbps	100
100 Mbps	19
1 Gbps	4
10 Gbps	2

Lower total cost = better path. Faster links have lower cost = preferred. (Notice the costs aren't a neat formula like 10/9/8 — they're specific values defined by the standard, which is why you just memorize them. Faster = lower, always.)

9.4 Port States (The Traffic Lights)

In classic STP, a port moves through states before it forwards data:

Order	State	Roughly how long	What happens
1	**Blocking**	~20 s	Listens to BPDUs, forwards nothing
2	**Listening**	~15 s	Works out the topology
3	**Learning**	~15 s	Builds its MAC table, still forwards nothing
4	**Forwarding**	—	Finally passes traffic

- **Blocking:** No data, just listens for BPDUs.
- **Listening:** Preparing, no data yet.
- **Learning:** Building the MAC table, still no data.
- **Forwarding:** Finally sending data!
- **Disabled:** Turned off.

Why does a port "waste" ~30–50 seconds crawling through these states instead of forwarding right away? Because forwarding too soon could create the very loop STP exists to prevent. Picture what happens when a new link comes up: for a brief moment, this switch and its neighbor might **both** think they should forward on it — and if they both start instantly, you get a temporary loop and a broadcast storm before STP finishes its calculations. The waiting states are a deliberate **"measure twice, cut once"** safety pause:

- **Listening** gives every switch time to exchange BPDUs and agree on the final tree, so no two switches make a conflicting choice.
- **Learning** lets the port build up its MAC table before it starts forwarding, so when it does go live it isn't flooding everything blindly.

The timers (20s blocking, 15s + 15s) are STP being cautious — it would rather be **slow and safe** than fast and looping. This caution is exactly why people invented the faster RSTP (next), and why PortFast exists for ports that provably can't cause a loop (a single PC).

9.5 Faster STP: RSTP (802.1w)

Classic STP is slow (~30-50 seconds to recover). **RSTP (Rapid Spanning Tree, 802.1w)** does the same job in **a few seconds**. It's the modern default. RSTP port roles: Root, Designated, **Alternate** (backup to root), and **Backup**.

9.6 PortFast & BPDU Guard (Helpful Extras)

- **PortFast:** Lets a port to a PC skip the waiting states and go straight to forwarding. Use ONLY on ports with end devices (never switch-to-switch!).
- **BPDU Guard:** If a PortFast port suddenly receives a BPDU (meaning someone plugged in a switch!), it shuts the port down. Great protection.

```
SW1(config)# interface fa0/1
SW1(config-if)# spanning-tree portfast
SW1(config-if)# spanning-tree bpduguard enable
```

Why is PortFast SAFE on a PC port but DANGEROUS on a switch port? A loop needs **at least two switches** — a single PC has only one cable and physically cannot loop traffic back on itself. So skipping the safety wait on a PC port risks nothing; there's no loop to worry about. But if you enabled PortFast on a **switch-to-switch** link, that port would jump straight to forwarding without the "measure twice" pause — the exact recipe for the instant loop STP is supposed to prevent. PortFast is safe precisely where a loop is impossible, and reckless where a loop is possible.

Why do we ALSO need BPDU Guard, and why does it pair with PortFast? Here's the risk PortFast creates: you told the switch "trust me, this port only has a harmless PC." But what if someone (a confused user, or an attacker) unplugs the PC and plugs in a **switch**? That port would happily fast-forward and could form a loop — the very thing you disabled the safety wait for. **BPDU Guard is the safety net.** Only switches send BPDUs, so if a PortFast port ever receives a BPDU, something is plugged in that shouldn't be. BPDU Guard immediately **shuts the port down** rather than risk a loop. So the two features are a team: **PortFast** makes PC ports fast, and **BPDU Guard** makes that speed safe by slamming the door if a switch ever appears where only a PC was allowed.

Why is PortFast also just... nice for users? Without it, a PC plugging in waits ~30 seconds in the STP states before it can talk — long enough that a PC might give up on getting a DHCP address ("no network!") before the port even wakes up. PortFast removes that delay, so devices get online instantly. Practical and it dodges a real support headache.

> **PortFast and BPDU Guard protect** access **ports.** Two more guards protect the rest of the topology — **Root Guard** and **Loop Guard** — and section **9.9** puts all four side by side so you don't mix them up on the exam.

9.7 Making a Switch the Root (Best Practice)

Don't let STP pick a random root. Force your best central switch to be root:

```
SW1(config)# spanning-tree vlan 10 root primary   ! become root for VLAN 10
! or set priority directly (must be multiples of 4096):
SW1(config)# spanning-tree vlan 10 priority 4096
```

9.8 STP Flavors — PVST+, Rapid PVST+ and Why Cisco Runs One Per VLAN

So far we've talked about "STP" as one thing. In reality there are several **versions**, and the exam expects you to tell them apart.

Flavor	Standard	Speed	Instances
STP	802.1D	Slow (30–50 s)	One for the whole switch
PVST+	Cisco	Slow (30–50 s)	**One per VLAN**
RSTP	802.1w	Fast (seconds)	One for the whole switch
Rapid PVST+	Cisco (802.1w per VLAN)	**Fast (seconds)**	**One per VLAN**

Rapid PVST+ is the Cisco default on modern switches, and it's the one to assume unless a question says otherwise.

Why run a separate spanning tree for every VLAN — isn't that a lot of extra work? It is more work, and it buys something valuable: **you can use all your links instead of wasting half of them.** Remember what plain STP does — it blocks redundant links to break loops. With one spanning tree for everything, a blocked link is blocked for all traffic; you bought a second uplink and it sits idle doing nothing until something breaks.

With one tree per VLAN, you can deliberately make **different switches the root for different VLANs**:

Make **SW1** the root for VLAN 10 and **SW2** the root for VLAN 20, and the blocking decisions land on different links for each VLAN:

VLAN	Root	Left link	Right link
10	SW1	forwarding	blocked
20	SW2	blocked	forwarding

Each link is blocked for some VLANs and forwarding for others — so **both uplinks carry real traffic**, while every VLAN still has a loop-free tree.

Now each link is blocked for some VLANs and forwarding for others, so both uplinks carry real traffic and you get crude **load balancing** — while each VLAN still has a perfectly loop-free tree.

You pay in CPU (each VLAN runs its own election) and gain link utilization. That trade is why Cisco made per-VLAN the default.

The command that sets which switch wins the election is per-VLAN for exactly this reason:

```
SW1(config)# spanning-tree mode rapid-pvst         ! the modern default
SW1(config)# spanning-tree vlan 10 root primary    ! SW1 = root for VLAN 10
SW2(config)# spanning-tree vlan 20 root primary    ! SW2 = root for VLAN 20
```

9.9 Protecting the Topology: Root Guard, Loop Guard & BPDU Filter

PortFast and BPDU Guard (section 9.6) protect **access ports**. There are two more guards the exam asks about, and each defends against a different accident. The trick is keeping them straight — so here's what each one is afraid of.

Root Guard — "You Are Not Allowed to Become the Root"

The fear: someone plugs a switch into your network that has a **better (lower) bridge priority** than your carefully chosen root — maybe an old switch from a cupboard with priority 4096 still configured, or a small unmanaged switch someone brought from home. STP does exactly what it's designed to do: it holds a new election, that switch **wins**, and suddenly your entire network's traffic is being funneled through a slow switch sitting under someone's desk. Nothing is "broken" — everything is just mysteriously terrible.

The fix: on ports where a root bridge should **never** appear (ports facing access switches or other companies), enable Root Guard. If a **superior BPDU** arrives, the port is put into **root-inconsistent** state — it stops forwarding until the offending switch stops shouting.

```
SW(config-if)# spanning-tree guard root
SW# show spanning-tree inconsistentports
```

Loop Guard — "I Stopped Hearing From You, So I'll Stay Cautious"

The fear: this one is subtle. A **blocking** port stays blocking only because it keeps receiving BPDUs from the neighbor. If those BPDUs stop arriving — not because the link died, but because of a one-way (unidirectional) failure, like a broken fiber strand in one direction — the blocked port thinks "no more BPDUs, the loop must be gone," and helpfully **starts forwarding**. The link is still physically up in the other direction, so you've just created a **real loop** — a broadcast storm caused by the loop-prevention protocol itself.

The fix: Loop Guard says "if a port that was receiving BPDUs suddenly stops, don't assume the loop is gone — assume something is wrong." The port goes into **loop-inconsistent** state instead of forwarding, and recovers automatically when BPDUs return.

```
SW(config-if)# spanning-tree guard loop
```

BPDU Filter — "Don't Even Talk About Spanning Tree Here"

BPDU Filter **suppresses BPDUs** on a port. It's the odd one out, because it's the only one that makes the network less protected, and it's genuinely dangerous: filtering on a port that turns out to connect to another switch means **STP is blind to that link**, and a loop there will never be blocked. Use it only on ports facing networks you deliberately don't share STP with.

Keeping Them Straight (Exam Gold)

Feature	Put it on	Triggers when	Reasoning
PortFast	Access ports (PCs)	—	Skip the wait; PCs don't cause loops
BPDU Guard	Access ports (PCs)	A BPDU **arrives**	A PC port should never see a switch
Root Guard	Toward other switches	A **superior** BPDU arrives	That switch may not become root
Loop Guard	Blocking/non-designated ports	BPDUs **stop arriving**	Silence is suspicious, not safe
BPDU Filter	Rare, edge cases	—	Stops sending/processing BPDUs

The one-line memory hook: *BPDU Guard reacts to a BPDU showing up where it shouldn't. Loop Guard reacts to a BPDU disappearing where it should still be. One fears noise, the other fears silence.

9.10 STP Mechanics In Depth — Bridge IDs, Costs and Port Roles

> **Depth section.** Section 9.3 covered who wins; this covers the fields and the exact decision order — which is what troubleshooting (and CCNP) actually requires.

What a BPDU Carries

Switches converge by flooding one small message, the **BPDU**, every 2 seconds. Four fields decide everything:

Field	Meaning
Root Bridge ID	Who the sender believes is root
Root Path Cost	The sender's total cost to reach that root
Sender Bridge ID	Who is sending this BPDU

Field	Meaning
Port ID	Which port it left from (port priority + port number)

The Bridge ID Is Not Just a Priority

The Bridge ID is **8 bytes**, in three parts:

Part	Size	Contents
Priority	4 bits	0–61440, **only in steps of 4096**
Extended System ID	12 bits	The **VLAN number**
MAC address	6 bytes	The switch's own address — the final tie-breaker

Default: priority 32768 + the VLAN — which is why VLAN 1 displays as 32769.

Why is the priority forced into steps of 4096, instead of any number you like? Because the low 12 bits were **repurposed to hold the VLAN ID**. When Cisco moved to one spanning tree per VLAN (section 9.8), each instance needed its own distinct Bridge ID — but the field was full. The fix was to carve the VLAN number out of the priority field, which leaves only the top 4 bits adjustable: $2^4 = 16$ usable values, spaced 4096 apart. That's why `spanning-tree vlan 10 priority 4097` is rejected but 4096 is accepted — and why a switch's Bridge ID differs per VLAN even though the MAC is identical.

This also explains the default you see everywhere: priority 32768 for VLAN 1 displays as **32769** (32768 + 1).

How Root Path Cost Accumulates

Cost is **added on reception**, using the cost of the port the BPDU arrived on:

Link speed	Cost
10 Mbps	100
100 Mbps	19
1 Gbps	4
10 Gbps	2

Take a chain — ROOT, then SW2, then SW3, joined by 1 Gbps links (cost 4 each):

Switch	BPDU says	Adds its ingress cost	Advertises onward
ROOT	—	—	cost **0**
SW2	receives 0	+ 4	cost **4**
SW3	receives 4	+ 4	knows it is **8** from root

Why add on the way in rather than on the way out? Because a switch knows the speed of the link it received on, but not what the neighbour's outbound link will be. Charging the cost at ingress means every switch computes its own distance from local facts only — no coordination

needed. Notice costs are not linear with speed: 10 Mbps is 100 while 1 Gbps is 4. The scale was chosen so slow links look dramatically worse, not proportionally worse.

The Decision Order (Memorize This Sequence)

Every port role in the network comes from applying four tie-breakers, **in order**, stopping at the first that decides:

1. **Lowest Root Bridge ID** — who is root at all.
2. **Lowest Root Path Cost** — of the paths to root, which is cheapest.
3. **Lowest Sender Bridge ID** — if costs tie, which neighbour is preferable.
4. **Lowest Sender Port ID** — if even that ties (two links to the same neighbour), which of its ports.

Then roles fall out:

- **Root Port** — the one port on each non-root switch with the best path to root. Every non-root switch has exactly one.
- **Designated Port** — on each segment, the one port responsible for forwarding onto it. Every segment has exactly one. All ports on the root bridge are designated.
- **Blocking / Non-designated** — everything else.

Why does step 4 exist at all — when would two links tie on everything else? When a switch has **two parallel links to the same neighbour**. Same root, same cost, same sender Bridge ID — the first three tie-breakers are useless. Port ID is the final arbiter, and it's why the lower-numbered port becomes the root port and the other blocks. (If you'd rather bundle them than block one, that's exactly what EtherChannel in Chapter 10 is for.)

9.11 RSTP In Depth — Why It's Seconds Instead of Fifty

> **Depth section.** Rapid PVST+ is the modern default, so this is the behaviour you'll actually see.

Classic STP takes **30–50 seconds** to recover: 20 s max-age waiting to declare a BPDU stale, then 15 s listening, then 15 s learning. RSTP (802.1w) does the same job in **seconds**. It's not a faster timer — it's a different approach.

Fewer States, Clearer Roles

Classic STP state	RSTP state
Disabled / Blocking / Listening	**Discarding**
Learning	**Learning**
Forwarding	**Forwarding**

And two new **roles** that classic STP lacked:

Role	Meaning
Root	Best path to the root bridge
Designated	Forwards onto this segment
Alternate	A backup for the **root port**, learned from another switch
Backup	A backup for a **designated port**, on the same shared segment

This is the core of the speedup. Classic STP knew a port was blocked but not why or what it could replace. RSTP pre-computes the answer: an **alternate port** is already identified as "the standby route to root." When the root port fails, the alternate is promoted **immediately** — no waiting, no re-election, because the decision was made in advance. It's the same pre-warmed-backup idea as the OSPF BDR: the work is done before the failure, not after.

Alternate vs Backup trips people up. An **alternate** hears superior BPDUs from a different switch — it's a genuinely different path to root. A **backup** hears superior BPDUs from itself (two ports of the same switch on one shared segment, typically via a hub) — it's a spare on the same path, and is rare in modern switched networks.

Proposal / Agreement — the Handshake That Replaces Waiting

On a point-to-point link, two RSTP switches negotiate explicitly instead of waiting out timers:

1. **SW1 → SW2: proposal** — "I intend to be the designated port here."
2. **SW2 syncs** — it blocks its own other non-edge ports, so it cannot possibly create a loop no matter what SW1 does next.
3. **SW2 → SW1: agreement** — "Agreed, go ahead."
4. **SW1 forwards immediately** — no timers waited out.

Why is skipping the timers safe here, when classic STP insisted on them? Because the timers were a substitute for information. Classic STP had no way to ask "is it safe to forward?", so it waited long enough for any conflicting news to arrive. RSTP just **asks**, and the neighbour blocks its own downstream ports (the **sync**) before answering. Safety comes from an explicit agreement rather than from elapsed time.

This is also why **link type** matters:

- **Point-to-point** (full duplex) → handshake available → fast.
- **Shared** (half duplex, hub) → multiple neighbours, no single peer to negotiate with → falls back to timers.
- **Edge** (PortFast, host-facing) → forwards immediately.

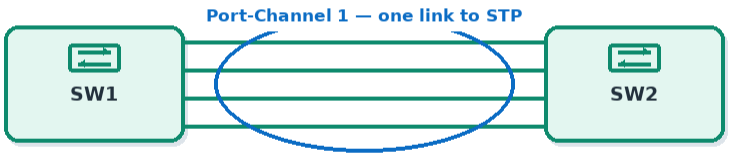
So a duplex mismatch doesn't just hurt throughput — it can silently drop a link into shared mode and cost you rapid convergence.

Finally, RSTP handles **topology change** differently. Classic STP notified the root, which set a flag causing everyone to age out MAC tables in 15 seconds. RSTP lets **any** switch flood a topology change directly and **flushes** affected MAC entries outright rather than aging them — so traffic re-learns its path at once instead of blackholing while a stale table expires.

Chapter 10: EtherChannel — Bundling Cables

10.1 The Idea

EtherChannel — four cables, one logical link (no STP blocking inside)



each flow hashes onto ONE member link — one download won't go faster

What if one cable between switches isn't fast enough? **EtherChannel** bundles **multiple physical cables** into **one logical link**. More speed, and if one cable dies, the others keep working!

Without EtherChannel, four cables between two switches give you the speed of **one**, because STP blocks the other three to prevent a loop. Bundle them into an EtherChannel and STP sees a **single logical link** — so all four carry traffic.

Bonus: STP sees the bundle as a **single link**, so it won't block the extra cables. You get all the bandwidth AND redundancy.

This "bonus" is actually the WHOLE POINT — let's understand WHY EtherChannel exists. Here's the problem it solves: imagine you connect two switches with 4 cables for more speed. What does STP do? Remember Chapter 9 — STP sees 4 paths between the same two switches as a **loop**, so it **blocks 3 of them!** You wanted 4× the bandwidth, but STP leaves you with just **1 active cable** and 3 sitting idle. Frustrating!

From STP's point of view, four parallel links between SW1 and SW2 are four chances to form a loop. It forwards on **one** and blocks the other **three**. You paid for four cables and got the speed of one.

EtherChannel's clever trick: it makes STP see the 4 cables as **ONE logical link**, so there's no loop to block — and all 4 cables carry traffic together. That's the "why": EtherChannel is the peace treaty between "I want more bandwidth (many cables)" and "STP blocks redundant cables." It gets you the bandwidth of all the cables **and** keeps the redundancy (if one cable dies, the bundle just keeps going on the rest, with no STP recalculation delay).

Why does losing a cable NOT cause an outage? Because to STP and to the switch's logic, nothing changed — it's still "one link," just now made of 3 cables instead of 4. Traffic keeps flowing on the survivors instantly. Compare that to STP failover (unblocking a backup port), which takes seconds. EtherChannel failover is nearly instant because the "backup" was already active and part of the bundle.

10.2 Two Ways to Negotiate

- **PAgP** (Port Aggregation Protocol): Cisco's own. Modes: `desirable`, `auto`.
- **LACP** (802.3ad): The industry standard (works with any vendor). Modes: `active`, `passive`.
- **Static "on":** Force it with no negotiation.

Why have a negotiation protocol at all — why not just force the bundle "on"? Because bundling cables only works if **both ends agree and are configured compatibly**. If one switch bundled 4 ports but the other didn't, you'd get chaos — mismatched links, possible loops, and errors. PAgP/LACP make the two switches **check with each other first** ("Hey, want to bundle these 4 ports? Are your settings compatible?") before forming the channel. It's a safety handshake. **Static "on" skips the handshake** — faster to set up, but risky: if you fat-finger the config on one side, there's no negotiation to catch the mistake, and you can create a loop. That's why LACP (with negotiation) is generally preferred over static.

Why prefer LACP over Cisco's PAgP? Because LACP is the **open industry standard (802.3ad)** — it works between Cisco and non-Cisco gear (Arista, HP, etc.). PAgP is Cisco-only. In a mixed network, LACP is the safe universal choice.

Which combos form a channel?

Side A	Side B	Forms?
LACP active	LACP active	Yes
LACP active	LACP passive	Yes
LACP passive	LACP passive	No (both just wait)
PAgP desirable	PAgP desirable	Yes
PAgP desirable	PAgP auto	Yes
PAgP auto	PAgP auto	No

10.3 Configuring EtherChannel (LACP)

```
SW1(config)# interface range gi0/1 - 2      ! the two physical ports
SW1(config-if-range)# channel-group 1 mode active   ! LACP active
SW1(config-if-range)# exit
SW1(config)# interface port-channel 1       ! configure the bundle as one
SW1(config-if)# switchport mode trunk
```

> **Rule:** All bundled ports must have the **same** settings (speed, duplex, VLANs, mode). If they don't match, the channel won't form.

Check it:

```
SW1# show etherchannel summary
```

10.4 How EtherChannel Actually Splits Traffic (And Why One Download Won't Go Faster)

> **Depth section.** Explains the single most misunderstood thing about link bundles.

Section 10.1 said an EtherChannel of four 1 Gbps links gives you "4 Gbps." That's true in aggregate — and misleading for any single conversation. Here's the mechanism.

An EtherChannel does **not** send packet 1 down link 1, packet 2 down link 2, and so on. Instead it runs a **hash** over selected header fields and uses the result to pick a link:

The switch takes selected header fields — source/destination MAC, or IP, or port numbers — runs them through a **hash**, and uses the result to pick one of the member links. Same conversation, same hash, **same physical link every time**.

Because the same conversation always hashes to the same value, **every frame in a given flow takes the same physical link**.

Why deliberately pin a flow to one link instead of spraying packets across all four? Because round-robin would **reorder** packets. Links are never perfectly equal — different queue depths, different momentary load — so packets sent in order across four links arrive out of order. TCP interprets out-of-order arrival as loss, fires duplicate ACKs, retransmits, and cuts its congestion window. You'd have built a 4 Gbps bundle that performs worse than a single link. Hashing trades per-flow speed for **in-order delivery**, which is the right trade.

The consequence to remember: one file transfer between two hosts uses **one link** and is capped at that link's speed. EtherChannel scales many conversations, not one. A backup server pushing a single huge stream sees no benefit at all.

Cisco lets you choose which fields feed the hash:

```
SW(config)# port-channel load-balance src-dst-ip     ! common on L3 boundaries
SW(config)# port-channel load-balance src-dst-mac    ! typical default
SW# show etherchannel load-balance
SW# show etherchannel summary          ! P = in port-channel, (SU) = in use
```

Why does the choice matter? Consider a switch uplinking to a router: every frame leaving the subnet has the **router's MAC** as its destination, so a MAC-based hash sees almost no variation and dumps nearly everything on one link. Switching to `src-dst-ip` restores variety, because the real destinations differ even though the next-hop MAC doesn't. Pick the fields that actually vary in your traffic — otherwise the bundle silently runs on one strand.

Powers of two matter too. With 2, 4 or 8 links the hash space divides evenly. With 3, 5, 6 or 7, some links receive a larger share of hash buckets than others and the load sits permanently lopsided.

Chapter 11: IP Addresses — The Home Address of Every Device

11.1 What Is an IP Address?

An **IP address** is the **logical address** of a device on a network — like a **home address** for mail. Unlike a MAC address (permanent name), an IP address can **change** and works **across the whole internet**.

IPv4 addresses look like this: `192.168.1.10`. They're **32 bits** long, split into **4 parts** (called **octets**), each 8 bits, separated by dots.

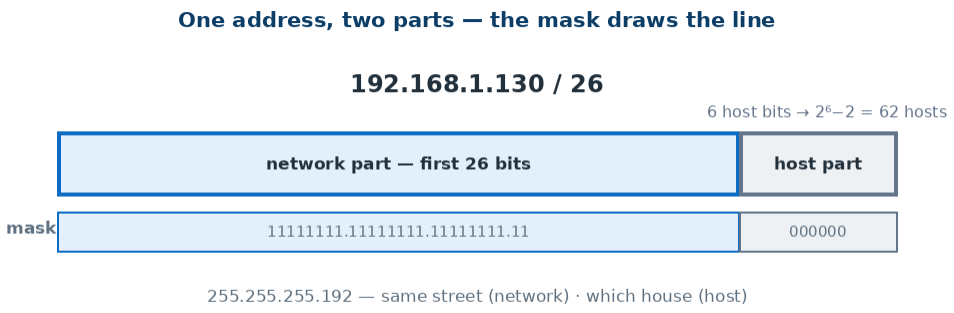
An IPv4 address is **four octets**, each 0–255:

Format	octet 1	octet 2	octet 3	octet 4
Decimal	192	168	1	10
Binary	11000000	10101000	00000001	00001010

Four octets × 8 bits = **32 bits** in total.

Each octet is one byte (8 bits), so it can be **0 to 255**. (Remember from Chapter 4: 11111111 = 255.)

11.2 Two Parts: Network & Host



Every IP address has two parts:

- **Network part:** Which network you're on (like a street name).
- **Host part:** Which specific device you are (like a house number).

The **subnet mask** tells us where the split is. More in Chapter 12!

In `192.168.1.10`, the address splits into two parts:

Part	Value	Think of it as
Network	`192.168.1`	The street

Part	Value	Think of it as
Host	`10`	The house on that street

Why split an address into "network" and "host" at all? Why not just give every device one flat number? This is the single design choice that makes the internet possible, so it's worth really understanding. Imagine if IP addresses had **no structure** — just random serial numbers like MAC addresses. Then a router trying to reach a device would need a list of **every single device on Earth** (billions of entries) to know where to send traffic. No router could hold that, and it could never keep up.

The network/host split fixes this with the same trick the **postal system** uses. The mail sorter in New York doesn't memorize every person in California. It just knows "anything addressed to California → send it west." Only when the letter arrives in the right town does someone look at the specific street and house.

IP works identically:

- Routers only care about the **network part** ("this address is in the 192.168.1 network → send it that way"). They keep a small list of networks, not devices — millions of times smaller.
- Only the **final** switch on the destination network cares about the **host part** ("ah, device .10 specifically").

So the split is what lets routers stay small and fast: they think in terms of **neighborhoods (networks)**, not individual houses (hosts). This is called **hierarchical addressing**, and it's why a handful of routing entries can reach the entire internet. Everything in Chapter 12 (subnetting) is just you deciding where to draw that network/host line to make neighborhoods the right size.

11.3 IP Address Classes (The Old System)

Long ago, IP addresses were divided into **classes** by their first number:

Class	First Octet Range	Default Mask	Use
A	1 – 126	255.0.0.0 (/8)	Huge networks
B	128 – 191	255.255.0.0 (/16)	Medium networks
C	192 – 223	255.255.255.0 (/24)	Small networks
D	224 – 239	—	Multicast (groups)
E	240 – 255	—	Experimental

> Note: **127** (like 127.0.0.1) is reserved for **loopback** — a device talking to itself ("localhost"). It's the "am I alive?" test address.

Why did classes exist, and WHY were they abandoned? Classes were an early, simple way to decide the network/host split just by looking at the first number. Easy — but incredibly **wasteful**, and that waste is why they died:

- A **Class A** network gives you **16 million** host addresses. If a company needed, say, 5,000 addresses, they'd get a Class A (16 million) or a Class B (65,000) — and **waste millions of addresses** they'd never use.
- With only ~4 billion IPv4 addresses total, handing them out in giant fixed chunks burned through them frighteningly fast.

So the industry switched to **classless addressing (CIDR)** — the idea that you can put the network/host split **anywhere**, not just on the class boundaries. That company needing 5,000 addresses can get exactly the right-sized block instead of a wasteful giant one. **That flexibility to choose the split is literally what subnetting is** (Chapter 12), and it's why we barely use classes today except to explain the history and the private-range groupings.

11.4 Private vs. Public IP Addresses

- **Public IPs:** Unique on the whole internet. Given out by authorities. Like a real street address anyone can mail to.
- **Private IPs:** Free to use inside your own network, NOT routable on the internet. Many networks reuse them. A router uses **NAT** (Chapter 17) to share one public IP.

Private ranges to memorize:

Class	Private Range
A	10.0.0.0 – 10.255.255.255
B	172.16.0.0 – 172.31.255.255
C	192.168.0.0 – 192.168.255.255

Story Time: Private IPs are like apartment numbers **inside** a building. Lots of buildings have an "Apt 1," so they're not unique to the world. The building's **street address** (public IP) is what the outside world sees. NAT is the front-desk clerk who forwards mail between the two.

Why do private IPs even exist? WHY not just give every device a public one? Because **we ran out.** IPv4 has only ~4.3 billion addresses, and there are way more than 4.3 billion phones, laptops, TVs, and gadgets. If every device needed a unique public address, we'd have run dry decades ago. Private addresses are the brilliant workaround: **billions of home and office networks all reuse the same private ranges** (that's why your home network and your neighbor's both use 192.168.1.x — and it's fine, because they never directly touch). The whole building shares **one** public address to face the internet, and **NAT** (Chapter 17) translates between the shared public address and the many private ones. So private IPs exist to stretch a limited supply of public addresses across an unlimited number of devices — a patch that (along with IPv6) kept the internet growing.

11.5 Special Addresses

- **Network address:** The **first** address in a subnet (all host bits = 0). Names the network. Can't be given to a device.

- **Broadcast address:** The **last** address (all host bits = 1). Sends to everyone on the subnet. Can't be given to a device.
- **Loopback:** 127.0.0.1 — yourself.
- **APIPA:** 169.254.x.x — a device gives itself this when it can't reach a DHCP server (a "something's broken" sign).

Why can't you assign the first (network) and last (broadcast) addresses to a device? Because each one is already "spoken for" as a special meaning, and giving it to a device would create confusion:

- The **network address** (all host bits 0) is the name of the whole subnet itself — it's how routers refer to the group ("the 192.168.1.0 network"). If you also gave it to a PC, there'd be an ambiguity: does "192.168.1.0" mean the network or that one PC? So it's reserved as the label for the group.
- The **broadcast address** (all host bits 1) is the "**everyone on this subnet**" address. When a device sends here, every device on the subnet must receive it. If you assigned it to a single PC, then "send to everyone" and "send to that one PC" would be the same address — chaos. So it's reserved for the shout-to-all function.

This is exactly why the host formula is 2^h minus 2: every subnet loses its first address (the network name) and its last address (the broadcast) as usable host addresses. Now that "−2" isn't a random rule you memorize — you know why those two are off-limits.

11.6 Default Gateway

The **default gateway** is the **router's address** that a device uses to reach **other networks**. It's the "door out of your neighborhood."

A PC at `192.168.1.10` that wants something outside its own network hands the packet to its **default gateway**, `192.168.1.1` — the router — which forwards it on toward the internet. The PC itself needs to know nothing beyond that one address.

If a PC wants to talk to a device on its **own** network, it sends directly. If the destination is on a **different** network, it sends to the **default gateway** to forward.

But HOW does the PC decide "same network" vs "different network"? And WHY does it need a gateway for one but not the other? The PC does a quick piece of math with its **subnet mask** every time it sends: it compares the network part of its own IP with the network part of the destination IP.

- **Same network part?** ("You live on my street.") The PC can reach the destination **directly** — it just ARPs for the destination's MAC and sends the frame straight over. No router needed, because they share the same local wire/broadcast domain.
- **Different network part?** ("You live in another town.") The PC has **no way to reach another network by itself** — Layer 2 only reaches the local wire. So it hands the frame to its **default gateway** (the router) and says "please forward this onward." The router, whose whole job is connecting networks, takes it from there.

This is why a device with no default gateway can still talk to its local neighbors but can't reach the internet — it has no "door" to hand off-network traffic to. It's also why a wrong gateway address is

such a common real-world problem: local stuff works, but nothing outside does, which is a huge troubleshooting clue (you'll see this in Chapter 23).

But hold on — what if the gateway itself dies? Every device on the subnet is pointed at that one address, so if that router fails, the whole subnet loses the outside world at once. That's a big enough problem to have its own family of protocols; we'll solve it properly in section **14.7 (FHRP/HSRP)** once you've met routers in detail.

Chapter 12: Subnetting — Cutting the Network Cake

12.1 What Is Subnetting?

Subnetting means **splitting one big network into smaller networks** (subnets). It's one of the MOST important CCNA skills. Don't worry — we'll make it painless.

Story Time: Imagine you have one giant pizza (a network). If the whole class shares one pizza, it's chaos. Instead, you **cut it into slices** (subnets) so each group gets its own slice. Subnetting is just deciding **where to cut**.

But WHY cut the network up at all? Why not leave everything in one big network? Two powerful reasons, and they're the same problems you met earlier — now with a fix:

1. **Broadcast control (performance).** Remember: every device in a network hears every broadcast. Put 1,000 devices in one flat network and each device is constantly interrupted by everyone else's ARP/DHCP shouting — the network crawls. Subnetting slices that into smaller broadcast domains (e.g., ten groups of 100), so each device only hears its own group. Smaller pizza slices = less noise per person.
2. **Security & organization.** Separate subnets let you put a **router (with rules)** between groups — so you control exactly who reaches what (students can't reach payroll). Same idea as VLANs (Chapter 7); in fact, each VLAN gets its own subnet.
3. **Stopping waste.** Instead of assigning a giant network to a tiny office (wasting thousands of addresses), you cut a slice that's just the right size. This is the classless flexibility that replaced the wasteful old classes.

So subnetting isn't math for math's sake — it's how you keep networks fast, secure, and efficient. The math is just the tool for deciding **where to cut**.

> **Pair this chapter with the Subnetting Drill Sheet.** Subnetting is the one CCNA skill that has to be fast, not just correct — the drill sheet's 60 worked problems are built to turn the methods below into reflexes. Read here, drill there.

12.2 The Subnet Mask

The **subnet mask** decides which part of an IP is "network" and which is "host." It looks like an IP address but is made of **1s then 0s**:

Line the address up against the mask:

Item	Value
IP	`192.168.1.10`
Mask	`255.255.255.0`
Mask in binary	`11111111.11111111.11111111.00000000`

The **24 ones** mark the network portion; the **8 zeros** mark the host portion.

- Wherever the mask has a **1**, that part is **network**.
- Wherever the mask has a **0**, that part is **host**.

WHY does a "mask" made of 1s and 0s work — what's actually happening? The word mask is literal: it's like a stencil laid over the IP address that **covers up** part of it. The computer does a bit-by-bit **AND** operation between the IP and the mask, and the result is the **network address**. Here's the intuition without heavy math:

- Where the mask is **1**, the IP's bit "shows through" → that's the **network** part (it's kept).
- Where the mask is **0**, the IP's bit gets "blocked out" to 0 → that's the **host** part (erased to find the network's starting address).

So a device can instantly find "what network does this address belong to?" by applying the mask. That's the whole point of the mask existing — it's the tool every device uses to answer the "same network or different?" question from Chapter 11.

And WHY must the mask be all 1s THEN all 0s — never mixed like 11010011? Because the network part has to be **contiguous** (one solid block on the left). Remember hierarchical addressing: routers summarize whole networks by their leading bits ("everything starting with 192.168.1 goes this way"). That only works if all the network bits are grouped together at the front. A scattered mask like `11010011` would make it impossible to describe a network as a clean "starts-with" prefix, breaking route summarization. So the "1s then 0s" rule isn't arbitrary — it's what keeps addresses summarizable and routing efficient. This is also why we can shorten a mask to a single number (the count of 1s) — the CIDR "/24" notation you'll see next.

12.3 CIDR / Slash Notation

Instead of writing `255.255.255.0`, we count the 1s and write **/24**. This is called **CIDR** (say "sider") or **prefix notation**.

Mask	Slash	# of 1s
255.0.0.0	/8	8
255.255.0.0	/16	16

Mask	Slash	# of 1s
255.255.255.0	/24	24
255.255.255.128	/25	25
255.255.255.192	/26	26
255.255.255.224	/27	27
255.255.255.240	/28	28
255.255.255.248	/29	29
255.255.255.252	/30	30

Why can we get away with just writing a single number like "/24"? Because of the "1s then 0s" rule you just learned! Since the 1s are always packed together on the left, the **only** thing that varies is how many of them there are. Once you know the count (24), you know the entire mask (24 ones, then 8 zeros). No information is lost. That's why /24 and 255.255.255.0 are exactly the same thing — /24 is just the shorthand, and it's why engineers prefer it (faster to write, easier to compare: /26 is obviously "more network bits" than /24).

12.4 How Many Hosts? The Magic Formula

The number of usable devices in a subnet:

```
Usable hosts = 2^(host bits) - 2
```

Why minus 2? Because you **can't use** the **network address** (first) or the **broadcast address** (last).

Example: A /24 has 8 host bits (32 − 24 = 8).

```
2^8 - 2 = 256 - 2 = 254 usable devices
```

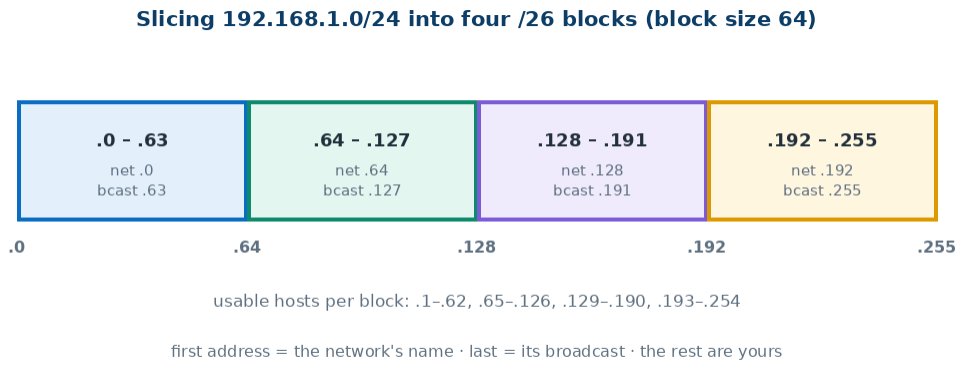
Example: A /26 has 6 host bits (32 − 26 = 6).

```
2^6 - 2 = 64 - 2 = 62 usable devices
```

Powers of 2 (memorize these!)

Power	Value		Power	Value
2^1	2		2^5	32
2^2	4		2^6	64
2^3	8		2^7	128
2^4	16		2^8	256

12.5 The "Block Size" Trick (Subnetting Made Easy)



The easiest way to subnet: find the **block size** (how big each subnet is).

Block size = 256 − (the interesting octet of the mask).

The "interesting octet" is the last octet in the mask that isn't 255 or 0.

Why does this trick work? Why 256 minus the mask value? Because the "block size" is literally how far apart each subnet starts — and that spacing is controlled by the lowest network bit. Here's the intuition: in the interesting octet, the mask value (like 192) uses up the high bits for the network, leaving the low bits for hosts. The value **256 − 192 = 64** is exactly the value of that **lowest network bit** — the smallest "step" you can take before you roll into the next subnet. So subnets land at multiples of 64 (0, 64, 128, 192) because each one is 64 addresses "wide." The block size is just the size of one slice, and 256 − mask gives it to you instantly without drawing out all the binary. That's why this shortcut is a lifesaver on the timed exam — it turns binary work into simple counting.

Worked Example: 192.168.1.0 /26

1. **Mask /26** = 255.255.255.**192**. Interesting octet = 192.
2. **Block size** = 256 − 192 = **64**.
3. **Subnets start at multiples of 64:** 0, 64, 128, 192.

Now list them:

Subnet	Network	First Host	Last Host	Broadcast
#1	.0	.1	.62	.63
#2	.64	.65	.126	.127
#3	.128	.129	.190	.191
#4	.192	.193	.254	.255

How to read each row:

- **Network** = the block start (.0, .64, ...).
- **Broadcast** = one below the next block (.63 is just below .64).
- **First host** = network + 1.
- **Last host** = broadcast − 1.

Try It: Which subnet is 192.168.1.100 /26 in?

Block size 64 → subnets at 0, 64, 128, 192. 100 falls between **64 and 127**, so it's in the **.64 subnet** (hosts .65–.126, broadcast .127). Done!

12.6 A Full Subnetting Walkthrough

Question: You have `172.16.0.0/16` and need **subnets with at least 500 hosts each**. What mask?

1. Need 500 hosts → find host bits: 2^9 = 512 (≥ 502 after −2). So **9 host bits**.
2. Total bits = 32. Network bits = 32 − 9 = **23**. So mask = **/23** (255.255.254.0).
3. Each /23 gives 2^9 − 2 = **510 hosts.**

12.7 VLSM — Different-Sized Slices

VLSM (Variable Length Subnet Mask) means making subnets of **different sizes** to avoid waste. Give big departments big subnets and tiny point-to-point links tiny subnets.

Story Time: Cutting a pizza into equal slices wastes food if some people are only a little hungry. VLSM lets you cut big slices for hungry people and small slices for snackers — no waste!

Example plan for 192.168.1.0/24:

Group	Hosts Needed	Subnet	Mask	Range
Sales	100	192.168.1.0	/25 (126 hosts)	.0–.127
IT	50	192.168.1.128	/26 (62 hosts)	.128–.191
HR	25	192.168.1.192	/27 (30 hosts)	.192–.223
Link	2	192.168.1.224	/30 (2 hosts)	.224–.227

Golden VLSM rule: Always assign the **biggest** subnet first, then smaller ones. This prevents overlap.

Why must you assign biggest-first? What actually goes wrong otherwise? Because subnets can only start on **boundaries that match their size** (a /25 must start at .0 or .128; a /26 at .0/.64/.128/.192; and so on). Big subnets have **few** legal starting points, while small subnets fit almost anywhere. So if you place the big, picky subnets **first**, they grab their valid boundaries while everything is still open. If you place small subnets first, they scatter around and **block the boundaries the big subnet needed** — now your /25 has nowhere legal to fit, even though there's technically enough total space. It's like packing a suitcase: put the big rigid items in first, then tuck small soft things around them. Do it backwards and the big item won't fit. That's the whole reason for the rule — it's about respecting alignment boundaries, not just tidiness.

12.8 The /30 and /31 — Tiny Subnets for Router Links

A **/30** gives exactly **2 usable hosts** — perfect for a cable between two routers (each end needs one IP). A **/31** is a special case that gives 2 hosts with no waste (used on point-to-point links). Memorize: **/30 = 2 hosts.**

Why use a tiny /30 for a router-to-router link instead of a normal /24? Because a point-to-point link between two routers has **exactly two devices** on it — one router on each end. A /24 would reserve 254 host addresses for a link that can only ever use 2, **wasting 252 addresses**. Multiply that across dozens of WAN links in a real network and you've thrown away thousands of addresses for nothing. A /30 gives you precisely the 2 addresses you need (plus its own network and broadcast). This is VLSM's whole philosophy in action: match the slice size to the actual need. The /31 takes it even further — a special rule (RFC 3021) that squeezes a point-to-point link down to literally 2 addresses with no network/broadcast waste at all, because on a link with only two ends, "broadcast" and "unicast to the other guy" mean the same thing anyway.

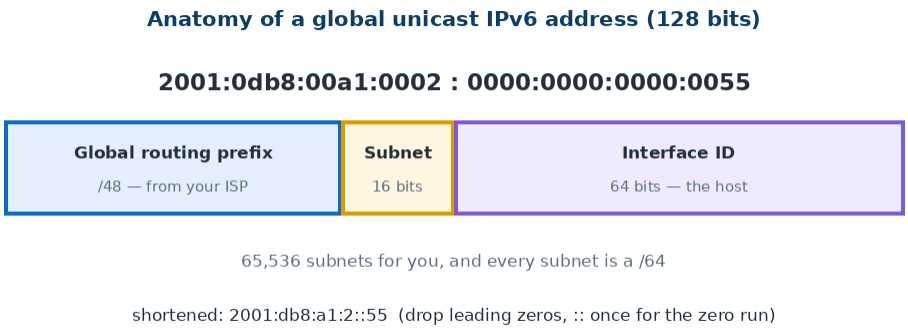
Chapter 13: IPv6 — The New, Giant Address System

13.1 Why IPv6 Exists

IPv4 has about **4.3 billion** addresses. That sounds like a lot, but with phones, laptops, TVs, watches, and fridges all online, **we ran out!**

IPv6 fixes this with a mind-bogglingly huge number of addresses: **340 undecillion** (that's 340 followed by 36 zeros). Enough for every grain of sand to have trillions of addresses.

13.2 What IPv6 Looks Like



IPv6 is **128 bits** (four times bigger than IPv4's 32 bits). It's written in **hexadecimal**, in **8 groups** of 4 hex digits, separated by **colons**:

An IPv6 address is written as **8 groups of 4 hex digits**, separated by colons:

`2001:0db8:0000:0000:0000:ff00:0042:8329`

Each group is 16 bits, so 8 × 16 = **128 bits** in total.

13.3 Shortening IPv6 (The Two Rules)

IPv6 is long, so there are two rules to shorten it:

Rule 1 — Drop leading zeros in each group:

```
2001:0db8:0000:0000:0000:ff00:0042:8329
2001:db8:0:0:0:ff00:42:8329        (leading zeros removed)
```

Rule 2 — Replace ONE run of all-zero groups with `::` (double colon), only once:

```
2001:db8:0:0:0:ff00:42:8329
2001:db8::ff00:42:8329             (the three 0 groups became ::)
```

> **Important:** You can use `::` **only once** per address (otherwise it's ambiguous how many zeros it hides).

Why can `::` only be used ONCE? Because the whole trick works by counting: when you see `::`, you figure out how many zero-groups it hides by subtracting the visible groups from 8. If an address had **two** `::`, you'd have no way to know how to split the zeros between them — `2001::5::1` could mean several different addresses. One `::` keeps it unambiguous: "fill the gap with exactly enough zeros to reach 8 groups." That's why the rule exists — it's what makes the shorthand reversible.

Why do these shortening rules exist at all? Because IPv6 addresses are **enormous** (128 bits = 32 hex digits), and real addresses are full of zeros (the designers deliberately left big zero-filled gaps for future structure). Writing `2001:0db8:0000:0000:0000:0000:0000:0001` by hand every time would be miserable and error-prone. The rules let you write `2001:db8::1` instead — same address, far less to type and mistype. It's purely a human-convenience feature; the computer always uses the full 128 bits internally.

Try It: Shorten `2001:0db8:0000:0000:0000:0000:0000:0001`

- Drop leading zeros: `2001:db8:0:0:0:0:0:1`
- Collapse the zeros: `2001:db8::1`

13.4 Types of IPv6 Addresses

Type	Starts With	Meaning
Global Unicast	2000::/3	Public, internet-routable (like a public IPv4)
Link-Local	FE80::/10	Only works on the local link; auto-made on every interface
Unique Local	FC00::/7	Private, like IPv4 private addresses
Multicast	FF00::/8	One-to-many group
Loopback	::1	Yourself (like 127.0.0.1)
Unspecified	::	"No address yet"

> **Note:** IPv6 has **no broadcast!** It uses **multicast** instead. This is a favorite exam fact.

Why did IPv6 KILL the broadcast? Because broadcasts are **rude and wasteful** — a broadcast forces every device on the network to stop what it's doing, pull in the frame, and inspect it, even if the message has nothing to do with them. On a busy IPv4 network, all that broadcast interruption adds up to real wasted CPU and bandwidth (you saw this pain in the VLAN chapter). IPv6's designers looked at this and said: why bother everyone when we can talk only to the devices that care? So IPv6 replaces broadcast with **targeted multicast** — messages go only to the specific group that signed up to listen. For example, instead of IPv4's ARP (a broadcast that yells at everyone "who has this IP?"), IPv6 uses **Neighbor Discovery** with a multicast that reaches only the one relevant device. Same job, far less interruption. Fewer devices bothered = a more efficient network. That's the "why" behind one of the exam's favorite facts.

Why does every IPv6 interface automatically make a "link-local" (FE80::) address? Because IPv6 was designed so devices can **always talk to their direct neighbors instantly**, even before any DHCP, router, or manual config exists. The link-local address is a self-assigned "I can at least reach the devices on my own wire" address. Why is that useful? It lets crucial startup processes work immediately — like discovering the router, or two devices coordinating — without waiting for any address setup. It's the networking equivalent of being able to talk to people in the same room without needing a phone number yet. (Link-local only works on the local link — routers never forward it — which is exactly why it's safe to auto-generate on every interface.)

13.5 EUI-64 — Making a Host Part from a MAC

IPv6 can build the second half of its address automatically from the device's MAC address using **EUI-64**:

1. Split the 48-bit MAC in half.
2. Insert `FFFE` in the middle (making 64 bits).
3. Flip the 7th bit of the first byte.

Step	Result
Start with the MAC	`00:1A:2B:3C:4D:5E`
Split it and insert `FF:FE` in the middle	`00:1A:2B:FF:FE:3C:4D:5E`
Flip the 7th bit of the first byte (`00` → `02`)	`02:1A:2B:FF:FE:3C:4D:5E`
Interface ID	`021A:2BFF:FE3C:4D5E`

13.6 Configuring IPv6 on a Router

```
R1(config)# ipv6 unicast-routing        ! turn on IPv6 routing
R1(config)# interface gi0/0
R1(config-if)# ipv6 address 2001:db8:0:1::1/64   ! set an IPv6 address
R1(config-if)# ipv6 address fe80::1 link-local   ! optional link-local
R1(config-if)# no shutdown
```

13.7 How Devices Get IPv6 Addresses

- **SLAAC** (Stateless Address Auto-Configuration): The device makes its own address by listening to the router. No DHCP needed!
- **DHCPv6:** A server hands out addresses (stateful) or just extra info like DNS (stateless).
- **Static:** You type it in by hand.

13.8 NDP & SLAAC In Depth — How IPv6 Replaced ARP and DHCP

> **Depth section.** Section 13.7 said devices "can configure themselves." This is the mechanism — and it's a favourite CCNP topic.

IPv6 deleted two things you rely on in IPv4: **ARP** and **broadcasts**. Both were replaced by **NDP (Neighbor Discovery Protocol)**, which runs over ICMPv6.

The Five NDP Messages

Type	Message	Purpose
133	Router Solicitation (RS)	"Any routers here?"
134	Router Advertisement (RA)	"I'm a router, here's the prefix"
135	Neighbor Solicitation (NS)	"Who has this address?" (ARP's job)
136	Neighbor Advertisement (NA)	"I do, here's my MAC"
137	Redirect	"There's a better first hop for that"

Why Multicast Instead of Broadcast

IPv4's ARP shouts to the **broadcast** address, so **every** device on the segment must interrupt its CPU, inspect the packet, and almost always discard it. On a busy VLAN that's constant wasted work for everyone.

IPv6 sends its NS to a **solicited-node multicast** address, built from the last 24 bits of the target address:

Take the target address `2001:db8::a1b2:c3d4`, keep its **last 24 bits** (`b2:c3d4`), and append them to the fixed prefix `FF02::1:FF` — giving the solicited-node address **`FF02::1:FFb2:c3d4`**.

Why does this help, when it's still one-to-many? Because a NIC filters multicast **in hardware**. A host joins only the group matching its own address, so its network card silently ignores everything else — the CPU is never even notified. Since the group is derived from the low 24 bits, the odds of two hosts sharing it on one segment are tiny. Effectively, only the intended target wakes up. Same question as ARP, asked without bothering the whole room.

SLAAC — Building Your Own Address

1. The host boots and forms a **link-local** address (`FE80::/10` + its interface ID).
2. **DAD:** it sends an NS to its own tentative address — silence means nobody else has it, so it's safe to use.
3. It sends an **RS** to `FF02::2` (all-routers).
4. A router replies with an **RA**: "the prefix here is `2001:db8:a:b::/64`."
5. The host builds its address: **prefix + its own interface ID**.
6. It runs **DAD** once more, on the new address.

Why does the host generate the host portion itself rather than being assigned one? Because a /64 subnet holds 18 quintillion addresses. Collisions are so improbable that central bookkeeping — the entire reason DHCP exists — stops being worth it. The scarcity that made DHCP necessary in IPv4 simply doesn't exist here. **DAD (Duplicate Address Detection)** is the cheap safety check that makes self-assignment safe: ask for your own address, and silence is the confirmation.

The RA Flags Decide Everything

An RA carries flags that tell hosts how much autonomy they have:

Flag	Meaning	Result
A = 1	Autonomous	Build your own address by SLAAC
M = 1	Managed	Get your address from **stateful DHCPv6**
O = 1	Other config	Address by SLAAC, but get DNS etc. from DHCPv6

Why keep DHCPv6 at all if SLAAC works? Because SLAAC answers "what address should I use?" but originally said nothing about **DNS servers**, and it keeps **no record** of who has what. An

enterprise that must audit which device held which address — for security logs or compliance — needs the central ledger that only stateful DHCPv6 provides. SLAAC optimizes for zero administration; DHCPv6 optimizes for accountability. The M and O flags let you pick per network.

```
R1# show ipv6 neighbors            ! the NDP cache — IPv6's ARP table
R1# show ipv6 interface gi0/0      ! link-local, global, joined multicast groups
R1(config-if)# ipv6 nd prefix 2001:db8:a:b::/64 no-autoconfig   ! clear the A flag
```

Chapter 14: Routers & Routing — Finding the Path

14.1 What a Router Does

A **switch** moves data **inside** one network. A **router** moves data **between** different networks. It's the **GPS** of networking — it knows the paths to faraway networks and picks the best one.

A router sits between two different networks — say `192.168.1.0/24` on one side and `10.0.0.0/24` on the other — with an interface in each. It's the only device that knows how to reach **both**, so anything crossing between them goes through it.

14.2 The Routing Table

Every router keeps a **routing table** — a list of known networks and how to reach them. When a packet arrives, the router looks up the destination and forwards it toward the right next hop.

```
R1# show ip route
C   192.168.1.0/24 is directly connected, Gi0/0
C   10.0.0.0/24    is directly connected, Gi0/1
S   172.16.0.0/24  [1/0] via 10.0.0.2
O   192.168.5.0/24 [110/2] via 10.0.0.2
```

The letters on the left tell you how the route was learned:

- **C** = Connected (a network directly plugged into the router).
- **L** = Local (the router's own interface IP).
- **S** = Static (typed in by an admin).
- **O** = OSPF (learned dynamically).
- **D** = EIGRP. **R** = RIP. **B** = BGP.

14.3 How a Router Decides: Longest Prefix Match

If two routes could match a destination, the router picks the **most specific** one — the one with the **longest prefix** (biggest /number).

Example: A packet to 192.168.1.55 could match:

- `192.168.0.0/16` (broad)
- `192.168.1.0/24` (specific) ← **winner!** More exact.

Story Time: It's like directions. "Go to California" vs. "Go to 123 Maple St, San Diego, CA." The **more specific** address gets you exactly where you need to go, so the router prefers it.

Why does "most specific" win — why is that the right rule? Because a longer prefix means the route is describing a **smaller, more exact group of addresses**, which almost always means someone configured it deliberately for that specific destination. Think about the directions analogy again: "Go to California" (a /16, broad) and "Go to 123 Maple St, San Diego" (a /24, exact) might both technically be correct — but the exact one reflects real, specific knowledge of where you're going. The broad route is a fallback for "everything in this general area," while the specific route is "I know exactly where this one goes." A router honoring the most specific match means it uses the **most precise information available** and only falls back to broader routes when it has nothing better. The ultimate example is the default route (0.0.0.0/0) — the least specific route possible ("everywhere else") — which only wins when literally nothing else matches. That's why longest-prefix-match and default routes work together so cleanly.

14.4 Administrative Distance (Who to Trust)

If a router learns about the **same** network from **two different sources**, which does it believe? It uses **Administrative Distance (AD)** — a **trust score** where **lower = more trusted**.

Source	AD (trust)
Connected	0 (most trusted)
Static	1
EIGRP	90
OSPF	110
RIP	120
Unknown/unreachable	255 (never used)

Memory trick: Lower AD = "closer to the truth." A directly connected network (AD 0) is undeniable — you can see it! A guess from RIP (AD 120) is less trusted.

Why do we even need AD — and why these specific rankings? Because a router might hear about the same destination from multiple sources at once (a static route AND OSPF AND RIP), and those sources can **disagree**. The router needs a consistent tie-breaker to decide who to believe. AD is that tie-breaker, and the rankings reflect **how trustworthy each source's information is**:

- **Connected = 0 (total certainty).** The network is physically plugged into this router. There's no guessing — the router can literally see the wire. You can't get more trustworthy than "I'm directly attached to it," so it wins over everything.
- **Static = 1 (a human said so).** An administrator deliberately typed this route. The logic is "a human's explicit instruction should override an automatic protocol's guess," so static beats the routing protocols. (It's 1, not 0, only because a real connected link is even more certain.)

- **The protocols (EIGRP 90, OSPF 110, RIP 120)** are automatic and can be wrong or slow to update, so they're trusted less than a human's static route. Among them, the ranking reflects how smart the protocol is: **EIGRP** (90) uses rich info (bandwidth + delay) and is very reliable; **OSPF** (110) is also smart (full map); **RIP** (120) is the dumbest (just counts hops), so it's trusted least. Better information = lower number = more trust.

So AD answers "WHO do I believe?" It's about the credibility of the messenger. Note this is different from the next concept, which answers "WHICH PATH is best?" — keep them separate!

14.5 Metric (Which Path Within One Protocol)

If one routing protocol knows **two paths** to the same place, it uses a **metric** to pick the best. Each protocol measures differently:

- **RIP:** hop count (fewest routers).
- **OSPF:** cost (based on bandwidth — faster links preferred).
- **EIGRP:** bandwidth + delay.

Why is metric SEPARATE from Administrative Distance — don't they do the same thing? No, and keeping them straight is a classic exam point. They answer two different questions, in order:

1. **AD picks the SOURCE first** ("who do I trust most?"). If both OSPF and a static route describe a destination, AD decides which routing method wins — before metric is even considered.
2. **Metric picks the PATH within that winning source** ("of the routes from my trusted source, which physical path is best?").

Think of planning a trip: **AD** is choosing which navigation app to trust (the reliable one vs. the flaky one). **Metric** is that chosen app then picking the best route among several it found. You pick the app first, then the app picks the road. You never compare a road from one app against the trust-level of another app — different questions, done in sequence.

Why do different protocols use different metrics? Because they were designed with different levels of "smart." **RIP** just counts routers (hops) — simple, but dumb: it would happily pick a 1-hop path over a slow modem instead of a 2-hop path over gigabit fiber, because it only counts hops, not speed! **OSPF** fixed this by measuring **bandwidth** (cost), so it prefers faster links even if they have more hops. **EIGRP** goes further, blending bandwidth and delay. So the metric a protocol uses tells you how cleverly it judges "best" — and it's a big reason OSPF and EIGRP are preferred over old RIP.

14.6 The Router's Boot Process & Interfaces

Routers boot like this: **POST** (self-test) → load **IOS** → load **config**. Router interfaces are **shut down by default** — you must turn them on with `no shutdown` and give them an IP:

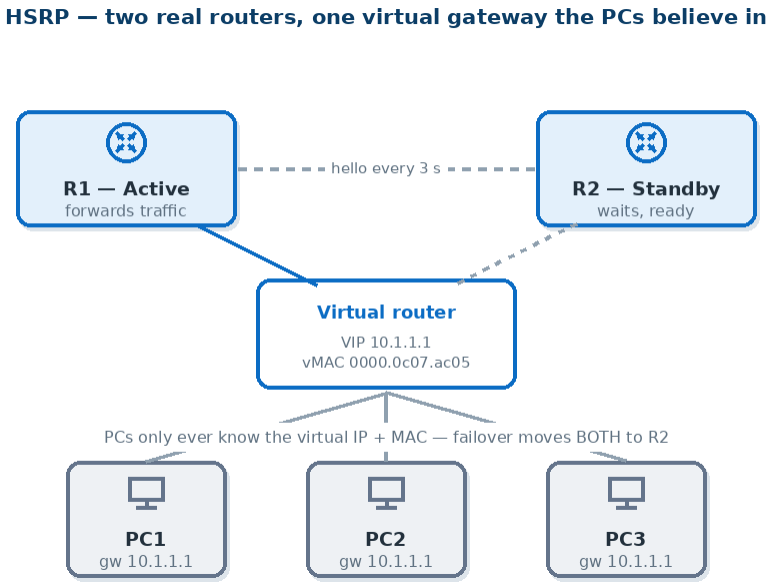
```
R1(config)# interface gi0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# description Link to LAN
R1(config-if)# no shutdown              ! turn the interface ON
```

Check interfaces quickly:

```
R1# show ip interface brief
Interface   IP-Address    OK? Method Status  Protocol
Gi0/0       192.168.1.1   YES manual up      up
Gi0/1       10.0.0.1      YES manual up      up
```

"up / up" means the cable is good (Layer 1) **and** the protocol is working (Layer 2). "administratively down" means someone forgot `no shutdown`.

14.7 First Hop Redundancy (FHRP & HSRP) — A Backup for the Default Gateway



Back in section 11.6 we learned that every device needs a **default gateway** — the router it hands packets to when the destination isn't local. Now let's ask an uncomfortable question about that arrangement.

What happens when the default gateway dies?

Picture PC1, PC2 and PC3 on a switch, all configured with the same default gateway: **R1 at 192.168.1.1**, which reaches the internet.

Now R1 dies. Every PC keeps sending to `192.168.1.1` — and nothing answers. The entire subnet is offline.

Here's the painful part: **you can add a second router, and it doesn't help.** Suppose you install R2 with address `192.168.1.2` as a backup. R2 is powered on, healthy, and perfectly capable of forwarding traffic — but every PC on the subnet was configured (by DHCP or by hand) to use `192.168.1.1`. They will keep sending frames to a router that no longer exists. The backup sits there unused while everyone is offline.

Why don't the PCs just switch to the other router? Because a PC's default gateway is a **single, static setting**. A PC has no protocol for discovering "my gateway died, let me find

another one" — that's simply not part of how hosts work. You could reconfigure every PC by hand, or wait for DHCP leases to renew with a new gateway, but both take far too long during an outage.

So the problem is really this: the hosts can't change, so the routers must pretend to be one router.

The Solution: A Virtual Router

FHRP (First Hop Redundancy Protocol) is the family of protocols that solves this. Two or more real routers share a **virtual IP address** and a **virtual MAC address**. The PCs use that virtual IP as their default gateway and never know — or care — which physical router is actually doing the work.

Two real routers share one **virtual identity**:

Device	Address	Role
The virtual router	IP `192.168.1.1`, MAC `0000.0c07.acXX`	What every PC is configured to use
R1	192.168.1.2	**Active** — actually forwarding
R2	192.168.1.3	**Standby** — watching, ready

When R1 fails, **R2 takes over that same virtual IP** and **MAC**. The PCs keep sending to `192.168.1.1` and notice nothing at all.

Why does the virtual MAC address matter as much as the virtual IP? This is the detail that makes the whole thing work invisibly, and it's a favorite exam point. Remember how a PC actually sends a packet off-subnet: it ARPs for the gateway's IP, caches the resulting **MAC address**, and addresses its frames to that MAC. If failover only moved the IP, every PC would still be sending frames to R1's old MAC — stuck in their ARP caches for minutes — and traffic would keep flowing to a dead router.

By moving the **virtual MAC** to the standby router as well, the frames PCs are already sending arrive at the new active router **unchanged**. No ARP cache needs to expire, no host needs to relearn anything. The hosts genuinely cannot tell that anything happened — which is the entire goal.

The Three FHRPs

Protocol	Origin	Roles	Notes
HSRP	Cisco proprietary	Active / Standby	The Cisco classic — know this one best
VRRP	Open standard (RFC)	Master / Backup	Same idea, vendor-neutral
GLBP	Cisco proprietary	AVG / AVF	Also **load balances** across routers

Why does GLBP exist if HSRP already works? Because HSRP and VRRP are **active/standby** — the backup router forwards nothing while the primary is healthy. You bought two routers and use one. GLBP shares the load across several routers at once by handing out **different virtual**

MACs to different PCs when they ARP, so traffic naturally splits. You can get similar load sharing from HSRP by running two groups and making each router active for one — a common real-world trick.

How HSRP Elects Its Active Router

Routers in the same **HSRP group** exchange hello messages (every 3 seconds by default) and elect an active router:

1. **Highest priority wins** (default 100, range 0–255).
2. **Tie? Highest interface IP address wins.**

If hellos stop arriving for the **hold time** (10 seconds by default), the standby declares the active router dead and takes over.

Configuring HSRP

```
R1(config)# interface gi0/0
R1(config-if)# ip address 192.168.1.2 255.255.255.0
R1(config-if)# standby 1 ip 192.168.1.1        ! the VIRTUAL IP the PCs use
R1(config-if)# standby 1 priority 110          ! higher = preferred active
R1(config-if)# standby 1 preempt               ! take the job back after recovery

R2(config)# interface gi0/0
R2(config-if)# ip address 192.168.1.3 255.255.255.0
R2(config-if)# standby 1 ip 192.168.1.1        ! SAME virtual IP, SAME group
R2(config-if)# standby 1 preempt
```

Why is `preempt` needed — and what breaks without it? Without it, a recovered router **stays standby forever**. Picture it: R1 (priority 110) is active, it reboots, R2 takes over. R1 comes back healthy — and simply stays the backup, because HSRP won't disturb a working active router without being told to. Your carefully designed "the powerful router should be primary" plan is now silently inverted, and it stays that way until someone notices. `preempt` means "when I'm healthy and I have the better priority, take the role back." **Note that the router with the higher priority needs `preempt`** for this to work at all — it's the one doing the taking.

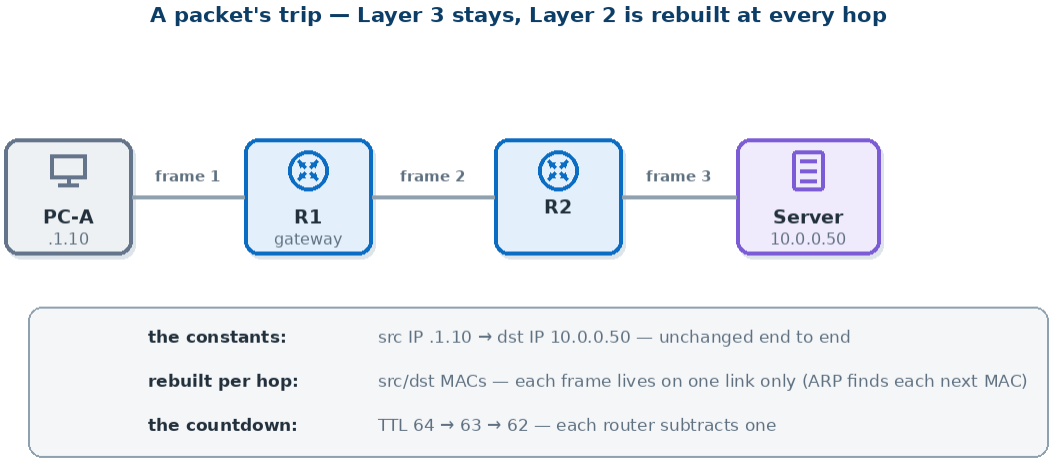
Verifying

```
R1# show standby brief
Interface  Grp  Pri P State    Active          Standby         Virtual IP
Gi0/0      1    110 P Active   local           192.168.1.3     192.168.1.1

R1# show standby              ! full detail: timers, virtual MAC, state changes
```

Read that output as a sentence: group 1, priority 110, P = preempt enabled, this router is Active, the standby is at .3, and together they present 192.168.1.1 to the world.

14.8 A Day in the Life of a Packet — Everything, End to End



> **Depth section — and the most valuable page in this book.** Every chapter so far taught one piece. This traces a single packet through all of them at once. If one idea makes the rest click, it's this one.

PC-A wants to reach a server. They're on different networks, two routers apart.

The path, end to end:

Device	Addresses
PC-A	IP `192.168.1.10`, MAC `aaaa.1111`
R1	Gi0 `192.168.1.1` (MAC `bbbb.2222`) · Gi1 `10.1.1.1`
R2	Gi0 `10.1.1.2` (MAC `cccc.3333`) · Gi1 `10.0.0.1`
SERVER	IP `10.0.0.50`, MAC `dddd.4444`

PC-A connects to a switch, which uplinks to R1; R1 links to R2; R2 reaches the server's network.

Step 1 — "Is this local?" (PC-A does the math)

PC-A ANDs its own address and the destination with its mask (Chapter 12):

Address (AND [redacted])	Result
`192.168.1.10`	`192.168.1.0` — **my** network
`10.0.0.50`	`10.0.0.0` — **different**

Different network, so PC-A sends the frame to its **default gateway**. **This is the decision that makes everything else happen** — and note what PC-A does not do: it never ARPs for the server. Asking "who has 10.0.0.50?" on this segment would be pointless; nobody there can answer.

Step 2 — ARP for the gateway (not the destination)

PC-A needs the **MAC** of 192.168.1.1. It broadcasts an ARP request; R1 replies `bbbb.2222`.

Step 3 — The frame leaves PC-A

Field	Value	Points at
L2 DST MAC	`bbbb.2222`	**R1 (router)**
L2 SRC MAC	`aaaa.1111`	PC-A
L3 SRC IP	`192.168.1.10`	PC-A
L3 DST IP	`10.0.0.50`	**server**
L3 TTL	64	—

Stare at that mismatch — it's the whole idea. The Layer 2 address says "router," the Layer 3 address says "server." Layer 2 answers "who's my next hop on this wire?"; Layer 3 answers "where is this ultimately going?" Confusing these two is the single most common source of confusion in networking.

Step 4 — The switch forwards (and thinks about nothing else)

The switch looks up `bbbb.2222` in its MAC table and forwards out that port (Chapter 6). It never examines the IP header. If it doesn't know the MAC, it floods — and it learns `aaaa.1111` is on the port the frame arrived from.

Step 5 — R1 routes

R1 accepts the frame because the destination MAC is its own. It strips the Ethernet header, reads `10.0.0.50`, and consults its routing table using **longest prefix match** (14.3). Match: send to R2 at `10.1.1.2`. Then R1:

1. **Decrements the TTL** 64 → 63.
2. **Recomputes the IP header checksum** (the TTL changed).
3. **Builds a brand-new Layer 2 header** for the next link.

Field	Was	Now	Changed?
L2 DST MAC	`bbbb.2222`	`cccc.3333`	**rewritten**
L2 SRC MAC	`aaaa.1111`	`bbbb.2222`	**rewritten**
L3 SRC IP	`192.168.1.10`	same	unchanged
L3 DST IP	`10.0.0.50`	same	unchanged
L3 TTL	64	63	decremented

New MACs every hop; the IP addresses never change.

Why throw away a perfectly good Layer 2 header at every hop? Because it was only ever valid on **one wire**. MAC addresses have no meaning beyond the local segment — `aaaa.1111` is not

reachable, or even known, on the far side of R1. The IP addresses survive end to end because they identify the conversation; the MACs are rewritten because they identify only the current handoff. (Unless NAT is involved, Chapter 17 — that's exactly what makes NAT special: it breaks this rule deliberately.)

And why decrement TTL? It's the loop insurance. If a routing mistake sends this packet in a circle, TTL hits zero, some router discards it and returns an ICMP "time exceeded." Without it, one bad route could circulate a packet forever. That ICMP reply is also precisely how `traceroute` maps a path — send TTL=1, see who complains, then TTL=2, and so on.

Step 6 — R2 delivers

R2 repeats the process, but this time `10.0.0.0/24` is **directly connected**. So R2 ARPs for `10.0.0.50` itself, gets `dddd.4444`, and builds the final frame with TTL 62.

Step 7 — The return trip is a whole new decision

The server replies — and **none of the outbound decisions are reused.** It runs its own local-or-remote check, consults its own gateway, and each router independently routes the reply. This is why "I can reach it but it can't reach me" is possible: routing is per-direction, and asymmetric or missing return routes are a classic fault.

Where Each Chapter Lives in This Story

Step	Chapter
Local-or-remote decision, masks	11, 12
ARP for the gateway	5
Switch MAC learning & flooding	6
VLAN tagging if the path crosses a trunk	7, 8
Loop-free path selection	9
Routing table lookup, longest prefix match	14
How R1 learned about 10.0.0.0/24	15, 16
Gateway redundancy if R1 dies	14.7
Address rewriting for the internet	17
Permit/deny along the way	19

When something is broken, walk these steps in order. Nearly every fault is one of them failing: wrong mask (step 1), no ARP reply (step 2), wrong VLAN (step 4), missing route (step 5), or no return route (step 7).

Chapter 15: Static Routing

15.1 What Is a Static Route?

A **static route** is a path you **type in by hand**. You're telling the router: "To reach network X, send packets to next-hop Y." Simple and predictable, but you must update it yourself if the network changes.

Story Time: A static route is like writing directions on a sticky note: "To get to Grandma's, turn left at the big oak tree." It works great — until they cut down the tree and nobody updates your note. That's the downside: static routes don't adjust automatically.

15.2 When to Use Static Routes

- **Small networks** with few paths.
- **Stub networks** (only one way in/out).
- **Default routes** to the internet.
- When you want **total control**.

Downsides: lots of manual work and no automatic healing if a link fails.

15.3 Configuring a Static Route

The command format:

```
ip route <destination-network> <subnet-mask> <next-hop-IP>
```

Example: Router R1 wants to reach `10.0.0.0/24`, which is behind R2 at `192.168.1.2`:

```
R1(config)# ip route 10.0.0.0 255.255.255.0 192.168.1.2
```

You can also point out an **exit interface** instead of a next hop:

```
R1(config)# ip route 10.0.0.0 255.255.255.0 gi0/1
```

15.4 The Default Route (The "Everywhere Else" Route)

A **default route** matches **any** destination the router doesn't otherwise know. It's how routers say "if you don't know where it goes, send it this way (usually toward the internet)." Written as `0.0.0.0 0.0.0.0` (means "all networks"):

```
R1(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.1
```

Story Time: A default route is like a mail clerk's rule: "Any letter I don't recognize? Just send it to the main post office downtown; they'll figure it out." That "main post office" is your ISP.

Why does `0.0.0.0 0.0.0.0` mean "everything"? And why is it the LAST resort? The magic is in the mask: a mask of `0.0.0.0` means "**match zero bits** of the address" — in other words, don't require any part of the destination to match at all. So every possible address matches it. That makes it the **shortest prefix possible (/0)** — the least specific route there is. Remember longest-prefix-match from Chapter 14: the router always prefers a more specific route. So the default route naturally sits at the very bottom of the priority list and only gets used when **nothing more specific matches**. That's exactly the behavior you want: "use real knowledge when I have it; otherwise, shove it toward the internet and let a bigger router deal with it." Your home router does this constantly — it knows your local network specifically, and has a default route pointing everything else (all of the internet) at your ISP. It doesn't need to know where google.com is; it just needs to know "not local → send to ISP."

15.5 Floating Static Route (Backup Path)

You can make a **backup** static route that only activates if the main path dies, by giving it a **higher administrative distance**:

```
R1(config)# ip route 10.0.0.0 255.255.255.0 192.168.1.2       ! main (AD 1)
R1(config)# ip route 10.0.0.0 255.255.255.0 192.168.9.2 5     ! backup (AD 5)
```

The backup "floats" unused until the main route disappears — then it drops in to save the day.

Why does raising the AD make it a "backup" — how does that create standby behavior? Recall AD is the trust score, and **lower wins**. Both routes describe the same destination (10.0.0.0/24), so the router must pick one. The main route has AD 1; the backup we deliberately gave AD 5. Since 1 < 5, the router installs **only the main route** and keeps the AD-5 one sitting on the bench, unused. Here's the clever part: a static route only stays valid while its path is usable. If the main link goes down, that AD-1 route **disappears** from consideration — and now the AD-5 backup is the best remaining option, so the router instantly promotes it into the routing table. When the main link recovers, the AD-1 route returns and wins again, benching the backup once more. So "floating" is just AD doing its job: we manufactured a deliberate trust difference so one route automatically yields to the other. It's an elegant failover with no fancy protocol — just clever use of the trust score you already understand. (You could even point the backup out a slower/pricier link, so it's only used in an emergency — exactly why you'd want it dormant normally.)

15.6 IPv6 Static Routes

Same idea, IPv6 style:

```
R1(config)# ipv6 unicast-routing
R1(config)# ipv6 route 2001:db8:0:2::/64 2001:db8:0:1::2
R1(config)# ipv6 route ::/0 2001:db8:0:1::2        ! default route
```

Chapter 16: Dynamic Routing & OSPF

16.1 What Is Dynamic Routing?

Dynamic routing means routers **talk to each other** and **learn paths automatically**. If a link breaks, they discover a new path on their own. No sticky notes to update!

```
  Static:  You update every route by hand.
  Dynamic: Routers share maps and adapt automatically.
```

16.2 Two Families of Routing Protocols

- **Distance Vector** (e.g., **RIP**): "I heard network X is 3 hops that way." Simple, shares its whole table with neighbors. Can be slow and short-sighted.
- **Link State** (e.g., **OSPF**): Every router builds a **full map** of the network, then calculates the best path itself. Smarter and faster.

Why is "link state" smarter than "distance vector"? What's the real difference? It comes down to how much each router actually knows.

- A **distance-vector** router is like asking for directions by only trusting **rumors from your immediate neighbors**: "My neighbor says network X is 3 hops that way." You don't see the map yourself — you just trust what the next router tells you. This has two problems: it's **slow to react** (bad news has to pass router-to-router-to-router, like a game of telephone), and it can believe **wrong information** during changes (leading to loops where routers point at each other).
- A **link-state** router is like being handed a **complete map of the whole city**. Every router shares facts about its own links, and each router assembles all those facts into an identical full map (the LSDB). Then each router **calculates the best path itself** using that map, rather than trusting neighbor rumors.

Why does having the full map matter so much? Because when you can see the whole network, you can (a) instantly compute the truly shortest path, and (b) react fast and correctly when something breaks — you just update your map and recalculate, no waiting for rumors to propagate hop by hop. That's why OSPF converges (settles after a change) faster and avoids the loop problems that plague RIP. **More knowledge = better, faster, safer decisions.**

16.3 Meet OSPF (Open Shortest Path First)

OSPF is the star of the CCNA. It's a **link-state** protocol that:

- Builds a complete map of the network.
- Uses **cost** (based on bandwidth) to pick the best path.
- Reacts quickly to changes.
- Is an **open standard** (works with any vendor).

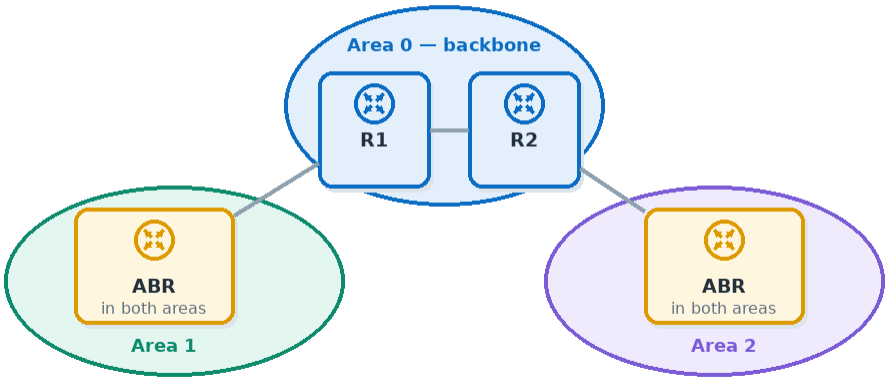
16.4 How OSPF Works (The Friendship Steps)

Story Time: OSPF routers are like new neighbors who introduce themselves, become friends, then share maps of the whole neighborhood.

1. **Hello:** Routers send **Hello** packets to find neighbors.
2. **Neighbor / Adjacency:** They become **neighbors**, then form an **adjacency** (a trusted friendship).
3. **Exchange LSAs:** They swap **Link-State Advertisements** — little facts about their links.
4. **Build the LSDB:** Each router assembles all the LSAs into a full **Link-State Database** (the map).
5. **Run SPF:** Each router runs the **Dijkstra (SPF) algorithm** to compute the shortest path to everywhere.
6. **Install routes:** The best paths go into the routing table.

16.5 OSPF Areas

OSPF areas — every area touches the backbone



ABRs summarize between areas — small LSDB per area, SPF stays cheap

Big OSPF networks are split into **areas** to keep maps small and updates local. The center is always **Area 0** (the backbone). All other areas connect to Area 0.

Area 0 is the backbone. Every other area — Area 1, Area 2 and so on — connects **to Area 0**, never directly to each other. Traffic between two non-backbone areas always passes through the backbone.

Why split OSPF into areas at all — isn't one big map simpler? For a small network, yes. But the "full map" that makes link-state smart becomes a **burden** as the network grows. Two things blow up:

1. **Map size & CPU.** Every router must store the entire network map (LSDB) and run the SPF calculation over all of it. In a network with 1,000 routers, that's a huge map and a heavy calculation for every router.
2. **Constant recalculation.** Here's the killer: in one giant area, **any** link flapping anywhere forces **every** router in the whole network to re-run SPF. One flaky cable in a far corner makes

1,000 routers stop and recompute. Wasteful and destabilizing.

Areas fix both by **containing the detail**. Inside an area, routers know the full local map. But between areas, they only share **summaries** ("Area 1 can reach these networks") — not every internal link. So a flapping link in Area 1 only forces Area 1's routers to recalculate; Area 2 doesn't even hear about it. This is the same hierarchical-addressing idea from IP: keep detail local, share summaries globally. **Why is Area 0 mandatory as the center?** Because you need a single, consistent backbone for all areas to exchange their summaries through — if areas connected in random chains, you could get inconsistent or looping inter-area information. Forcing everything through Area 0 keeps the hierarchy clean: local traffic stays local, and cross-area traffic has one well-defined path through the backbone.

16.6 OSPF Cost — How It Picks Paths

OSPF cost formula (default): **Cost = Reference Bandwidth ÷ Link Bandwidth**. Default reference = 100 Mbps.

Link	Cost
100 Mbps	1
10 Mbps	10
1 Gbps	1 (needs tuning — see note)

> Modern links are all faster than 100 Mbps, so admins raise the reference bandwidth so fast links get different costs: `auto-cost reference-bandwidth 10000`.

Why is cost based on bandwidth (and why does dividing work)? Because OSPF's whole goal is to prefer **faster** paths, and dividing by bandwidth naturally makes faster links cheaper. A bigger-bandwidth link puts a bigger number on the bottom of the fraction, giving a smaller cost — and OSPF prefers the lowest total cost. So a gigabit link ends up "cheaper" than a 10 Mbps link automatically. **Why does the reference bandwidth need tuning today?** Look at the table's problem: with the default reference of 100 Mbps, both a 100 Mbps link and a 1 Gbps link (and a 10 Gbps link!) all round down to cost **1** — OSPF can't tell them apart, so it might pick the slower one. That's an accident of the 1980s default, when 100 Mbps was blazing fast. Raising the reference bandwidth (e.g., to 10,000) restores meaningful differences between modern fast links so OSPF again prefers the genuinely faster path. Understanding the formula is what makes this "gotcha" obvious rather than mysterious.

16.7 OSPF Router ID

Each OSPF router needs a unique **Router ID** (looks like an IP). It's chosen by: 1. A manually set `router-id`, OR 2. The highest **loopback** interface IP, OR 3. The highest active physical interface IP.

Best practice: set it manually for predictability.

Why does OSPF need a Router ID, and why prefer a loopback for it? OSPF needs a **stable, unique name** for each router so it can label who-said-what in the map (every LSA is tagged with

the Router ID of its author). If a router's identity kept changing, the map would get confused about who's who. That's why a **loopback** is preferred as the source: a loopback is a virtual interface that **never goes down** (it has no cable to unplug, no hardware to fail). If OSPF instead used a physical interface's IP as its ID, and that interface went down, the router's very identity could change — forcing disruptive recalculation. A loopback gives OSPF a rock-solid, permanent name. And setting the `router-id` manually is best of all: then you know each router's ID (great for troubleshooting) instead of leaving it to whichever interface happens to have the highest IP.

16.8 Configuring OSPF (Try It!)

```
R1(config)# router ospf 1                  ! start OSPF, process ID 1 (local)
R1(config-router)# router-id 1.1.1.1       ! set a clear router ID
R1(config-router)# network 192.168.1.0 0.0.0.255 area 0   ! advertise this network
R1(config-router)# network 10.0.0.0 0.0.0.3 area 0
```

Wait — what's `0.0.0.255`? That's a **wildcard mask** — the opposite of a subnet mask. Where a subnet mask has 1s, the wildcard has 0s.

Mask type	Value
Subnet mask	`255.255.255.0`
Wildcard mask	`0.0.0.255` — flip every bit

A **0** in the wildcard means "must match exactly"; a **255** means "anything goes."

Check OSPF:

```
R1# show ip ospf neighbor      ! did we make friends?
R1# show ip route ospf         ! what did we learn? (O routes)
R1# show ip protocols          ! routing settings summary
```

16.9 OSPF Neighbor Requirements (Why Friendships Fail)

For two routers to become OSPF neighbors, these must match:

- Same **area**.
- Same **subnet** (on the connecting link).
- Same **Hello** and **Dead** timers.
- Matching **authentication** (if used).
- Compatible **MTU**.

If any of these differ, they won't become neighbors — a very common exam troubleshooting scenario!

Why must all these match — why is OSPF so picky? Because two routers can only build the **same shared map** if they agree on the ground rules. Each requirement guards against a specific way the map could get corrupted or the friendship could silently fail:

- **Same area:** Areas define which map you're part of. Two routers in different areas are working on different maps, so they have no shared map to sync — becoming full neighbors

would be meaningless.

- **Same subnet on the link:** OSPF assumes neighbors share a direct wire. If their IPs are on different subnets, they aren't really "next to each other" in IP terms, and their Hello packets look invalid to each other.
- **Same Hello/Dead timers:** These are the heartbeat. Hello says "I'm alive" every few seconds; Dead is "if I don't hear a Hello for this long, assume you died." If the timers disagree, one router might declare the other dead while the other thinks everything's fine — chaos. They must agree on the heartbeat rhythm.
- **Matching authentication:** If OSPF security is on, a router without the right password is deliberately rejected — that's the whole point of authentication (keeping rogue routers out of your map).
- **Compatible MTU:** MTU is the biggest frame size a link allows. If they disagree, one router might send a map-update packet too big for the other to receive, so the sync **stalls forever** at the exchange stage — a nasty, subtle failure.

So OSPF isn't being difficult for no reason — each match requirement protects the integrity of the shared map or the reliability of the neighbor relationship. This is exactly why "OSPF neighbors won't form" is such a common troubleshooting question: you just walk down this list checking which agreement broke.

16.10 OSPF Network Types & the DR/BDR Election

Section 16.9 listed what must match for OSPF neighbors to form. One item on that list — **network type** — deserves a proper explanation, because it changes how OSPF behaves on a link and it brings in two roles the exam loves: **DR** and **BDR**.

The Problem: Too Many Friendships

Picture five OSPF routers all connected to the **same switch** (one shared Ethernet segment). OSPF's instinct is for every router to become neighbors with every other router. How many adjacencies is that?

If all five routers on a segment became neighbors with each other, you'd get a full mesh:

Routers	Adjacencies — n(n−1)/2
5	5 × 4 ÷ 2 = **10**
10	10 × 9 ÷ 2 = **45**
20	20 × 19 ÷ 2 = **190**

The count grows with the **square** of the number of routers — all flooding the same information over one shared segment.

Every one of those adjacencies means hellos, database exchanges, and updates. Add a router and the count grows roughly with the **square** of the number of routers. On a segment with 20 routers that's 190 adjacencies flooding the same information over and over — enormous waste, since they're all on **one shared segment where everyone hears everything anyway**.

The Solution: Elect a Spokesperson

On broadcast segments, OSPF elects a **Designated Router (DR)** and a **Backup Designated Router (BDR)**. Every other router (called **DROTHER**) forms a full adjacency **only with the DR and BDR** — not with each other.

Instead, the segment elects a spokesperson:

Router	Role	Who it's fully adjacent to
R1	**DR**	Everyone
R2	**BDR**	Everyone — listening, ready to take over
R3, R4, R5	**DROTHER**	Only the DR and BDR

Adjacencies drop from 10 to 7 — and the saving grows quickly as routers are added.

Routers send updates to the DR at multicast address **224.0.0.6**; the DR redistributes to everyone at **224.0.0.5**. The DR is a librarian: instead of every router telling every other router the news, everyone tells the librarian and the librarian announces it once.

Why is there a BDR — why not just re-elect a new DR if the DR fails? Because a fresh election plus rebuilding every adjacency takes time, and OSPF would be blind during it. The BDR has been **listening to everything all along**, so its database is already current — it can step up almost instantly. The backup is pre-warmed, not recruited after the fire starts.

How the DR Is Elected

1. **Highest OSPF priority wins** (default 1; range 0–255).
2. **Tie? Highest Router ID wins** (the same RID from section 16.7).
3. **Priority 0 means "never eligible"** — a useful way to keep a weak router out of the job.

```
R1(config-if)# ip ospf priority 100     ! make this router the DR
R1(config-if)# ip ospf priority 0       ! this router must never be DR/BDR
```

The trap that catches people: the DR election is **not preemptive**. Bring a brand-new router with priority 255 onto a segment that already has a DR, and it will not take over — it becomes a DROTHER and waits. The existing DR keeps the job until it fails or OSPF is restarted (`clear ip ospf process`). This is the opposite of HSRP with `preempt` — a classic exam comparison, so keep them apart: HSRP can be told to take its role back; an OSPF DR never takes the role back on its own.

The Two Network Types You Must Know

Network type	Where it's used	DR/BDR?	Hello / Dead timers
Broadcast	Ethernet (LANs, switches)	**Yes**	10 s / 40 s
Point-to-point	Serial links, router-to-router	**No**	10 s / 40 s

Why does point-to-point skip the election entirely? Because the whole DR concept solves a problem that doesn't exist there. With exactly **two** routers on a link, there's only one possible adjacency — n(n-1)/2 = 1. Electing a spokesperson to reduce one adjacency to one adjacency would just add delay for nothing. No crowd, no need for a spokesperson.

This is also why a common optimization exists: a link between two routers on Ethernet is technically a broadcast segment, so OSPF holds a pointless election and waits through it. Telling OSPF the truth about the link skips that:

```
R1(config-if)# ip ospf network point-to-point   ! only 2 routers here — skip DR/BDR
```

Remember from 16.9: hello and dead timers **must match** between neighbors, and network type must be compatible — mismatches are a top cause of adjacencies that never form.

Verifying

```
R1# show ip ospf neighbor
Neighbor ID   Pri  State           Dead Time  Address      Interface
2.2.2.2       1    FULL/DR         00:00:35   10.1.1.2     GigabitEthernet0/0
3.3.3.3       1    FULL/BDR        00:00:33   10.1.1.3     GigabitEthernet0/0
4.4.4.4       1    2WAY/DROTHER    00:00:31   10.1.1.4     GigabitEthernet0/0
```

Read that third line carefully — `2WAY/DROTHER` is not a failure. Two DROTHERs on a segment deliberately stop at **2WAY** and never reach FULL with each other, because they only need full adjacency with the DR and BDR. Seeing 2WAY between DROTHERs is **normal and correct**; panicking about it is a classic beginner mistake. FULL with the DR and BDR, 2WAY with everyone else — that's a healthy broadcast segment.

> **One item on that list needs its own section.** "Network type" decides whether OSPF holds an election on the link — which brings in the **DR** and **BDR** roles. That's section **16.10**, next.

16.11 OSPF Internals — Adjacency States, LSAs and SPF

> **Depth section.** CCNA asks you to configure single-area OSPF and read `show ip ospf neighbor`. CCNP assumes you know what's underneath. This is that layer.

The Neighbor State Machine

`show ip ospf neighbor` prints a **state**, and each one means a specific step completed. When an adjacency is stuck, the state tells you exactly where:

State	What's happening	Stuck here usually means
Down	No hellos heard	Interface, cabling, or OSPF not enabled

State	What's happening	Stuck here usually means
Init	I heard their hello, but I'm not listed in it	One-way communication — often an ACL or wrong subnet mask
2-Way	We each see ourselves in the other's hello	**Normal** between two DROTHERs (16.10)
ExStart	Electing master/slave to sequence the exchange	**MTU mismatch** — the classic cause
Exchange	Trading database descriptions (DBDs)	MTU or unstable link
Loading	Requesting the LSAs I'm missing	Rare; usually resolves
Full	Databases synchronized	Healthy

Why is there a master/slave election in ExStart, when the routers are peers? Because the database exchange needs a **single sequence-number owner**. If both sides numbered their own DBD packets, neither could tell a retransmission from a new one. Electing one side (**higher Router ID wins**) to drive the sequence makes the conversation unambiguous.

Why does an MTU mismatch strand adjacencies at ExStart/Exchange specifically? Because DBD packets can be large. If one router's MTU is smaller, it silently drops the oversized DBD — hellos (small) still arrive, so the neighbour appears reachable, but the exchange never completes. This is why "stuck in EXSTART" should make you check `show interface | include MTU` before anything else.

LSA Types — the Building Blocks of the Map

OSPF doesn't advertise routes; it advertises **link-state advertisements** that every router assembles into an identical map.

Type	Name	Describes	Scope
1	Router LSA	A router's own links	Within its area
2	Network LSA	A broadcast segment, generated by the **DR**	Within its area
3	Summary LSA	A network in another area, from the **ABR**	Between areas
4	ASBR Summary	How to reach an ASBR	Between areas
5	External LSA	A route redistributed from outside OSPF, from the **ASBR**	Whole domain
7	NSSA External	An external route inside a not-so-stubby area	That area only

Types **1 and 2** are the ones a CCNA-level single-area network uses. The rest appear the moment you add areas or redistribution — which is precisely where CCNP begins.

Why does the DR generate a separate Type 2 instead of every router describing the segment? Because a shared segment is one shared object. Ten routers each describing "I connect to this Ethernet" would give ten partial views to reconcile. The DR — already elected as the segment's spokesperson — publishes **one** authoritative description listing everyone attached. One shared fact, one advertiser.

From Database to Routing Table

The pipeline runs in three steps:

1. **LSAs flood** across the area, filling an **LSDB** that is identical on every router.
2. Each router independently runs **Dijkstra's SPF** over that database, building a shortest-path tree with **itself** at the root.
3. The best path to each destination drops into that router's **routing table**.

Why must every router in an area hold an identical database? Because each one computes its own tree from it. If two routers disagreed about the map, they'd compute incompatible paths and could forward packets to each other in a loop. **Identical input + deterministic algorithm = consistent forwarding**, with no router needing to trust another's conclusion. That's the fundamental difference from distance-vector protocols like RIP, which believe what neighbours tell them and can loop when neighbours are wrong.

Cost is `reference bandwidth ÷ interface bandwidth`, with a default reference of **100 Mbps**. That default is a fossil: it makes 100 Mbps, 1 Gbps and 10 Gbps all compute to cost 1, because the answer floors at 1. On any modern network you raise it:

```
R1(config-router)# auto-cost reference-bandwidth 100000   ! in Mbps = 100 Gbps
```

Set it **identically on every router** — mismatched reference bandwidths mean routers disagree about which path is cheapest.

Area Types (Your CCNP Runway)

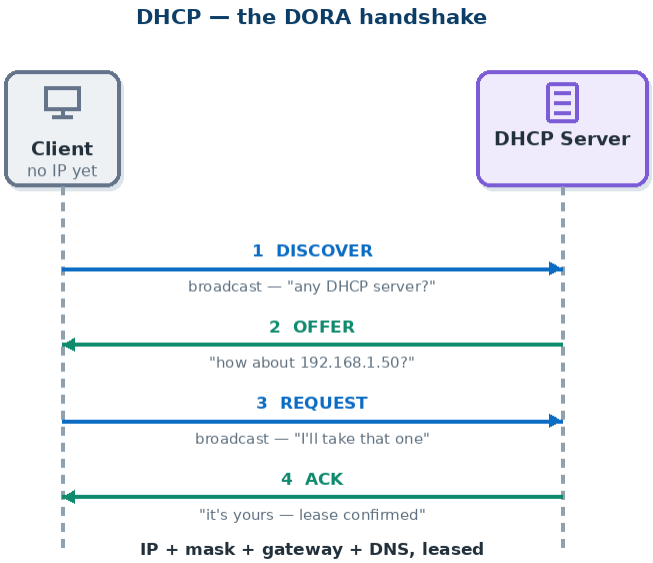
Areas exist to bound flooding and SPF work. **Special area types go further, trading detail for simplicity:**

Area type	Blocks	Routers get instead
Standard	nothing	full detail
Stub	Type 5 externals	a default route
Totally stubby (Cisco)	Types 3, 4, 5	a default route
NSSA	Type 5, but allows Type 7	local externals + default

Why would you deliberately deny a router information? Because detail costs memory and CPU on every SPF run, and a branch office with one exit link cannot use the detail. If every route leaves through the same door, knowing 10,000 destinations beyond it changes nothing — "everything else goes that way" is equally correct and vastly cheaper. **NSSA** exists for the awkward middle case: a stub area that nonetheless has its own external route to inject, which plain stub rules would forbid.

Chapter 17: DHCP, DNS, NAT & Other Helpers

17.1 DHCP — Automatic IP Addresses



DHCP (Dynamic Host Configuration Protocol) hands out IP addresses **automatically**. Without it, you'd type an IP into every phone and laptop by hand. Yuck!

The DHCP Handshake: D-O-R-A

Story Time: When your laptop joins Wi-Fi, it does a little four-step dance to get an address:

Step	Direction	The message
1. **DISCOVER**	Client → (broadcast)	"Any DHCP servers out there?"
2. **OFFER**	Server → Client	"Yes — here's an address you can use."
3. **REQUEST**	Client → Server	"Great, I'd like that one please."
4. **ACK**	Server → Client	"It's yours. Here are the details."

Remember **DORA**: **D**iscover, **O**ffer, **R**equest, **A**ck.

DHCP gives you: an **IP address**, **subnet mask**, **default gateway**, and **DNS server** — everything needed to get online.

Why does the very first step (Discover) have to be a BROADCAST? Because of a chicken-and-egg problem: the laptop needs an IP address to communicate normally... but it doesn't have one yet, and it doesn't know the DHCP server's address either! It can't send a normal unicast message ("Hey server 10.0.0.5, give me an IP") because it knows neither its own address nor the server's. So it does the only thing possible: it **shouts to everyone** (broadcast

`FF:FF:FF:FF:FF:FF`) — "Is anybody here a DHCP server?" A broadcast is the one kind of message you can send when you know nothing about the network yet. That's why DHCP must start with a broadcast — it's the only way to talk before you have an identity.

Why does the client "REQUEST" an address it was just OFFERED — didn't the server already give it one? Two reasons. First, there might be **multiple DHCP servers**, each sending an Offer. The client picks one and the Request announces "I'm taking this one" so the others know to release their offers. Second, the Offer is only a tentative reservation; the Request→Ack step is the formal "yes I want it / it's confirmed yours" handshake, so both sides agree on the final lease. It's like being offered a hotel room — you still have to say "yes, I'll take it" before it's truly booked.

Configuring a DHCP Server on a Router

```
R1(config)# ip dhcp excluded-address 192.168.1.1 192.168.1.10  ! don't hand these out
R1(config)# ip dhcp pool LAN_POOL
R1(dhcp-config)# network 192.168.1.0 255.255.255.0   ! range to give out
R1(dhcp-config)# default-router 192.168.1.1          ! the gateway
R1(dhcp-config)# dns-server 8.8.8.8                  ! DNS to use
R1(dhcp-config)# lease 7                             ! address lasts 7 days
```

Why "exclude" some addresses, and why do leases expire? You exclude the low addresses (like .1–.10) because those are usually your **fixed infrastructure** — the router/gateway, switches, servers — which need permanent, predictable addresses. If DHCP handed .1 to a random laptop, it might collide with your router! So you carve those out. And **leases expire** because addresses are a limited, reusable resource: when a guest's laptop leaves the coffee shop, you want its address back in the pool for the next customer. A lease is a "rental with a return date" — if the device is still around, it renews; if it vanished, the address is reclaimed. Without expiring leases, a busy network would slowly run out of addresses given to devices that left long ago.

DHCP Relay (Helper Address)

DHCP uses broadcasts, which routers don't forward. If your DHCP server is on **another** network, configure a **helper address** so requests get relayed:

```
R1(config-if)# ip helper-address 10.0.0.5   ! forward DHCP to this server
```

Why is a helper address even necessary? Here's the conflict: DHCP Discover is a **broadcast**, but (from Chapter 6/11) **routers deliberately do NOT forward broadcasts** — that's their job, to contain broadcast domains! So a laptop's "any DHCP servers?" shout dies at the router and never reaches a server sitting on a different network. In the real world, companies don't put a DHCP server on every single subnet — they run **one central server** for everyone. The `ip helper-address` command is the fix: it tells the router "when you hear a DHCP broadcast, make an exception — convert it to a unicast and forward it to the real server at 10.0.0.5." So the helper address is the deliberate bridge across the very broadcast boundary the router normally enforces — letting one central DHCP server serve many subnets.

> **When DHCP fails, the client tells you.** A PC that gets no answer assigns itself a **169.254.x.x** APIPA address — the single fastest clue in networking that DHCP is unreachable. Section **23.6**

> covers reading that (and the other three numbers) on Windows, macOS and Linux.

17.2 DNS — The Internet's Phone Book

DNS (Domain Name System) turns **names** (like `www.google.com`) into **IP addresses** (like `142.250.72.4`). Humans remember names; computers need numbers. DNS is the translator.

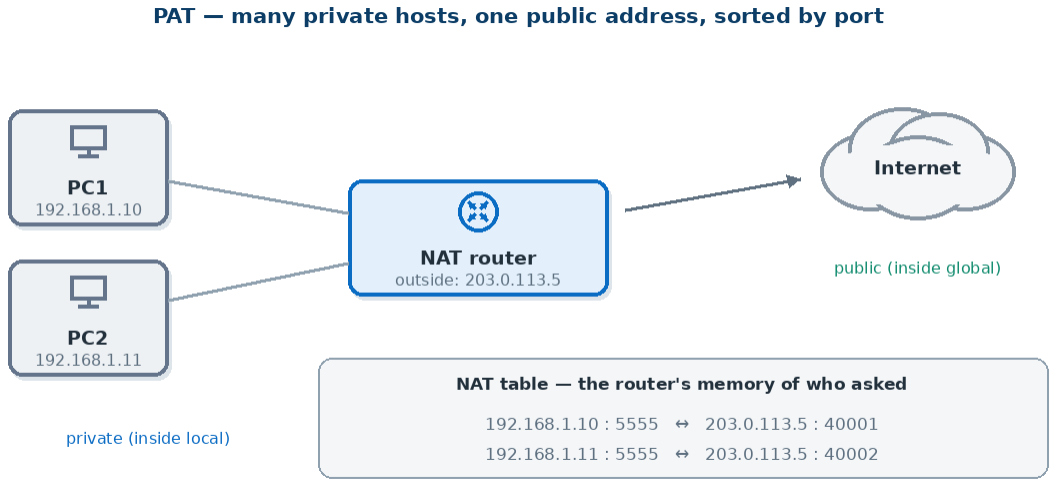
Story Time: DNS is like the contacts app on your phone. You tap "Mom," and the phone dials her actual number. You don't memorize the number — DNS remembers it for you.

1. You type **`www.google.com`**.
2. A **DNS server** answers: "that's `142.250.72.4`."
3. Your browser connects to **`142.250.72.4`** — the name was only ever a lookup key.

Common DNS record types:

- **A** = name → IPv4 address.
- **AAAA** = name → IPv6 address.
- **CNAME** = an alias (nickname) for another name.
- **MX** = mail server for a domain.
- **PTR** = reverse (IP → name).

17.3 NAT — Sharing One Public Address



NAT (Network Address Translation) lets **many private devices** share **one public IP** to reach the internet. This is why your whole house full of gadgets can use the internet with just one address from your provider.

Several private addresses on the inside — `192.168.1.10`, `.11`, `.12` — all reach the internet through **one router running NAT**, which translates every one of them to the single public address **`203.0.113.5`**.

Types of NAT

- **Static NAT:** One private IP ↔ one public IP (permanent). Used for servers you want reachable.
- **Dynamic NAT:** A pool of public IPs shared as needed.
- **PAT (Port Address Translation) / NAT Overload: Many** private IPs share **ONE** public IP, using **port numbers** to tell conversations apart. This is what home routers do!

Story Time: PAT is like an apartment building with one street address. Mail for everyone uses the same address, but the **apartment number** (port) makes sure each letter reaches the right person.

How does PAT actually keep everyone's traffic straight with only ONE public IP? Why do ports make this work? This is the clever core of how your home internet works, so let's trace it. When three PCs all browse the web through one public IP, the router needs a way to remember "which reply belongs to which PC." It uses a **translation table** keyed by **port numbers**:

PAT translation table (simplified):

Private (inside)	becomes	Public (outside)
`192.168.1.10 : 51000`	→	`203.0.113.5 : 40001`
`192.168.1.11 : 49200`	→	`203.0.113.5 : 40002`
`192.168.1.12 : 51000`	→	`203.0.113.5 : 40003`

When PC .10 sends a request, the router rewrites the source to `203.0.113.5 : 40001` and **notes that in the table**. When the web server replies to `203.0.113.5 : 40001`, the router looks up port 40001, sees "that's really 192.168.1.10," and forwards it back to the right PC. The port number is the claim ticket — it's how the router tells apart hundreds of conversations that all share one public address. Notice even when two PCs happen to use the same private port (51000 above), the router just assigns them **different public ports** (40001 vs 40003), so there's never confusion. **Why does this matter so much?** Because it's the trick that lets a whole house — or a whole company — share a single public IPv4 address, which (along with private addressing) is a huge reason we didn't run out of IPv4 years ago. One public address can support thousands of simultaneous conversations, each tracked by its port.

Configuring PAT (NAT Overload)

```
R1(config)# access-list 1 permit 192.168.1.0 0.0.0.255   ! who gets translated
R1(config)# interface gi0/0
R1(config-if)# ip nat inside              ! the private side
R1(config)# interface gi0/1
R1(config-if)# ip nat outside             ! the internet side
R1(config)# ip nat inside source list 1 interface gi0/1 overload   ! PAT!
```

Configuring Static NAT (One-to-One)

```
R1(config)# ip nat inside source static 192.168.1.50 203.0.113.10
R1(config)# interface gi0/0
R1(config-if)# ip nat inside
R1(config)# interface gi0/1
R1(config-if)# ip nat outside
```

When would you want this instead of PAT? When traffic needs to reach in from the internet — a web or mail server. PAT builds its table only when an **inside** device starts a conversation, so an outsider has no entry to match and can't get in. A **static** mapping is permanent and works in **both** directions, giving that one server a fixed, reachable public address.

Configuring Dynamic NAT with a Pool

```
R1(config)# ip nat pool MYPOOL 203.0.113.10 203.0.113.20 netmask 255.255.255.0
R1(config)# access-list 1 permit 192.168.1.0 0.0.0.255
R1(config)# ip nat inside source list 1 pool MYPOOL      ! add 'overload' for PAT on the pool
```

Here devices are handed a public address from the pool **as needed**. The catch: when the pool runs out, further devices simply **fail** to get translated — which is exactly why PAT (with `overload`) is far more common. Adding `overload` to the same command lets the whole pool be shared by ports too, combining both ideas.

Verifying Any Flavor of NAT

```
R1# show ip nat translations      ! the live translation table
R1# show ip nat statistics        ! hits, misses, which pool/ACL is in use
R1# clear ip nat translation *    ! wipe dynamic entries (useful when testing)
```

Why must you label interfaces `inside` and `outside`? Because NAT has to know which direction traffic is going to translate it correctly — it rewrites private→public on the way out and public→private on the way back in. Telling the router which side is the private "inside" and which is the internet-facing "outside" is what lets it apply the translation in the right direction. And the keyword `overload` is literally what turns plain NAT into PAT — it means "overload one public address with many conversations by using ports," exactly the port-tracking trick above.

17.4 NTP — Keeping Time in Sync

NTP (Network Time Protocol) keeps all devices' clocks in sync (UDP port **123**). Why care? Three concrete reasons the exam and real life share:

- **Correlating logs.** When something breaks, you line up syslog entries from five devices to reconstruct the story. If R1 thinks it's 09:14 and R2 thinks it's 09:03, the story is unreadable — you literally cannot tell which failure came first, which is the entire question.
- **Certificates and authentication.** Certificates are valid between two timestamps. A device with a wildly wrong clock rejects perfectly good certificates (breaking SSH/HTTPS) — or accepts expired ones.
- **Time-based features.** ACLs, key rotation and scheduled jobs all trust the clock.

How it's organized: stratum. NTP arranges time sources into levels called **stratum** — think "how many steps from the atomic clock." **Stratum 0** is the reference hardware itself (atomic clock, GPS), **stratum 1** is a server directly attached to it, **stratum 2** learns from stratum 1, and so on. **Lower stratum = closer to the truth = more believable.** A device that syncs from a stratum 3 server becomes stratum 4 itself. Why does this matter? Given several candidate servers, NTP prefers the lower stratum — and if a device advertises stratum 15+, it's considered

barely better than a guess.

```
R1(config)# ntp server 129.6.15.28        ! be a CLIENT of this server
R1(config)# ntp master 3                  ! also SERVE time (as stratum 3)
R1# show ntp status                       ! synchronized? to whom? what stratum?
R1# show ntp associations                 ! all servers it's talking to
```

The usual design: one or two routers sync to a public NTP server on the internet, then act as `ntp master` for everything inside. **Why not point all 200 devices at the internet?** The same centralization logic as AAA and DHCP: one controlled place to manage — and if the internet link dies, your devices keep agreeing with each other, which is what log correlation actually needs.

17.5 Syslog — The Diary of Events

Syslog collects **log messages** from devices (errors, warnings, events) in one place. Messages have **severity levels 0–7** (lower = more urgent):

Level	Name	Meaning
0	Emergency	System unusable!
1	Alert	Act now!
2	Critical	Serious problem
3	Error	Something failed
4	Warning	Might be a problem
5	Notice	Normal but notable
6	Informational	Just info
7	Debug	Super detailed

Memory trick: "**E**very **A**wesome **C**isco **E**ngineer **W**ill **N**eed **I**ce cream **D**aily" → Emergency, Alert, Critical, Error, Warning, Notice, Informational, Debug.

17.6 SNMP — Watching Devices

SNMP (Simple Network Management Protocol) lets a management station monitor and manage many devices (CPU, memory, interface stats). The moving parts have names the exam expects:

Piece	What it is
NMS / manager	The monitoring station (the software with the dashboards)
Agent	The small process on each device that answers questions
MIB	The Management Information Base — the tree of everything askable

Piece	What it is
OID	One dotted address into that tree (e.g. `1.3.6.1.…` = "interface 1's byte count")

Two directions of conversation, and the difference matters:

- **Polling (manager asks):** `Get` reads a value, `Set` changes one. The manager sweeps its devices every few minutes — reliable, but everything between polls is invisible.
- **Traps (agent shouts):** the device sends an unsolicited **trap** the moment something happens ("interface down!"). **Why have both?** Because polling alone means a link can flap and recover between two polls and you'd never know; traps alone means a dead device simply goes silent — no agent left to shout. Polling proves devices are alive; traps catch events instantly. Real monitoring uses both.

Versions: **v1** is ancient; **v2c** added better bulk reads but still "secures" everything with a plaintext **community string** — a shared password sent in the clear, usually `public` (read) and `private` (write). **v3** finally adds real **authentication and encryption** — always prefer it, and if you must run v2c, make it read-only with an ACL restricting who may ask.

17.7 QoS — Deciding Who Goes First When the Road Is Full

Every service so far has been about making things work. QoS is about what happens when the network is **busy** and not everything can go at once.

Why is this needed at all — doesn't the network just deliver everything? It does, until a link fills up. When more traffic arrives at an interface than the link can send, the excess waits in a **queue**. If the queue fills, packets are **dropped**. By default that happens **blindly** — first come, first served — and that default is a disaster for certain traffic:

- A **file download** that arrives 200 ms late: nobody notices. TCP resends a lost packet and the file is fine.
- A **voice call** that arrives 200 ms late: the call breaks up. And a resent voice packet is useless — the moment it belonged to has already passed.

That's the core insight: **different traffic has different needs, and the network can't guess them.** A big backup transfer will happily consume every bit of bandwidth available, drowning out the phone call sharing that link — not maliciously, just because nothing told it otherwise. QoS is how you tell the network what matters when there isn't room for everything.

What Voice and Video Actually Need

Term	Means	Voice tolerance
Bandwidth	How much capacity	Small (~80 Kbps per call)
Delay (latency)	How long the trip takes	< 150 ms one way
Jitter	Variation in delay	< 30 ms
Loss	Packets dropped	< 1%

Why does jitter get its own row — isn't it just delay again? No, and it's the one people underestimate. Voice must be played back at a **steady rate**. If packets arrive at 20 ms, then 60 ms, then 15 ms apart, the audio stutters even though the average delay looks acceptable. Consistency matters more than speed here — a steady 100 ms beats an erratic 40 ms.

The QoS Toolkit (Per-Hop Behavior)

QoS is applied **hop by hop** — each device makes its own decision about traffic passing through it. That's the **PHB (Per-Hop Behavior)**.

1. Classification — identify what the traffic is (voice? video? email?), by port number, protocol, or incoming marking.

2. Marking — write that decision **into the packet header** so later devices don't have to redo the work. The Layer 3 marking is **DSCP** (6 bits in the IP header's ToS field); the Layer 2 marking is **CoS** (3 bits in the 802.1Q tag).

Traffic	Common DSCP	Value
Voice	**EF** (Expedited Forwarding)	46
Interactive video	AF41	34
Signaling (call setup)	CS3	24
Best effort (everything else)	BE / Default	0

Why mark once at the edge instead of classifying at every hop? Because deep inspection is expensive, and every router repeating it wastes effort. Classify once where the traffic **enters** the network, stamp the verdict on the packet, and every device afterward just reads the stamp. Do the hard thinking at the door; everyone downstream reads the badge.

This is also why **trust boundaries** matter: a device decides whether to believe incoming markings. You trust the marking from a company IP phone; you do **not** trust it from a random PC, because any user could mark their game traffic EF and promote themselves to the front of every queue.

3. Queuing — when traffic is waiting, decide the order it leaves. **LLQ (Low Latency Queuing)** gives voice a **strict priority queue** that is always served first.

4. Congestion avoidance — rather than letting a queue fill and then dropping everything at once (**tail drop**), **WRED** drops some lower-priority packets early to signal TCP senders to slow down. **Why deliberately drop packets early?** Because tail drop causes **global synchronization**: every TCP sender loses packets simultaneously, all back off together, the link empties, all speed up together, and the link floods again — a sawtooth that wastes capacity. Dropping a few early and selectively keeps the flow smooth.

5. Policing vs. Shaping — both limit traffic to a rate, but they differ in what they do with the excess, and that difference is a favorite exam question:

Tool	Over the limit?	Adds delay?	Feels like
Policing	**Drops** the excess	No	Harsh — traffic is simply chopped off

Tool	Over the limit?	Adds delay?	Feels like
Shaping	**Buffers** it and sends it later	Yes (needs buffers)	Gentle — the bursts are smoothed out

Policing throws away the excess; shaping makes it wait its turn. Policing is typically applied to **incoming** traffic (an ISP enforcing what you paid for), shaping to **outgoing** traffic (smoothing your own traffic so the far end doesn't have to drop it).

Chapter 18: Network Security Basics

18.1 The Three Goals: The CIA Triad

Security has three big goals, remembered as **CIA**:

Goal	Means	In plain terms
Confidentiality	Secret	Only the right people can see it
Integrity	Correct	The data isn't secretly altered
Availability	Working	It's up when you need it

Why THREE goals — why not just "keep the bad guys out"? Because "secure" means different things depending on what you're protecting, and these three cover all the ways security can fail. Miss any one and you're vulnerable, even if the other two are perfect:

- **Confidentiality** protects against the wrong people reading your data (a hacker stealing passwords). But confidentiality alone isn't enough...
- **Integrity** protects against data being secretly changed — imagine a bank transfer where the amount is confidential (nobody can read it) but an attacker still flips $100 to $10,000 in transit. It was secret, but it was tampered with. Integrity guards the correctness of data.
- **Availability** protects against the service being knocked offline — a hacker doesn't need to read or change your data to hurt you; they can just **flood your website until it crashes** (a DoS attack) so real customers can't use it. Perfectly secret, perfectly correct data is useless if nobody can reach it.

So the triad is a checklist that forces you to think about every angle: Can the wrong people read it? Can they change it? Can they take it down? Real security needs all three. This is why the exam frames threats by which part of CIA they attack — it tells you what defense you need.

18.2 Common Threats (The Bad Guys' Tricks)

- **Malware:** Bad software (viruses, worms, ransomware).
- **Phishing:** Fake emails/sites tricking you into giving passwords.
- **Denial of Service (DoS):** Flooding a system so it can't work.

- **Man-in-the-Middle (MITM):** Secretly sitting between two parties to spy or change data.
- **Spoofing:** Pretending to be someone else (fake IP or MAC).
- **Brute Force:** Trying many passwords until one works.

18.3 Layer 2 Attacks & Defenses (CCNA Favorites)

Attack	What It Does	Defense
MAC flooding	Fills the switch's MAC table so it floods everything	**Port security**
VLAN hopping	Sneaks into another VLAN	Disable DTP, change native VLAN
DHCP spoofing	Fake DHCP server hands out bad info	**DHCP snooping**
ARP spoofing	Fakes MAC-to-IP mappings (MITM)	**Dynamic ARP Inspection**
Rogue devices	Unauthorized switch/AP	802.1X, BPDU Guard

Why does MAC flooding work — how does filling a table let an attacker spy? This one is beautifully sneaky, and it makes sense once you connect it to Chapter 6. Remember: a switch only sends a frame to the right port if it knows which port the destination MAC is on. And what does a switch do when it doesn't know? It **floods** the frame out every port. Now here's the attack: the switch's MAC table has **limited memory**. An attacker blasts the switch with thousands of fake source MAC addresses until the table is **completely full**. With no room left to learn real MACs, the switch is forced to **flood almost everything out every port** — meaning the attacker's port now receives copies of traffic meant for other people. They've turned a smart switch back into a dumb hub, and can eavesdrop. The defense (port security) works by limiting how many MACs a port can present — so an attacker can't flood thousands of fakes from one port. Understanding why the attack works is what makes the defense obvious.

Why does DHCP spoofing (a rogue DHCP server) matter so much? Because whoever hands you your network settings controls where your traffic goes! Recall DHCP also gives you your **default gateway** and **DNS server**. A fake DHCP server can tell you "the gateway is my address" — so now all your internet traffic flows through the attacker (a man-in-the-middle), who can spy or tamper. That's why DHCP snooping exists: it only trusts DHCP replies from the ports you designate (toward the real server) and blocks "offers" coming from user ports where a rogue server might hide.

DHCP Snooping (Trust the Right Ports)

Marks which ports are allowed to send DHCP offers (trusted = toward the real server):

```
SW1(config)# ip dhcp snooping
SW1(config)# ip dhcp snooping vlan 10
SW1(config)# interface gi0/1
SW1(config-if)# ip dhcp snooping trust     ! this port leads to the real DHCP server
```

18.4 AAA — Who Are You, and What Can You Do?

AAA stands for:

- **Authentication:** Who are you? (username/password)
- **Authorization:** What are you allowed to do?
- **Accounting:** What did you do? (logging)

Servers like **RADIUS** and **TACACS+** provide AAA centrally instead of a password on every device.

Feature	RADIUS	TACACS+
Made by	Open standard	Cisco
Protocol	UDP	TCP
Encrypts	Only password	Whole packet
Best for	Network access (Wi-Fi)	Device admin control

Why split security into THREE separate A's? Because "letting someone in" and "controlling what they do" and "recording what they did" are genuinely different jobs, and you often want them handled differently:

- **Authentication** (who are you?) just proves identity — like showing ID at a door.
- **Authorization** (what can you do?) is separate because proving who you are doesn't mean you can do everything. A junior tech might log in successfully (authenticated) but only be allowed to view configs, not change them (authorization). Splitting these lets you give different people different powers with the same login system.
- **Accounting** (what did you do?) creates a record — crucial for security investigations ("who changed this at 3am?") and compliance. You want this even for people who were allowed to act.

Why centralize AAA on a server instead of a password on each device? Imagine 500 switches each with their own local password. An employee leaves — now you must change the password on **all 500** by hand (and you'll miss some). With a central AAA server, you disable **one** account and they're locked out everywhere instantly. One place to manage identities = far more secure and less error-prone.

Why choose TACACS+ vs RADIUS? Their design differences map to their best use:

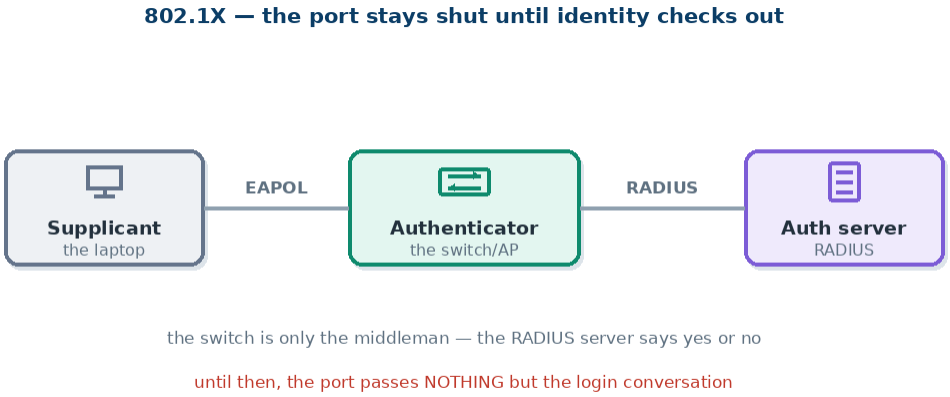
- **TACACS+** encrypts the entire packet and separates the three A's cleanly, and it's great for **controlling device administrators** (exactly which commands each admin can run). Encrypting everything matters when you're protecting powerful admin actions.
- **RADIUS** encrypts only the password and combines authentication+authorization, which is lighter and fine for **network access** (letting users onto Wi-Fi). It's an open standard, so it works across all vendors.

So it's not "which is better" — it's "which fits the job": TACACS+ for admin control, RADIUS for user network access.

18.5 Passwords & Best Practices

- Use **strong, long** passwords.
- Use `enable secret` (encrypted) not `enable password` (weak).
- Use **SSH**, never **Telnet**.
- Turn on `service password-encryption`.
- Shut down **unused ports** and put them in an unused VLAN.
- Use a **login banner** warning.
- Keep IOS **updated**.

18.6 802.1X — The Bouncer at the Port



802.1X makes a device **prove who it is** before the switch port lets it onto the network. Three players:

- **Supplicant:** The device trying to connect.
- **Authenticator:** The switch/AP (the bouncer).
- **Authentication Server:** RADIUS (checks the ID).

The device asks to join; the **switch** passes the credentials to a **RADIUS** server; RADIUS answers **allowed** or **denied**, and the switch opens or keeps the port shut accordingly.

18.7 VPNs — Secret Tunnels

A **VPN (Virtual Private Network)** creates an **encrypted tunnel** across the public internet so data travels safely, as if on a private line.

- **Site-to-Site VPN:** Connects two offices (via **IPsec**).
- **Remote-Access VPN:** Connects a single user from home (e.g., Cisco AnyConnect/SSL).

Office A and Office B are joined by an **encrypted tunnel** across the public internet. Anyone intercepting the traffic in between sees only scrambled gibberish.

18.8 Beyond Passwords — MFA, Certificates & Biometrics

Section 18.5 covered making passwords strong. But the exam also expects you to know **why the industry is moving past passwords entirely**, and what replaces them.

What's actually wrong with passwords? Every weakness comes from the same root: a password is **something you know**, and knowledge can be copied without you noticing. If someone phishes, guesses, or steals your password from a breached website, they now have exactly what you have — and nothing about the login looks unusual. You can't tell "the real user typed it" from "an attacker typed it," because they're the same event.

The Three Authentication Factors

Factor	Means	Examples
Something you know	Knowledge	Password, PIN
Something you have	Possession	Phone app, token, smart card
Something you are	Inherence	Fingerprint, face, retina

MFA (Multi-Factor Authentication) requires **two or more different factors**. The emphasis on different is the whole point: a password plus a security question is **not** MFA, because both are things you know and a single phishing page harvests both. A password plus a code from your phone is MFA — the attacker now needs to steal a **physical object** too, which is dramatically harder to do remotely and at scale.

Certificates replace the shared secret with a **key pair**. Your device proves its identity by signing something with a private key that **never leaves the device** — so there's no secret in transit to intercept and nothing on the server worth stealing. This is what 802.1X (section 18.6) uses in its strongest form, and why certificate-based Wi-Fi authentication beats a shared password.

Biometrics are convenient but come with a permanent catch worth stating plainly: **you can't change your fingerprint.** A leaked password is replaced in seconds; leaked biometric data is compromised for life. That's why biometrics are best used as one factor unlocking a local device — not as the single credential guarding a system.

18.9 Security Is People, Too — Program Elements

Every control so far has been technical. The exam explicitly covers the **non-technical** side, and there's a good reason: attackers routinely skip the technology entirely.

Why does this belong in a networking exam? Because the strongest firewall on earth does nothing when an employee is talked into handing over a password by someone claiming to be IT, or when a visitor walks into an unlocked wiring closet and plugs into a switch. Those attacks don't defeat your controls — they go **around** them. A security program has to cover the paths that aren't cables.

- **User awareness** — organization-wide communication that keeps threats visible: simulated phishing emails, posters, reminders about tailgating. The goal is recognition — an employee who has seen a fake login page before is far likelier to spot a real attack.

- **User training** — deeper, often role-specific instruction, usually formal and repeated: how to handle sensitive data, how to report an incident, acceptable use. Awareness raises alertness; training builds skill.
- **Physical access control** — locks, badge readers, cameras, and secured server rooms and wiring closets. **Why does this matter so much?** Because physical access effectively defeats most software security: someone at your switch can plug into any port, reset a device's password through the console, or simply walk out with hardware. Console access and a paperclip beat any password you configured.

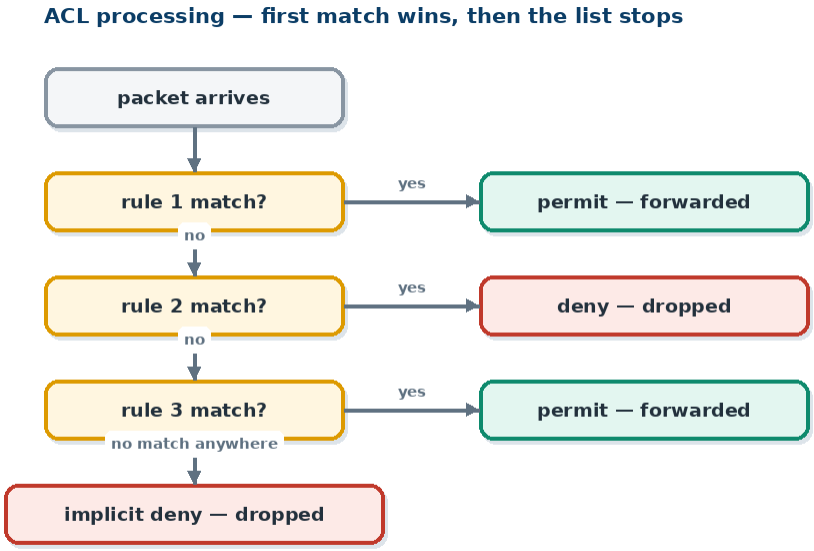
Chapter 19: Access Control Lists (ACLs)

19.1 What Is an ACL?

An **ACL (Access Control List)** is a set of **rules** that tell a router which traffic to **allow** or **deny**. Think of it as a **bouncer with a guest list** at the door of an interface.

Story Time: A bouncer reads names off a list from top to bottom. The FIRST matching rule wins — once he finds your name (allow or deny), he stops reading. And there's a secret rule at the bottom: "anyone NOT on the list, get out!" (the implicit deny).

19.2 How ACLs Are Read (Very Important!)



later rules never see a packet an earlier rule already matched — order is everything

- Rules are checked **top to bottom**.
- The **first match wins** — the rest are ignored.
- At the very end there's an **invisible "deny all"** (implicit deny). So if nothing matches, traffic is **blocked**.

- **Order matters!** Put specific rules before general ones.

An ACL is read **top down**, and the **first match wins**:

1. Does the packet match **rule 1**? If yes — do it (permit or deny) and **stop**.
2. If not, try **rule 2**. Match? Do it and **stop**.
3. …and so on down the list.
4. If nothing matched, the **implicit** `deny any` at the end blocks it.

Nothing after the first match is ever considered — which is why rule order matters so much.

Why does "first match wins" mean ORDER is everything? Because the router stops reading the instant it finds a match — so a rule placed too early can "steal" traffic before a more specific rule below it ever gets a chance. Classic mistake: if you put `permit any` (allow everyone) at the top, every packet matches it immediately and stops — your `deny` rules below are **never even read**. It's like a bouncer whose first list entry says "let everyone in" — the rest of his list is pointless. So the rule is: specific rules first, general rules last. This isn't a style preference — it's forced by the first-match-wins behavior.

Why is there a hidden "deny all" at the bottom? This is a deliberate **security-first** design choice called implicit deny. The philosophy is: "if you didn't explicitly say this traffic is allowed, then it's forbidden." The safe default is **block**, not allow — because forgetting to block something dangerous is far worse than forgetting to allow something harmless. That's why the moment you put ANY ACL on an interface, everything you didn't explicitly permit gets dropped. It also explains the #1 beginner mistake (below): people write a `deny` rule, forget to add `permit` for everyone else, and accidentally block the entire network because the invisible deny-all catches everything they didn't mention.

19.3 Two Kinds of ACLs

- **Standard ACL:** Filters by **source IP only**. Numbered **1–99** (and 1300–1999). Simple. Place it **close to the destination** (because it can only see the source, placing it too early might block too much).
- **Extended ACL:** Filters by **source AND destination IP, protocol, and port**. Numbered **100–199** (and 2000–2699). Powerful. Place it **close to the source** (block unwanted traffic early to save bandwidth).

Memory trick:

- **St**andard = **S**ource only → place near **destination**.
- **Ex**tended = **Ex**tra details → place near **source**.

Why does a STANDARD ACL go near the DESTINATION — isn't blocking early better? Here's the trap: a standard ACL can only see the **source** address, not the destination. So it's a blunt instrument — it says "block traffic FROM host X" without knowing where that traffic was headed. If you placed it near the **source**, it would block that host's traffic to everywhere — including places you wanted to allow! By placing it near the **destination**, you only filter the traffic right before its final stop, so you don't accidentally cut off that host's access to other, legitimate destinations along the way. Its bluntness forces you to apply it late, close to the one destination you actually mean to protect.

Why does an EXTENDED ACL go near the SOURCE? Because an extended ACL is **precise** — it can match source AND destination AND port, so it knows exactly which traffic to drop without collateral damage. Since it's that specific, you want to block unwanted traffic **as early as possible** — right where it enters, near the source. Why bother? Efficiency: why let a packet travel across your whole network, eating bandwidth on every link, only to be dropped at the far end? Kill it at the door. So the placement rules aren't arbitrary — they come directly from how much each ACL type can "see": blunt tools act late to avoid mistakes, precise tools act early to save resources.

19.4 Wildcard Masks (Again!)

ACLs use **wildcard masks** (the opposite of subnet masks). Remember:

- **0** = "must match exactly."
- **255** = "don't care, anything."

Examples:

- `0.0.0.0` = match one exact host.
- `0.0.0.255` = match a whole /24 network.
- `255.255.255.255` (keyword `any`) = match everything.
- The keyword `host 192.168.1.5` means the same as `192.168.1.5 0.0.0.0`.

19.5 Configuring a Standard ACL

Goal: Block PC `192.168.1.50` from reaching the network, but allow everyone else.

```
R1(config)# access-list 10 deny host 192.168.1.50   ! block that one PC
R1(config)# access-list 10 permit any               ! allow everyone else
R1(config)# interface gi0/1
R1(config-if)# ip access-group 10 out               ! apply it going OUT
```

> If you forget `permit any`, the implicit deny blocks EVERYONE. A classic mistake!

19.6 Configuring an Extended ACL

Goal: Let LAN users browse the web (HTTP/HTTPS) but block everything else to a server.

```
R1(config)# access-list 100 permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 80
R1(config)# access-list 100 permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 443
R1(config)# access-list 100 deny ip any any         ! (optional, makes deny visible)
R1(config)# interface gi0/0
R1(config-if)# ip access-group 100 in               ! apply going IN
```

Reading `permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 80`: "Allow TCP from any host in 192.168.1.x to the server 10.0.0.5 on port 80 (web)."

19.7 Named ACLs (Easier to Read)

Instead of numbers, use names:

```
R1(config)# ip access-list extended WEB_ONLY
R1(config-ext-nacl)# permit tcp any host 10.0.0.5 eq 443
R1(config-ext-nacl)# deny ip any any
```

19.8 Checking ACLs

```
R1# show access-lists              ! see all ACLs and hit counts
R1# show ip interface gi0/0        ! see which ACL is applied where
```

Direction tip: `in` filters traffic **entering** the interface; `out` filters traffic **leaving** it. Picture yourself standing inside the router looking at each door.

Chapter 20: Wireless Networking (Wi-Fi)

20.1 How Wireless Works

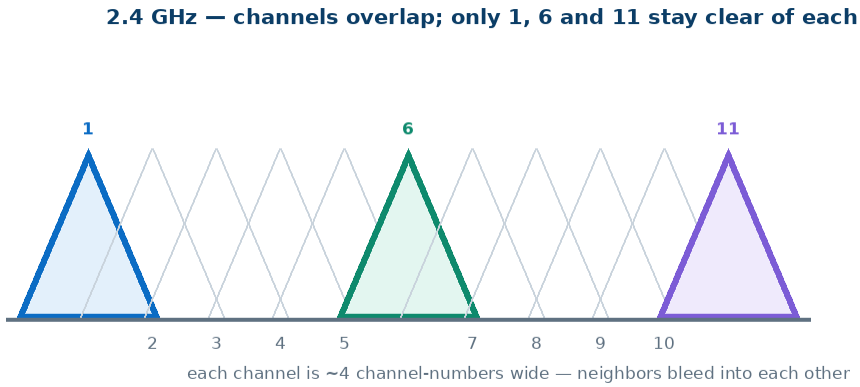
Wireless uses **radio waves** through the air instead of cables. An **Access Point (AP)** is the device that broadcasts the Wi-Fi and bridges wireless devices to the wired network.

Wireless clients — phones, laptops — connect over **radio waves** to an **access point**. The AP is itself plugged into a **switch** on the wired network, so it acts as the bridge between the two worlds.

20.2 SSID — The Network Name

The **SSID (Service Set Identifier)** is the **Wi-Fi network's name** you see when you connect (like "CoffeeShop_Guest"). It's just a friendly label for a wireless network.

20.3 Frequency Bands & Channels



Wi-Fi uses two main **bands**:

Band	Speed	Range	Crowded?
2.4 GHz	Slower	Longer range, better through walls	Very crowded
5 GHz	Faster	Shorter range	Less crowded
6 GHz (Wi-Fi 6E)	Fastest	Shortest	Newest, roomy

Why does 2.4 GHz go FARTHER but 5 GHz go FASTER — what's the tradeoff? This is physics, and it's the same tradeoff for all radio waves. **Lower frequencies (2.4 GHz) have longer, lazier waves** that push through walls and travel far, but they carry less data. **Higher frequencies (5/6 GHz) have shorter, tighter waves** that carry way more data (faster) but get absorbed by walls and fade over distance. So there's no "best" band — it's a genuine choice: need coverage far across a house through walls? 2.4 GHz. Sitting near the router and want speed? 5/6 GHz. It's like sound: a deep bass (low frequency) travels through walls to the next room, while a high-pitched whisper (high frequency) is clearer up close but doesn't carry. **Why is 2.4 GHz so "crowded"?** Because everything uses it — old Wi-Fi, Bluetooth, microwaves, baby monitors, garage doors — all crammed into a small band, so they interfere. 5/6 GHz have far more room, which is a big reason they're faster in practice.

Channels are like lanes on the road. On 2.4 GHz, the **non-overlapping channels are 1, 6, and 11** — using these avoids interference. (Memorize 1, 6, 11!)

In 2.4 GHz, only **channels 1, 6 and 11** are far enough apart not to overlap. Spacing your APs across those three keeps neighbouring cells from interfering.

Why specifically 1, 6, and 11 — why not 1, 2, 3? Because each 2.4 GHz channel is actually **wider than the spacing between channel numbers**, so neighboring channels physically overlap and bleed into each other. Picture each channel as a fat highway lane painted wider than the lines: channels 1 and 2 are so close their traffic smears together, causing interference — which is worse than being on the same channel! It turns out you need about **5 channels of gap** for two signals not to overlap. Do the math: 1, then 1+5=6, then 6+5=11. Those three are the only combination that fits in the 2.4 GHz band with **zero overlap**. That's why every Wi-Fi pro memorizes 1/6/11 — it's the one way to run three nearby access points without them stepping on

each other. (This overlap problem is another reason 5 GHz is nicer: it has many more non-overlapping channels to spread out on.)

20.4 Wireless Standards (802.11 Family)

Standard	Nickname	Band	Max Speed
802.11b	—	2.4 GHz	11 Mbps
802.11g	—	2.4 GHz	54 Mbps
802.11n	Wi-Fi 4	2.4 & 5 GHz	600 Mbps
802.11ac	Wi-Fi 5	5 GHz	~3.5 Gbps
802.11ax	Wi-Fi 6/6E	2.4/5/6 GHz	~9.6 Gbps

Why is Wi-Fi "half-duplex" and shared — why does it feel slower than the numbers suggest? Because radio is like **everyone talking in one room** — only one device can transmit at a time on a channel, or the signals collide (this is why Wi-Fi uses CSMA/**CA**, collision avoidance — it can't detect collisions in the air, so it politely waits and avoids them). The more devices on one AP, the more they take turns, so real speed per device drops. That's why wired connections (full-duplex, own wire) are still preferred for anything that needs guaranteed speed, and why packing 30 people onto one coffee-shop AP feels sluggish even on fast Wi-Fi.

20.5 Wireless Security

Standard	Safety	Notes
WEP	Broken	Ancient, easily cracked — never use
WPA	Weak	Old
WPA2	Good	Uses strong AES encryption; common
WPA3	Best	Newest, strongest — replaces the PSK handshake with **SAE**

What does WPA3's SAE actually fix? WPA2-Personal has a known weakness: an attacker can record the 4-way handshake from the parking lot and then guess passwords **offline**, millions per second, until one produces the same handshake. WPA3 replaces that handshake with **SAE (Simultaneous Authentication of Equals)** — each connection attempt requires a live exchange with the AP, so there's nothing useful to take home and crack. One guess per attempt, with the AP watching. Same password, but brute-forcing it now happens at the AP's pace instead of a GPU's.

Two modes:

- **Personal (PSK):** One shared password. Good for homes.
- **Enterprise (802.1X):** Each user logs in with their own account via RADIUS. Good for businesses.

Why does wireless need its OWN security when wired networks mostly don't? Because of one huge difference: **Wi-Fi signals leak out into the air, past your walls, into the street.** On a wired network, an attacker has to physically plug in to eavesdrop. But Wi-Fi is literally broadcasting your data as radio waves that anyone nearby can silently capture from a parked car — no plugging in required. That's why Wi-Fi must encrypt everything by default: without encryption, your traffic is a public radio broadcast. This is also why the security standards keep evolving (WEP→WPA→WPA2→WPA3): as attackers found ways to crack each one, we needed stronger encryption, because the stakes (invisible, remote eavesdropping) are so high.

Why choose Personal vs Enterprise mode? With **Personal (PSK)**, everyone shares one password. Fine at home, but in a company it's a nightmare: if one employee leaks the password or leaves, you must change it and re-tell everyone. And you can't tell who's who — all users look identical. **Enterprise (802.1X)** fixes this by giving each person their own login (via a RADIUS server, from Chapter 18). Why bother? You can disable one person's access without affecting anyone else, and you get a record of who connected. Same reason we centralize AAA: individual accountability and easy revocation.

20.6 AP Types & Wireless LAN Controllers

- **Autonomous AP:** Standalone, configured one-by-one. Fine for a few APs.
- **Lightweight AP + WLC:** Many APs managed centrally by a **Wireless LAN Controller (WLC)**. The WLC handles settings, security, and roaming for all APs at once — much easier for big networks.

A **WLC** (Wireless LAN Controller) is the brain: the **lightweight APs** below it hold no configuration of their own and simply carry out its instructions — which is what makes managing hundreds of APs practical.

The lightweight APs talk to the WLC using a tunnel protocol called **CAPWAP**.

Why use a central WLC instead of just configuring each AP? Two big reasons that appear the moment you have more than a handful of APs:

1. **Management sanity.** Configuring 200 autonomous APs one-by-one — and changing the Wi-Fi password on all 200 by hand — is a nightmare (same "500 switches" problem from the security chapter). A WLC lets you set policy **once** and push it to every AP. Change the password in one place, done.
2. **Seamless roaming.** This is the subtle one. When you walk through a building on a video call, your phone hands off from AP to AP. Without a controller, each AP is an island and that handoff is clunky (drops, re-authentication). The WLC **coordinates all the APs as one system**, so it can move your connection smoothly from one AP to the next, manage which channels each AP uses to avoid interference, and balance load. That coordination is impossible when each AP acts alone — which is exactly why big networks use the lightweight-AP-plus-WLC model.

> **How does a ceiling-mounted AP get power?** Almost always from the switch, over the same Ethernet cable that carries its data — see section **3.7 (PoE)**. It's also why APs and WLCs need the trunk/access port planning from Chapters 7 and 8.

The four deployment architectures — the exam expects you to compare them, and each exists because it fixes the previous one's pain at a particular size of network:

Architecture	Where the "brain" lives	Best for
Autonomous	In each AP individually	A handful of APs
Split-MAC (lightweight + WLC)	Central WLC appliance; APs tunnel to it via CAPWAP	Campus with many APs
Cloud-based (e.g. Cisco Meraki)	A controller in the vendor's cloud, managed from a web dashboard	Many small sites, lean IT teams
Embedded / Mobility Express	WLC software running **inside** a switch or inside one of the APs	Small/branch sites that want central management without buying a controller

Why is the lightweight model called "split-MAC"? Because the 802.11 (MAC-layer) job gets **split in two**. The **real-time** work that can't tolerate delay — transmitting frames, acknowledgements, encryption, beacons — stays **on the AP**, right at the radio. The **management** work — authentication decisions, roaming, channel/power planning, applying policy — moves **to the WLC**. Time-critical stays local; decisions centralize. CAPWAP is the pipe between the halves: a **control tunnel (UDP 5246)** carrying the WLC's instructions, and a **data tunnel (UDP 5247)** carrying client traffic — control messages are encrypted with **DTLS**; data usually isn't unless you turn it on.

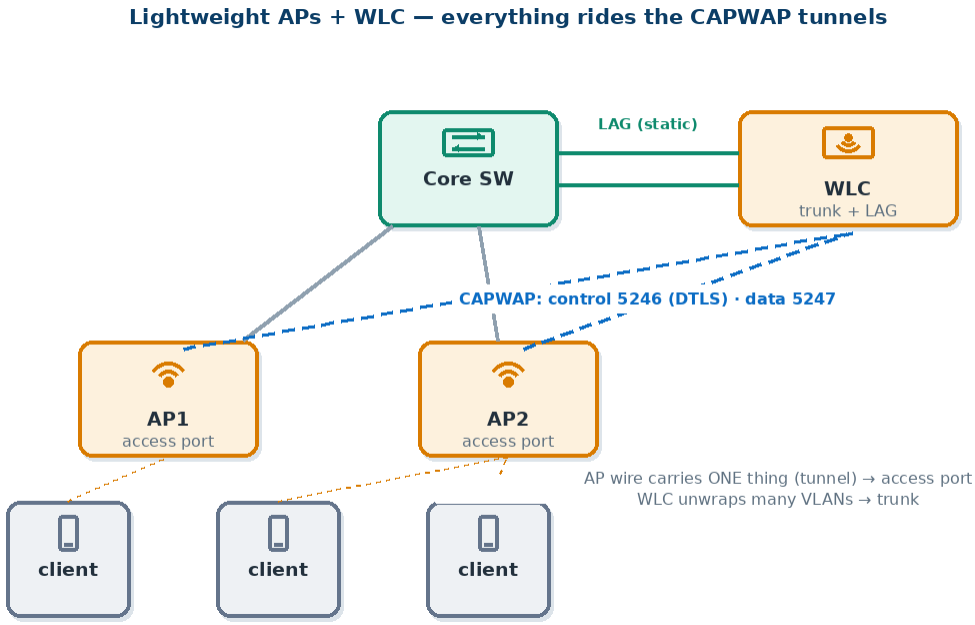
20.7 AP Modes — One AP, Many Jobs

A lightweight AP isn't locked into serving clients; the WLC can assign it a **mode**. Two forward traffic, the rest turn the AP into a dedicated tool:

Mode	What the AP does
Local	The default — serves clients, tunnels **all** traffic to the WLC through CAPWAP
FlexConnect	Serves clients but can **switch traffic locally** at the site instead of tunneling it — and keeps working if the WAN link to the WLC dies
Monitor	No clients — radio only **listens**, hunting rogue APs and interference
Sniffer	Captures 802.11 frames and streams them to an analyst's Wireshark
Rogue detector	Sits on the **wired** side, matching wired MACs against rogue APs heard over the air to prove an intruder AP is on your network
Bridge / Mesh	Point-to-point or mesh backhaul — a wireless cable between buildings
SE-Connect	Dedicated **spectrum analysis** — finds non-Wi-Fi interferers (microwaves, cordless phones)

Why does FlexConnect exist — isn't tunneling everything the whole point? Picture a branch office in another city, its WLC back at HQ. In **local mode**, a print job from a branch laptop to the branch printer ten feet away would travel across the WAN to HQ and back — silly. And if the WAN link fails, Wi-Fi at the branch dies entirely, even though the AP and the printer are both fine. **FlexConnect lets the AP drop local traffic onto the local LAN directly**, keeping only management centralized — and if the WLC becomes unreachable, the AP keeps serving its existing WLANs on its own. Central control when available, local survival when not. That's the exam's favorite fact about it.

20.8 Wiring It Up — How APs and WLCs Physically Connect



This is pure Chapter 7/8 knowledge applied to wireless, and the logic falls out of one question: who needs to see which VLANs?

- **A lightweight AP plugs into an access port.** Surprising until you remember: every client's traffic rides **inside the CAPWAP tunnel**, so the wire only ever carries one thing — CAPWAP packets between AP and WLC. One kind of traffic = one VLAN = access port. The VLAN sorting happens at the WLC end, where the tunnel is unwrapped.
- **The WLC plugs into a trunk port.** The reverse reasoning: the WLC unwraps tunnels from every AP and must drop each client onto the right VLAN — employee traffic to the employee VLAN, guest to guest. Many VLANs on one link = trunk.
- **An autonomous AP with multiple SSIDs also needs a trunk** — with no tunnel to carry the sorting elsewhere, it must tag each SSID's VLAN itself.
- **A WLC bundles its trunk ports with LAG** (link aggregation — the same EtherChannel idea from Chapter 10): more bandwidth for hundreds of APs' worth of tunnels, and no single-cable failure takes down the whole wireless network. One WLC quirk to remember: Cisco WLCs use **static LAG only — no LACP/PAgP negotiation** (`channel-group X mode on` at the switch end).

The WLC also keeps several **logical interfaces**, each answering a different "who's talking to me?":

WLC interface	Purpose
Management	Its main IP — CAPWAP from APs, SSH/HTTPS from admins
Virtual	A fake, unroutable address (often 192.0.2.1) used for client-facing chores like DHCP relay and the web login page
Dynamic	One per client VLAN — the exit door a WLAN's traffic uses onto the wired network
Service port	Out-of-band management — a lifeline that works even if the network config is broken
Redundancy	Heartbeat link to a standby WLC

20.9 Managing APs and the WLC

Same menu as any Cisco device (21.1), applied to wireless — with the same security instincts:

- **CLI:** console, Telnet (cleartext — disabled by default on WLCs), or **SSH**.
- **GUI:** the WLC is mostly driven from its **web interface** — HTTP () or **HTTPS**. This is the practical difference from switches: day-to-day WLC work is GUI work, which is why the exam asks you to recognize its screens.
- **Centralized admin login:** point the WLC at **TACACS+ or RADIUS** (the AAA story from 18.4), so admin accounts live in one place instead of on every controller.
- A lightweight AP normally isn't managed directly at all — that's the entire point of owning a WLC — though its console port still matters for troubleshooting an AP that can't find its controller.

20.10 Creating a WLAN in the WLC GUI (What the Exam Shows You)

The exam gives you **screenshots** of this workflow, so walk it once. A **WLAN** on a WLC is a profile that ties three things together: an **SSID** (what clients see), a **dynamic interface** (which VLAN the traffic exits onto), and a **security policy**. Order of operations:

1. **Create the dynamic interface** (Controller → Interfaces): name, VLAN ID, IP address — the WLAN's door to the wired network.
2. **Create the WLAN** (WLANs → Create New): Profile Name (internal label), SSID (broadcast name), then **enable it** — new WLANs start disabled, a classic "why isn't it showing up?" gotcha.
3. **General tab:** map the WLAN to that dynamic interface.
4. **Security tab:** Layer 2 security — **WPA2/WPA3**, then either **PSK** (type the passphrase) or **802.1X** (point at your RADIUS server).
5. **QoS tab:** pick a profile — **Platinum (voice), Gold (video), Silver (best effort — the default), Bronze (background)**. This decides how the WLC marks the traffic for the QoS machinery from 17.7; guest Wi-Fi often gets Bronze so it can't crowd out real work.

6. **Advanced tab:** the long tail — session timeouts, band steering, FlexConnect local switching.

Memory hook for the QoS tiers: medals in order — Platinum, Gold, Silver, Bronze — and voice always takes the podium.

20.11 How a Device Joins Wi-Fi

1. **Discover:** Device listens for beacons or probes for the SSID.
2. **Authenticate:** Proves it's allowed (password/802.1X).
3. **Associate:** Officially joins the AP.
4. **Get IP (DHCP):** Receives an address and starts using the network.

Chapter 21: Network Management & Monitoring

21.1 Ways to Connect to a Device

- **Console (out-of-band):** A direct cable — works even if the network is down. Your lifeline for first setup or emergencies.
- **SSH (in-band):** Secure remote access over the network. The everyday way.
- **Telnet (in-band):** Remote access but **unencrypted** — avoid!

Out-of-band vs In-band: Out-of-band uses a separate path (console cable) that doesn't rely on the network working. In-band uses the network itself.

Why keep an "out-of-band" console path when SSH over the network is so much more convenient? Because of a brutal catch-22: the times you most need to fix a device are exactly the times the network is broken — and if the network is down, **SSH (which travels over that same network) won't reach the device either!** It's like keeping a spare key outside the house: locking yourself out is precisely when the key inside is useless. The **console cable** is a completely separate, physical path that works even when the device has no IP, no config, or a totally broken network. That's why every network pro keeps console access: it's the lifeline for a brand-new switch (no config yet), a misconfiguration that killed remote access, or a network meltdown. In-band (SSH) is for everyday convenience; out-of-band (console) is for emergencies — and you need both.

21.2 Discovering Neighbors: CDP & LLDP

- **CDP (Cisco Discovery Protocol):** Cisco devices automatically learn about **directly connected Cisco neighbors** (name, model, port, IP). Great for mapping.

- **LLDP (Link Layer Discovery Protocol):** The **vendor-neutral** version (802.1AB) — works across brands.

```
R1# show cdp neighbors           ! quick list of neighbors
R1# show cdp neighbors detail    ! includes their IP addresses
R1# show lldp neighbors
```

> Security tip: Turn CDP/LLDP off on ports facing untrusted networks — they reveal device info.

Why is CDP incredibly useful AND a security risk at the same time? Both come from the same fact: CDP freely broadcasts details about your device (its name, model, IOS version, IP, and which port connects where) to whatever's plugged in.

- **Why it's useful:** In a big messy wiring closet, you can sit on one switch and instantly map the whole neighborhood — "port 5 connects to Switch-B's port 12, which is a Catalyst running this IOS." You can rebuild a network diagram without physically tracing a single cable. A huge time-saver for documentation and troubleshooting.
- **Why it's a risk:** That same free information is a **gift to an attacker.** If a hacker plugs into a port that's leaking CDP, they instantly learn your device models and IOS versions — which tells them exactly which known vulnerabilities to try. So the rule is: keep CDP/LLDP ON internally (where it helps you) but turn it OFF on ports facing untrusted areas (guest ports, internet edges), because there's no reason to hand strangers a map of your gear.

21.3 Backing Up & Restoring Configs

Save configs to a **TFTP/FTP server** so you can restore after a failure:

```
R1# copy running-config tftp        ! back up to a server
R1# copy tftp running-config        ! restore from a server
R1# copy running-config startup-config   ! save locally (do this always!)
```

21.4 Managing IOS Images

The IOS image lives in **flash**; upgrading means copying a new image in, telling the device to boot from it, and reloading. The exam cares about the workflow and the two safety habits:

```
R1# show flash:                        ! space free? current image name?
R1# show version                       ! running version + config register
R1# copy tftp flash:                   ! pull the new image (or FTP/SCP/USB)
R1# verify /md5 flash:new-image.bin    ! checksum BEFORE trusting it
R1(config)# boot system flash:new-image.bin
R1# copy running-config startup-config
R1# reload
```

- **Why verify the checksum?** A corrupted transfer (or a tampered image) can leave you a device that won't boot — comparing the MD5/SHA hash against Cisco's published value costs one command and prevents the 2 a.m. surprise.

- **Why keep the old image in flash?** If the new one fails, you point `boot system` back at the known-good file. Delete it only after the new image has survived a reload.
- **TFTP vs FTP:** TFTP is the classic, dead-simple choice (UDP 69, no login); FTP adds authentication and handles big files more reliably. Both appear on the blueprint as "ways to move configs and images."

21.5 Password Recovery (The Config Register)

If you're locked out, the fix hinges on one 16-bit value: the **configuration register**, which tells the router how to boot. Normally it's `0x2102` — boot the IOS, load startup-config as usual. Changed to `0x2142`, one bit flips meaning: **ignore the startup config at boot**.

The recovery walk (console cable required — this cannot be done remotely, which is the point):

1. Power-cycle and break into **ROMMON** (Ctrl+Break during boot).
2. `confreg 0x2142`, then `reset` — the router boots fresh, no config, no passwords.
3. `copy startup-config running-config` — pull the real config back in (now you're logged in around it).
4. Change the passwords, set the register back with `config-register 0x2102`, save, reload.

Why does this feature exist — isn't it a security hole? It's the deliberate trade-off from 18.9: anyone with **physical access** to the console effectively owns the device, so Cisco chose recoverable-by-owner over bricked-by-forgotten-password. That's exactly why wiring closets get locks. And the exam angle: if a router boots up ignoring its startup config "for no reason," check `show version` — someone left the register at 0x2142.

21.6 Useful Monitoring Commands

Command	Shows
`show processes cpu`	CPU usage
`show memory`	Memory usage
`show interfaces`	Detailed port stats, errors
`show logging`	Syslog messages stored locally
`show ip arp`	IP-to-MAC mappings

Chapter 22: Automation & Programmability

22.1 Why Automate?

Doing the same command on 500 switches by hand is slow and error-prone. **Automation** lets computers configure networks quickly, consistently, and without typos. This is a growing part of modern networking (and the CCNA).

Story Time: Imagine writing the same birthday card 500 times by hand vs. printing them. Automation is the printer — same result, way faster, no cramped hand!

22.2 Controller-Based Networking (SDN)

SDN (Software-Defined Networking) separates the network's "brain" from its "muscles":

- **Control plane** (the brain): Decides where traffic should go.
- **Data plane** (the muscles): Actually forwards the traffic.

In traditional networking, every device has its own brain. In SDN, a central **controller** is the brain for everyone, and devices just follow orders. Cisco's campus controller is **Cisco Catalyst Center** (renamed from **DNA Center** — the exam may use either name).

Model	Where the thinking happens
Traditional	Every device decides for itself
SDN	One **controller** decides, and the devices carry it out

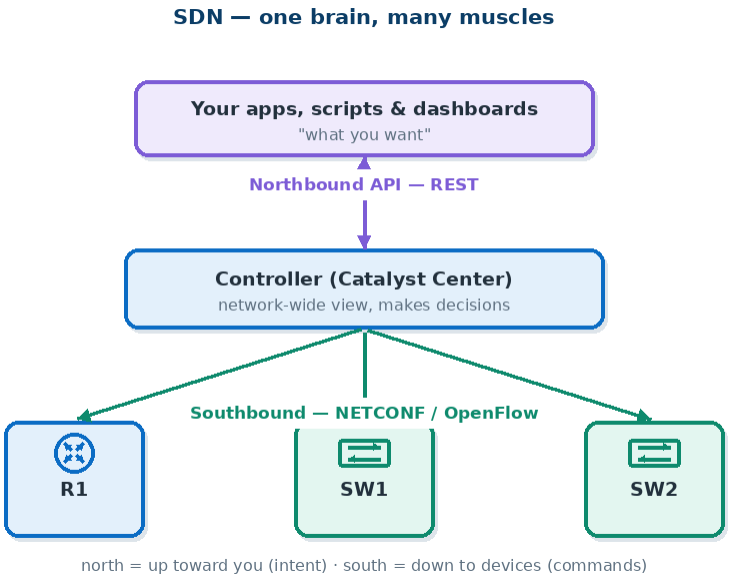
What does that difference feel like day to day? The exam asks you to compare managing a campus the traditional way against managing it through Catalyst Center:

Task	Traditional (per-device)	With Catalyst Center
Config change	SSH to each device, type CLI, hope you didn't typo one of them	Define the **intent** once; the controller pushes consistent config everywhere
New switch	Console in, configure by hand	**Plug-and-play** provisioning — it's discovered and configured automatically
Monitoring	Poll devices one by one (SNMP, `show` commands)	Continuous **assurance** — telemetry from everywhere, analyzed centrally
Software updates	Manually, per device, at 2 a.m.	Scheduled and rolled out by the controller
Interface	CLI per box	GUI dashboard + a **REST API** for automation

The traditional way still works — Catalyst Center even talks to the same IOS devices underneath. What changes is **where the intelligence sits** and how many places a human has to touch.

Why separate the "brain" from the "muscles" at all? Because in traditional networking, every switch and router has its own brain making its own decisions — which means to change network-wide policy, you have to log into every single device and configure it separately. With hundreds of devices, that's slow, and worse, they can drift into slightly different ("snowflake") configs that cause weird bugs. SDN's insight: **pull all the decision-making into one central controller**, and let the devices just be fast, simple forwarders that follow orders. Why is that powerful? Now you set policy **once** in the controller and it programs every device consistently — like conducting an orchestra from one podium instead of running to each musician individually. It also means the controller has a **complete view** of the whole network, so it can make smarter, coordinated decisions than any single device could on its own.

22.3 Northbound vs. Southbound APIs



The controller talks in two directions:

- **Northbound API:** Up to apps/humans (e.g., a REST API you program against).
- **Southbound API:** Down to the network devices (e.g., NETCONF, OpenFlow).

The controller sits in the middle, with an API facing each direction:

API	Faces	Talks to	Typically
Northbound	Upward	Apps and you	REST
Southbound	Downward	Switches and routers	NETCONF, OpenFlow

Why two different directions with different names? Because the controller sits in the middle and talks to two very different audiences, so it needs a "language" for each:

- **Northbound (upward, toward humans/apps):** This is how you (or your automation scripts, or a dashboard) tell the controller what you want — "make sure the sales VLAN reaches

these buildings." It's designed to be **easy for humans and software to use** (a friendly REST API), because the audience is people and programs. Think "up = toward the people giving orders."

- **Southbound (downward, toward devices):** This is how the controller pushes those decisions down to the actual switches and routers, using device-friendly protocols (NETCONF, OpenFlow). The audience is machines, so the language is more technical.

Why does this split matter? It means you can describe your intent in simple, human terms at the top, and the controller handles translating that into the nitty-gritty device commands at the bottom. **Memory hook:** North = up toward you (the boss giving orders); South = down toward the devices (the workers doing the job).

22.4 APIs & REST

An **API (Application Programming Interface)** is a way for programs to talk to each other. A **REST API** uses normal web methods:

Method	Meaning	Example
GET	Read data	"Show me this device's config"
POST	Create	"Add a new VLAN"
PUT	Update/replace	"Change this setting"
DELETE	Remove	"Delete this VLAN"

REST APIs usually exchange data in **JSON** format.

Those four methods have a name: CRUD — **C**reate, **R**ead, **U**pdate, **D**elete. They're the four things you can do to any piece of data, and REST deliberately maps them onto HTTP verbs that already existed:

CRUD action	HTTP verb
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE

Why reuse web verbs instead of inventing something for networking? Because the entire internet already speaks HTTP. Every programming language, firewall, load balancer, and proxy handles it, and any engineer already knows what GET means. Reusing it meant network APIs worked everywhere on day one, with no new protocol to learn — the same reason we don't invent a new alphabet every time we write a new book.

How does a REST API know who's calling? Since every request stands alone (see statelessness below), every request must carry proof of identity. The blueprint names the common schemes:

Auth type	How it works	Trade-off
Basic	Username:password sent (Base64-encoded — not encrypted) in a header	Simple, but only safe inside HTTPS
API key	A long secret string identifying the calling app	Easy to issue and revoke, but it's a password by another name
Bearer token / OAuth	You authenticate **once**, receive a short-lived **token**, and send that instead	Credentials aren't repeated on every call, and a stolen token expires soon

The pattern to remember: whatever the scheme, it rides along in every single request — and none of them are safe over plain HTTP.

REST APIs are also stateless: every request must carry **everything** needed to answer it (including authentication) — the server remembers nothing between calls. **Why insist on that?** Because a server that remembers nothing is a server you can duplicate freely. Any of ten identical servers can answer any request, so you scale by adding more, and a crashed server loses no conversation. Statelessness is what makes an API scalable.

The reply carries a **status code** telling you what happened — worth recognizing on sight:

Code	Family	Meaning
200 / 201	2xx Success	OK / Created
400	4xx **Your** mistake	Bad request (malformed)
401 / 403	4xx Your mistake	Not authenticated / not allowed
404	4xx Your mistake	Not found
500	5xx **Server's** mistake	Internal server error

*The shortcut: *4xx means you sent something wrong, 5xx means the server broke. That one distinction tells you immediately whether to fix your script or go look at the controller.

22.5 Data Formats: JSON, XML, YAML

JSON (most common) — uses `{ }` and `key: value`:

```
{
  "device": "Router1",
  "interfaces": ["Gi0/0", "Gi0/1"],
  "enabled": true
}
```

YAML — clean, uses indentation (popular with Ansible):

```
device: Router1
interfaces:
  - Gi0/0
  - Gi0/1
enabled: true
```

XML — uses tags like HTML:

```
<device>Router1</device>
```

22.6 Automation Tools

Tool	Language	Style	Agent Needed?
Ansible	YAML	Push, simple	No (agentless)
Terraform	HCL	Declarative, push	No (agentless)
Puppet	Ruby-like	Pull	Yes (agent)
Chef	Ruby	Pull	Yes (agent)

Ansible is the CCNA favorite because it's **agentless** (nothing to install on devices) and uses easy-to-read YAML "playbooks."

Ansible and Terraform are the two named in the current (v1.1) exam blueprint — Puppet and Chef are worth recognizing, but they're the older, agent-based generation. **Why does "agentless" keep coming up as an advantage?** Because an agent is software you must install, update, and secure **on every single device** — and plenty of network gear won't let you install anything at all. Agentless tools connect over SSH or an API that the device already offers, so there's nothing to deploy before you can start. See 22.9 for how Terraform's declarative style differs from a playbook that runs steps in order.

22.7 Config Management Idea: Intent & Consistency

Automation lets you describe the network you WANT ("intent"), and tools make reality match it — and keep it that way. No more "snowflake" devices that all drifted to slightly different settings.

22.8 AI & Machine Learning in Network Operations

Automation (so far) does **what you told it to do**, exactly and repeatedly. The newer question is whether the network can help decide **what should be done** — and this is now an explicit exam topic.

Why bring AI into networking at all? Because of a mismatch that keeps getting worse: a modern network produces a **staggering** amount of telemetry — every device, interface, wireless client, and application reporting constantly. No human team can read it all. So traditional monitoring uses **fixed thresholds** ("alert if CPU > 80%"), which fail in both directions: they scream about a harmless spike at 2 a.m., and they stay silent while a link slowly degrades from 5 ms to 40 ms of latency — never crossing a threshold, but ruining every call in the building.

Machine learning attacks that differently. Instead of a human guessing the right number, the system **learns what normal looks like** for this network — including that Monday mornings are busy and the backup runs at 1 a.m. — and flags **deviations from that baseline**. Nobody has to define "normal" in advance.

Term	What it means
AI	The broad goal: machines performing tasks that need human-like judgment
ML	The practical method: learning patterns from data instead of following fixed rules
AIOps	Applying AI/ML to IT and network operations
Predictive analytics	Spotting a trend and warning **before** it becomes an outage

Where it shows up in real networks:

- **Anomaly detection** — "this switch's traffic pattern is unlike anything in the last 90 days."
- **Predictive maintenance** — "this AP's error rate is rising steadily; it will likely fail soon."
- **Root cause analysis** — collapsing 400 simultaneous alarms into one sentence: "this uplink went down; everything else is a symptom."
- **Wireless optimization** — learning interference patterns and adjusting channels and power automatically.
- **Capacity planning** — projecting when a link will saturate based on real growth, not guesswork.

Predictive vs. generative — the exam's key distinction. The blueprint (v1.1) explicitly names **two kinds of AI**, and they do opposite jobs:

Aspect		
Job	**Forecasts** from patterns in data	**Creates** new content
Answers	"What is about to happen?"	"Write/make this for me"
In networking	Anomaly detection, failure prediction, capacity planning — everything described above	Drafting configs and ACLs, explaining a cryptic log in plain English, chat-style troubleshooting assistants, writing documentation
Example	"This AP will likely fail within two weeks"	"Generate the OSPF config for these three routers"

The one-line separator: predictive AI watches and warns; generative AI writes and explains. Modern tools combine them — an assistant built into Catalyst Center might use predictive models to spot the anomaly, then generative AI to explain it and draft the fix. And the same caution applies double to generative AI: it produces **plausible** output, not guaranteed-correct output, so a human reviews any config it writes before it touches production.

The honest limits — worth knowing, because the exam frames AI as an aid rather than a replacement. ML systems learn from **historical data**, so they inherit its blind spots: a network that was misconfigured for a year has learned that misconfiguration as "normal." They produce **probabilities, not certainties**, so they can be confidently wrong. And many models can't fully explain why they flagged something, which is uncomfortable when you're deciding whether to reroute production traffic. AI is a very fast assistant that reads everything and never gets tired —

not an engineer. A human still owns the decision.

22.9 Infrastructure as Code — Terraform & Friends

Section 22.6 introduced automation tools. One idea underneath them deserves its own name, because it's how modern infrastructure is actually managed: **Infrastructure as Code (IaC)** — you describe the **desired state** in a text file, keep that file in version control, and let a tool make reality match it.

Why is writing a file better than just configuring the device? Because a config typed into a device is **invisible history**. Six months later, nobody knows who changed that ACL, why, or what it looked like before. When the file is the source of truth, your infrastructure gains everything software engineering already figured out: **version control** (every change dated and attributed), **code review** (a second person checks before it ships), **rollback** (restore the previous file), and **reproducibility** (build an identical test environment from the same description).

Tool	Style	Agent needed?	Typical use
Ansible	Procedural-ish, push over SSH/API	**Agentless**	Device configuration, ad-hoc tasks
Terraform	**Declarative**, state-file driven	Agentless	Provisioning infrastructure (cloud, network)
Puppet / Chef	Pull-based, agent on each node	**Agent**	Server config (older, less common in CCNA v1.1)

Declarative vs. procedural is the distinction to hold onto. A **procedural** tool is a recipe: "add this VLAN, then this interface." Run it twice and you may get errors or duplicates. A **declarative** tool is a description: "this network has VLANs 10, 20, 30." The tool inspects what exists, computes the difference, and changes **only what doesn't match**. Run it ten times and nothing extra happens — a property called **idempotence**. You describe the destination, not the driving directions.

Terraform is the exam's named example of this style: it keeps a **state file** recording what it built, so it always knows the gap between "what you asked for" and "what exists."

Chapter 23: The Troubleshooting Toolbox

23.1 A Simple Method: Follow the Layers

When something's broken, check the OSI layers **from the bottom up**:

```
1. Physical: Is the cable plugged in? Link light on?
2. Data Link: Right VLAN? Interface up/up? MAC learned?
3. Network: Correct IP, mask, gateway? Can you ping?
4. Transport: Right ports open? ACL blocking?
5-7. App: Is DNS working? Is the service running?
```

Story Time: It's like a car that won't start. Don't rebuild the engine first! Check the simple stuff: Is there gas? Is it in park? Is the battery dead? Start low and simple, then work up.

Why troubleshoot from the BOTTOM (Layer 1) UP instead of top-down? Because the layers depend on each other — each one is built on the layer below it. Layer 3 (IP) literally cannot work if Layer 1 (the cable) is unplugged, and Layer 7 (your app) cannot work if Layer 3 has no route. So a problem at a low layer **makes everything above it look broken too.** If you start at the top, you might spend an hour debugging a "DNS problem" (Layer 7) when the real issue is a dead cable (Layer 1) — you'd be trying to fix the symptom while ignoring the cause. Checking bottom-up means you fix the **foundation first**: confirm the cable, then the VLAN/link, then the IP, then the ports, then the app. The lower stuff is also usually faster and cheaper to check — glancing at a link light takes 2 seconds, so it's silly to debug complex software before verifying the wire. You rule out the simple, foundational causes before touching the complicated ones. (There's also a "divide and conquer" approach where you start in the middle with a ping, but bottom-up is the safest default and the one the exam loves.)

23.2 The Essential Troubleshooting Commands

Command	What It Tests
`ping <ip>`	Can I reach that device? (basic connectivity)
`traceroute <ip>`	What path do packets take? Where do they stop?
`show ip interface brief`	Are my interfaces up with IPs?
`show ip route`	Do I know how to reach that network?
`show cdp neighbors`	What's connected to me?
`show mac address-table`	Which MAC is on which port?
`show vlan brief`	Are ports in the right VLANs?
`show interfaces`	Errors, drops, duplex mismatches?
`show running-config`	What's actually configured?

23.3 Ping — Your Best Friend

Ping sends a tiny "are you there?" message and waits for a reply.

```
R1# ping 192.168.1.1
!!!!!   ← exclamation marks = success! (5 replies)
.....   ← dots = no reply (failure)
```

Ping order for troubleshooting: 1. Ping **yourself** (127.0.0.1) — is my network software OK? 2. Ping your **own IP** — is my card OK? 3. Ping your **gateway** — can I reach the router? 4. Ping a **remote device** — can I get out? 5. Ping by **name** (ping google.com) — is DNS working?

If step 4 works but step 5 fails, it's a **DNS** problem!

Why ping in exactly THIS order — what's the logic? Because each step tests one more piece of the chain, moving outward from yourself to the far internet, so **the first step that fails**

points straight at the problem. It's a deliberate process of elimination:

1. **Ping yourself (127.0.0.1):** Tests whether your computer's own networking software (the TCP/IP stack) even works. If this fails, the problem is inside your PC — nothing external matters yet.
2. **Ping your own IP:** Tests your network card and its configuration. Now you've confirmed your PC can talk to itself through its real address.
3. **Ping your gateway:** Tests whether you can reach the router — i.e., is your local network (cable, switch, VLAN, gateway setting) working? If steps 1–2 pass but this fails, the problem is between you and the router (local network).
4. **Ping a remote device:** Tests whether the router can get you out to other networks (routing works). If the gateway pings but this doesn't, the problem is beyond your router (routing/ISP).
5. **Ping by name (google.com):** Tests **DNS** specifically. If step 4 (ping by IP) works but step 5 (ping by name) fails, everything except name resolution is fine — so it's a **DNS** problem, full stop.

See the beauty? By walking outward one hop at a time, **the exact step where pings start failing tells you which link in the chain is broken** — no guessing. This is the bottom-up layer method turned into five concrete commands, and it's why experienced engineers can pinpoint a fault in under a minute.

23.4 Common Problems & Fixes

Symptom	Likely Cause	Fix
Interface "administratively down"	Forgot `no shutdown`	Run `no shutdown`
Can ping IP but not name	DNS problem	Check DNS server settings
Two switches won't trunk	VLAN/native/mode mismatch	Match trunk settings
OSPF neighbors won't form	Area/subnet/timer mismatch	Match OSPF settings
PC gets 169.254.x.x	Can't reach DHCP	Check DHCP server/relay
Slow/errors on a link	Duplex mismatch	Set both ends the same
Port shut by security	Port-security violation	`shutdown` then `no shutdown`

23.5 Duplex & Speed Mismatch

If one side is full-duplex and the other half-duplex, you'll see **errors and slowness**. Best practice: let both **auto-negotiate**, or set BOTH ends the same manually.

23.6 Checking IP Settings on a PC (Windows, macOS & Linux)

Troubleshooting usually starts at the **user's computer**, not the router — and the exam expects you to know the client-side commands on all three operating systems.

Task	Windows	macOS	Linux
Show IP / mask / gateway	`ipconfig` (`/all` for detail)	`ifconfig`	`ip address` (`ip a`)
Show routing table	`route print`	`netstat -rn`	`ip route`
Show default gateway	`ipconfig`	`netstat -rn | grep default`	`ip route`
DNS lookup	`nslookup name`	`nslookup` / `dig`	`nslookup` / `dig`
Show ARP cache	`arp -a`	`arp -a`	`ip neighbor`
Release / renew DHCP	`ipconfig /release` `ipconfig /renew`	`sudo ipconfig set en0 DHCP`	`sudo dhclient -r` `sudo dhclient`
Clear DNS cache	`ipconfig /flushdns`	`sudo dscacheutil -flushcache`	varies by distro
Trace the path	`tracert`	`traceroute`	`traceroute`

The one that catches people: the trace command is **`tracert`** on Windows and **`traceroute`** everywhere else — an easy exam point to lose.

What Four Numbers to Read, and What Each Wrong Value Means

When you run `ipconfig /all`, four values tell you almost everything:

```
   IPv4 Address. . . . . . . . . . . : 192.168.1.47      ← do I have an address?
   Subnet Mask . . . . . . . . . . . : 255.255.255.0     ← right size network?
   Default Gateway . . . . . . . . . : 192.168.1.1       ← can I leave the subnet?
   DNS Servers . . . . . . . . . . . : 8.8.8.8           ← can I resolve names?
```

Reading the address is the fastest diagnosis in networking, because certain values are confessions:

- **169.254.x.x** — this is **APIPA**, the address a host assigns itself when **DHCP got no answer**. It is a symptom, not a configuration. It means the DHCP server is down, the cable/VLAN is wrong, or a relay (`ip helper-address`) is missing. Seeing 169.254 tells you to stop looking at the PC and go find out why DHCP is silent.
- **No default gateway** — local devices work, nothing outside the subnet does. Perfectly explains "I can reach the file server but not the internet."
- **Wrong subnet mask** — the sneaky one. The PC miscalculates which addresses are local, so some destinations work and others don't, seemingly at random.
- **Address correct, DNS blank or wrong** — `ping 8.8.8.8` succeeds but `ping google.com` fails. That exact pairing is the classic signature of a DNS problem, and it's a favorite exam scenario.

Follow the layered method from 23.1: confirm the PC's own settings first, then the gateway, then beyond. Most "the internet is broken" tickets are answered by these four lines.

Chapter 24: Exam Tips & Study Plan

24.1 About the CCNA Exam (200-301)

- **One exam** covers everything.
- Around **100–120 questions** in **120 minutes**.
- Question types: multiple choice, drag-and-drop, and **simulations** (you actually configure a virtual device!).
- You **cannot go back** to previous questions — answer carefully the first time.

24.2 What's on It (Topic Weights)

Area	Rough %
Network Fundamentals	20%
Network Access (switching, VLANs, Wi-Fi)	20%
IP Connectivity (routing, OSPF)	25%
IP Services (DHCP, DNS, NAT, NTP)	10%
Security Fundamentals	15%
Automation & Programmability	10%

24.3 Pick Your Timeline

> **The day-to-day method** — what "finish a chapter" means, the review-first daily frame, and the chapter-by-chapter path through all five resources — lives in the **Study Tips** document (`CCNA_Study_Tips.pdf`). The timelines below are the calendars; that document is the loop you run inside each day.

Two plans below: an **8-week** pace for studying alongside a job, and a **2-week sprint** if you've set yourself a deadline. Both cover the same 24 chapters, 140 questions and 60 drills — the sprint just removes the slack.

The 8-Week Plan (steady pace, ~1–1.5 hrs/day)

- **Weeks 1–2:** Fundamentals, OSI/TCP-IP, cables, binary/hex — including network architectures, virtualization and PoE (Chapters 1–5). → Question Bank Domain 1.
- **Weeks 3–4:** Switching, VLANs, trunking, STP and its guards, EtherChannel (Chapters 6–10). → Domain 2.
- **Week 5:** IP addressing & subnetting — practice DAILY with the Drill Sheet (Chapters 11–13).

- **Week 6:** Routing, static routes, OSPF (with DR/BDR), FHRP/HSRP (Chapters 14–16). → Domain 3.
- **Week 7:** IP services incl. NAT and QoS, security, ACLs, wireless (Chapters 17–20). → Domains 4 and 5.
- **Week 8:** Management, automation (incl. AI/ML and Terraform), troubleshooting, review, practice exams (Chapters 21–24). → Domains 6, 7 and 8.

Every week: work that week's domain in the **Practice Question Bank**, keep drilling subnetting, and run your **Anki reviews daily** (unlock that week's subdecks as you reach them). In the final week, use the **Blueprint Coverage Map** appendix as a checklist — anything you can't explain, go re-read.

The 2-Week Sprint (~3–4 hrs/day)

This is aggressive but genuinely doable — it's roughly 45 hours of focused work. The trade-off is honest: **you'll have less repetition**, so the review days and daily subnetting are not optional. They're what stops week 1 leaking out of your head by day 12.

Each day: read the chapters → do that day's drills → answer the listed questions → run your Anki reviews → note anything you couldn't explain out loud.

Anki is doing the heavy lifting on a two-week timeline — it's what keeps day 2's material alive on day 13. Import the deck on day 1 and never skip the reviews, even on a day you read nothing.

Day	Read	Subnetting	Questions
1	Ch 1–3 — networks, devices, architectures, cables, PoE	—	Domain 1, first half
2	Ch 4–5 — binary & hex, MAC addresses, Ethernet	Toolkit page + Section 1	Domain 1, rest
3	Ch 6–7 — switches, MAC tables, VLANs	Section 1 (repeat for speed)	Domain 2, first half
4	Ch 8–9 — trunks, inter-VLAN routing, STP + the four guards	Section 2	—
5	Ch 10–11 — EtherChannel, IP addressing, gateways	Section 2 (repeat)	Domain 2, rest
6	**Ch 12 — subnetting.** The big one.	Sections 3 + 4	—
7	**Catch-up & review.** Re-read anything shaky from days 1–6.	Section 5 (VLSM)	Re-do every question you missed
8	Ch 13–14 — IPv6, routing tables, AD & metric	10 mixed problems	Domain 3, first third

Day	Read	Subnetting	Questions
9	Ch 15 + 14.7 — static routes, floating statics, FHRP/HSRP	10 mixed problems	Domain 3, second third
10	Ch 16 — OSPF, including DR/BDR and network types	10 mixed problems	Domain 3, rest
11	Ch 17 — DHCP, DNS, NAT, NTP, syslog, SNMP, QoS	Section 6 (timed!)	Domain 4
12	Ch 18–19 — security, AAA, Layer 2 attacks, ACLs	10 mixed problems	Domain 5 + 5b
13	Ch 20–22 — wireless, management, automation, AI/ML, IaC	Section 7 (boss level)	Domain 6
14	Ch 23–24 + **Blueprint Coverage Map appendix** as a checklist	Section 6 again, timed	Domains 7 & 8, then re-do every miss

How to know the sprint is working. By **day 7** you should be able to subnet without the chart. By **day 11** you should be scoring above 80% on a domain the first time through. If you're not, you're reading too fast — slow down and cut something rather than skimming everything.

If you fall behind, cut in this order (protecting what the exam weights most — IP Connectivity is 25%, and subnetting runs through everything):

1. **The depth sections first** (2.7, 9.10, 9.11, 10.4, 13.8, 14.8, 16.11) — they go beyond what the exam asks. Come back to them after you pass; they're your CCNP head start. Exception: read 14.8, the packet walk — it makes everything else easier.
2. The "Story Time" boxes and the deeper why paragraphs — read the bold summary lines only.
3. Chapter 22's tool details (keep the concepts: controller-based networking, REST, JSON, Ansible/Terraform).
4. Chapter 4's binary drills — if you can already convert quickly.
5. **Never cut:** subnetting practice, OSPF, VLANs/trunking, or ACLs.

The day before the exam: don't learn anything new. Re-read the **cheat sheet (24.5)**, walk the **Blueprint Coverage Map** and say each topic out loud, do ten subnetting problems, and sleep. Cramming new material the night before mostly displaces what you already knew.

24.4 Golden Study Tips

1. **Subnet every day.** It's the #1 skill and appears everywhere. Do 5 practice subnets each morning.

2. **Build labs.** Work through the **Lab Workbook** — 17 labs for EVE-NG/GNS3 that follow this guide chapter by chapter, each with a break-and-fix challenge. (Cisco Packet Tracer covers the one thing emulators can't: the WLC GUI.)
3. **Type commands by hand**, don't just read them. Muscle memory matters in simulations.
4. **Learn the "why," not just the "what."** Understanding beats memorizing.
5. **Do practice exams** to get used to the style and timing.
6. **Use the flashcard deck daily.** `CCNA_Flashcards.apkg` imports into Anki (or `.tsv` into Quizlet) — 189 cards covering ports, AD values, STP costs, election rules and the depth material, split into subdecks by topic. **Spaced repetition is the only study method proven to beat forgetting**, and 15 minutes a day beats an hour on Sunday.
7. **Teach someone else** — if you can explain it simply, you truly know it.

Why these specific tips — what's the reasoning behind each? They're not random; each one targets how the exam actually tests you:

- **Why subnet DAILY (not just once)?** Because the exam is **timed** (~60–90 seconds per question) and subnetting appears everywhere. Knowing how to subnet isn't enough — you need it to be automatic, like a times-table, so it doesn't eat your clock. Speed only comes from daily reps; cramming it once won't build the reflex.
- **Why build LABS instead of just reading?** Because the exam has **simulations where you configure real virtual devices.** Reading a command and typing it under pressure are totally different skills — the second only comes from doing. Labs also reveal the little errors (a typo, a forgotten `no shutdown`) that reading never exposes.
- **Why learn the "WHY" (this whole guide's philosophy)?** Because the exam loves "**choose the BEST answer**" questions where several options look correct. Memorizers get stuck; people who understand the reasoning can eliminate the traps. Understanding also means you can reconstruct a fact you forgot — if you get why AD ranks connected over static over OSPF, you don't need to have memorized the exact list.
- **Why teach someone else?** Because explaining something simply is the ultimate test of whether you actually understand it. If you stumble explaining VLANs to a friend, you've found a gap — before the exam finds it for you.

The theme: study the way the exam tests — fast recall, hands-on config, and reasoning — not just passive reading.

24.5 Must-Memorize Cheat Sheet

Port numbers: FTP 20/21 · SSH 22 · Telnet 23 · SMTP 25 · DNS 53 · DHCP 67/68 · HTTP 80 · HTTPS 443 · SNMP 161.

Administrative Distance: Connected 0 · Static 1 · EIGRP 90 · OSPF 110 · RIP 120.

Private IP ranges: 10.0.0.0/8 · 172.16–31.0.0 · 192.168.0.0/16.

STP cost: 10M=100 · 100M=19 · 1G=4 · 10G=2.

Hosts formula: 2^(host bits) − 2.

DORA: Discover, Offer, Request, Ack.

TCP handshake: SYN → SYN-ACK → ACK.

Wildcard mask: flip the subnet mask (0 = match, 255 = any).

HSRP: highest priority wins (default 100) · ties go to highest IP · `preempt` needed to take the role back · hello 3 s / hold 10 s.

OSPF DR election: highest priority (default 1) · ties go to highest Router ID · priority 0 = never · **not preemptive** · 2WAY between DROTHERs is normal.

QoS markings: Voice = **EF (DSCP 46)** · Video = AF41 (34) · Signaling = CS3 (24) · Best effort = 0. Policing drops, shaping buffers.

PoE: 802.3af ≈ 15.4 W · 802.3at (PoE+) ≈ 30 W · 802.3bt ≈ 60–100 W.

Client OS: `ipconfig` (Windows) · `ifconfig` (macOS) · `ip address` (Linux) · **169.254.x.x = DHCP failed (APIPA)**.

REST: GET=Read · POST=Create · PUT=Update · DELETE=Delete · **4xx = your error, 5xx = server error**.

Appendix: Exam Blueprint Coverage Map

Cisco publishes an official topic list for the **CCNA 200-301 (v1.1)** exam. This table maps **every** blueprint item to where it's covered in this guide — use it as a final checklist before exam day, and to find the right section fast when a practice question catches you out.

1.0 Network Fundamentals (20%)

#	Blueprint topic	Where
1.1	Role and function of network components	1.4, 1.5, 3.7 (PoE), 20.6 (WLC/AP)
1.2	Network topology architectures (2-tier, 3-tier, spine-leaf, WAN, SOHO, cloud)	1.3, **1.7**, **1.8**
1.3	Physical interface and cabling types	3.2, 3.3, 3.6
1.4	Identify interface and cable issues	3.5, 23.4, 23.5
1.5	Compare TCP to UDP	2.5, 2.6
1.6	Configure and verify IPv4 addressing and subnetting	11, 12 (+ drill sheet)
1.7	Private IPv4 addressing	11.4
1.8	Configure and verify IPv6 addressing and prefix	13.2–13.6
1.9	IPv6 address types	13.4
1.10	Verify IP parameters for client OS	**23.6**
1.11	Wireless principles (channels, SSID, RF, encryption)	20.1–20.4
1.12	Virtualization fundamentals (VMs, containers, VRFs)	**1.9**
1.13	Switching concepts (MAC learning/aging, flooding, MAC table)	6.1, 6.2

2.0 Network Access (20%)

#	Blueprint topic	Where
2.1	Configure and verify VLANs across switches	7.1–7.5
2.2	Interswitch connectivity (trunks, 802.1Q, native VLAN)	8.1, 8.2
2.3	Layer 2 discovery protocols (CDP, LLDP)	21.2
2.4	EtherChannel (LACP)	10.1–10.3
2.5	Rapid PVST+ (root bridge, port states, PortFast, guards)	9.1–9.7, **9.8**, **9.9**
2.6	Cisco wireless architectures and AP modes	20.6, **20.7**
2.7	Physical infrastructure of WLAN components (AP/WLC ports, LAG)	**20.8**
2.8	AP and WLC management access connections	**20.9**, 21.1
2.9	Wireless LAN GUI configuration for client connectivity (WLAN, security, QoS profiles)	20.5, **20.10**

3.0 IP Connectivity (25%)

#	Blueprint topic	Where
3.1	Components of the routing table	14.2, 14.4, 14.5
3.2	How a router makes forwarding decisions	14.3, 14.4, 14.5
3.3	IPv4 and IPv6 static routing	15.1–15.6
3.4	Single-area OSPFv2 (adjacencies, point-to-point, broadcast DR/BDR, RID)	16.3–16.9, **16.10**
3.5	First hop redundancy protocols (FHRP)	**14.7**

4.0 IP Services (10%)

#	Blueprint topic	Where
4.1	Inside source NAT (static and pools)	17.3
4.2	NTP in client and server mode	17.4
4.3	Role of DHCP and DNS	17.1, 17.2
4.4	Function of SNMP	17.6
4.5	Syslog features (facilities and levels)	17.5
4.6	DHCP client and relay	17.1
4.7	Per-hop behavior for QoS (classification, marking, queuing, policing, shaping)	**17.7**
4.8	Remote access using SSH	6.6
4.9	Capabilities of TFTP/FTP	21.3, 21.4

5.0 Security Fundamentals (15%)

#	Blueprint topic	Where
5.1	Key security concepts (threats, vulnerabilities, exploits, mitigation)	18.1, 18.2
5.2	Security program elements (awareness, training, physical access)	**18.9**
5.3	Device access control using local passwords	6.5, 18.5
5.4	Password policies and alternatives (MFA, certificates, biometrics)	18.5, **18.8**
5.5	IPsec remote access and site-to-site VPNs	18.7
5.6	Configure and verify ACLs	19.1–19.8
5.7	Layer 2 security (DHCP snooping, DAI, port security)	6.8, 18.3
5.8	AAA concepts	18.4
5.9	Wireless security protocols (WPA, WPA2, WPA3)	20.5
5.10	Configure WLAN using WPA2 PSK in the GUI	20.5, 20.6

6.0 Automation and Programmability (10%)

#	Blueprint topic	Where
6.1	How automation impacts network management	22.1
6.2	Traditional vs. controller-based networking	22.2
6.3	Controller-based, software-defined architecture (overlay, underlay, fabric)	22.2, 22.3
6.4	AI (generative and predictive) and machine learning in network operations	**22.8**
6.5	Characteristics of REST-based APIs (CRUD, HTTP verbs, data encoding)	22.4
6.6	Configuration management mechanisms (Ansible, Terraform)	22.6, **22.9**
6.7	Interpret JSON-encoded data	22.5

> **Bold** sections are the ones most often skipped by self-study candidates — FHRP, QoS, virtualization, the STP guards, DR/BDR, and AI/ML. They're small topics individually, but together they're worth a meaningful slice of the exam. Don't leave them for the last night.

Appendix B: Beyond CCNA — The Road to CCNP

CCNA proves you understand a network. **CCNP proves you can design, scale and troubleshoot one.** This appendix maps what you've learned onto what comes next, so the depth sections in this guide have somewhere to go.

The Shape of the Certification

CCNP Enterprise is **two exams**, not one:

Exam	Role
ENCOR 350-401 (Core)	Required. Architecture, virtualization, infrastructure, assurance, security, automation

Exam	Role
One concentration	Your specialty — most commonly **ENARSI 300-410** (advanced routing)

The mindset shift matters more than the syllabus. CCNA asks "what does this do, and how do you configure it?" CCNP asks "why this design, what breaks at scale, and how do you prove it's working?" Questions stop having one clean answer and start having trade-offs — which is exactly why this guide keeps explaining why rather than only what.

What Carries Straight Over

These aren't re-taught at CCNP — they're **assumed**, on day one:

You learned	CCNP expects you to already be fluent
Subnetting (Ch 12)	Instantly, including VLSM in your head
The packet walk (14.8)	As your default troubleshooting method
VLANs, trunking, STP (Ch 7–9)	Plus the mechanics in 9.10 and 9.11
OSPF single-area (Ch 16)	Plus adjacency states and LSA types (16.11)
ACLs (Ch 19)	Extended, named, and used for far more than filtering
TCP mechanics (2.7)	When diagnosing "the network is slow"

If any of those feel shaky, strengthen them before moving on — CCNP builds directly on top and won't stop to review.

What's Genuinely New

Routing gets much bigger

- **BGP** — the protocol that runs the internet. Not on CCNA at all, and a large slice of CCNP. Path-vector rather than link-state: routing by **policy** ("prefer this provider") instead of purely by cost.
- **EIGRP in depth** — feasible successors, the DUAL algorithm, stuck-in-active.
- **Multi-area OSPF** — the ABR/ASBR roles, LSA types 3–7, and the stub area types previewed in 16.11.
- **Route redistribution** — moving routes between protocols, where routing loops are genuinely easy to create.
- **Route maps and prefix lists** — the tools for expressing policy.

Overlays and virtualization

- **GRE and IPsec tunnels** — a virtual link across someone else's network.
- **VRFs** — you met the idea in 1.9; CCNP makes them routine.

- **VXLAN and LISP** — the fabric technologies underneath modern data centers.
- **SD-Access and SD-WAN** — the controller-based architectures Chapter 22 introduced, in operational detail.

Assurance — proving it works

Largely absent from CCNA, and a real part of the job:

- **NetFlow** — who is talking to whom, and how much.
- **SPAN / RSPAN** — mirroring traffic to a capture tool.
- **IP SLA** — synthetic probes that measure the path continuously, often driving failover.

Automation deepens

Chapter 22 covered concepts. CCNP wants **working code**: Python scripts against device APIs, real Ansible playbooks, EEM applets, and JSON/YAML you can read fluently.

A Sensible Order After You Pass

1. **Build something real first.** A multi-site lab in GNS3/EVE-NG or CML beats reading. Break it deliberately.
2. **Learn BGP early.** It's the biggest new concept and it rewards time.
3. **Get comfortable in Python.** Automation is the fastest-growing part of the exam and the job.
4. **Then pick a concentration** — ENARSI if you like routing and troubleshooting, ENWLSI for wireless, ENAUTO for automation.

Where This Guide Already Took You Past CCNA

The depth sections deliberately go beyond exam requirements, because they're where CCNP starts:

Section	Beyond-CCNA content
2.7	Sliding windows, congestion vs flow control, MSS/MTU and tunnel breakage
9.10	BPDU fields, extended system ID, the four-step decision order
9.11	RSTP roles, proposal/agreement sync, link types
10.4	Load-balancing hash behaviour and its per-flow consequence
13.8	The full NDP message set, DAD, RA flags
14.8	The complete end-to-end forwarding walk

Section	Beyond-CCNA content
16.11	Neighbor state machine, LSA types 1–7, SPF, stub/NSSA areas

One last thing. The single habit that carries furthest isn't a protocol — it's insisting on knowing why something works before you accept that it does. That's what separates someone who passed an exam from someone you'd want fixing your network at 3 a.m.

Glossary (Quick Definitions)

- **ABR (Area Border Router):** An OSPF router joining two areas; generates Type 3 summary LSAs.
- **ACL:** Rules that permit or deny traffic.
- **AD (Administrative Distance):** Trust score for routing sources (lower = better).
- **AI/ML in networking:** Using learned baselines instead of fixed thresholds to spot anomalies and predict failures (see 22.8).
- **Alternate port:** RSTP's pre-computed standby for the root port — promoted instantly on failure.
- **AP (Access Point):** Broadcasts Wi-Fi and bridges to the wired network.
- **APIPA:** The 169.254.x.x address a host gives itself when DHCP doesn't answer — a symptom, not a setting.
- **ARP:** Finds a device's MAC from its IP.
- **ASBR:** Autonomous System Boundary Router — injects external routes into OSPF as Type 5 LSAs.
- **Bandwidth:** How much data a link can carry.
- **BGP:** The internet's path-vector routing protocol. Not on CCNA; a major CCNP topic.
- **BPDU:** Messages STP uses to find loops.
- **Broadcast:** A message sent to everyone on a network.
- **CAPWAP:** The tunnel protocol between a lightweight AP and its WLC — control on UDP 5246 (DTLS-encrypted), client data on UDP 5247.
- **Catalyst Center:** Cisco's campus SDN controller (formerly DNA Center) — intent-based config push, plug-and-play provisioning and assurance.
- **CDP/LLDP:** Protocols to discover neighbor devices.
- **CIDR:** Slash notation for subnet masks (e.g., /24).
- **Collapsed core:** A two-tier design where the core and distribution layers are merged; used at smaller sites.
- **Collision Domain:** An area where frames can collide.
- **Congestion control:** The sender's own guess about network capacity, inferred from packet loss.

- **Container:** An app packaged with its dependencies that shares the host OS kernel — smaller and faster to start than a VM.
- **DAD (Duplicate Address Detection):** An IPv6 host asking for its own tentative address; silence means it's free.
- **Default Gateway:** The router address used to leave your network.
- **DHCP:** Hands out IP addresses automatically.
- **DNS:** Turns names into IP addresses.
- **DR / BDR:** The Designated and Backup Designated Router OSPF elects on a broadcast segment to cut down adjacencies.
- **DSCP:** The 6-bit QoS marking in the IP header (voice = EF / 46).
- **Duplex:** One-way-at-a-time (half) vs both-ways (full).
- **Encapsulation:** Wrapping data with headers as it goes down the layers.
- **EtherChannel:** Bundling multiple links into one.
- **Extended System ID:** The 12 bits of the STP Bridge ID holding the VLAN — why priority moves in steps of 4096.
- **Fast retransmit:** Resending a segment after three duplicate ACKs, without waiting for a timeout.
- **FlexConnect:** AP mode that switches traffic locally at a branch and keeps Wi-Fi alive if the WLC becomes unreachable.
- **FHRP:** First Hop Redundancy Protocol — lets two routers share a virtual IP and MAC so the default gateway can fail over.
- **Flow control:** The receiver advertising how much buffer it has left, via the TCP Window field.
- **Frame:** Data unit at Layer 2.
- **Generative AI:** AI that creates content (configs, explanations, docs); predictive AI forecasts from patterns. Both are exam topics (22.8).
- **Gateway:** A door between networks.
- **GLBP:** Cisco FHRP that also load balances traffic across several routers.
- **Hex:** Base-16 numbers (0-9, A-F).
- **HSRP:** Cisco FHRP using Active and Standby roles.
- **Hypervisor:** Software that creates and runs virtual machines on physical hardware.
- **IaC (Infrastructure as Code):** Describing infrastructure in version-controlled files and letting a tool make reality match.
- **Idempotence:** A change that can be applied repeatedly with the same result — the point of declarative tools.
- **IP Address:** Logical address of a device.
- **Jitter:** Variation in delay. Voice needs it under ~30 ms, because uneven arrival makes audio choppy.
- **LAG:** Link aggregation on a WLC — its EtherChannel, always static (no LACP/PAgP).
- **LAN/WAN:** Local vs wide-area network.
- **Loop Guard:** Blocks a port that stops receiving BPDUs, in case a one-way link failure would otherwise create a loop.

- **LSA:** Link-State Advertisement — the pieces OSPF floods and assembles into a map.
- **LSDB:** Link-State Database — the map, identical on every router in an area.
- **MAC Address:** Permanent hardware ID (Layer 2).
- **Metric:** How a routing protocol ranks paths.
- **MSS:** Maximum Segment Size — largest TCP payload, typically 1460 on Ethernet.
- **MTU:** Maximum Transmission Unit — largest frame payload a link carries (1500 on Ethernet).
- **NAT/PAT:** Sharing private IPs behind a public IP.
- **NDP:** Neighbor Discovery Protocol — IPv6's replacement for ARP, using ICMPv6 and multicast.
- **NSSA:** Not-So-Stubby Area — a stub area still permitted its own external routes (Type 7).
- **OSI Model:** 7-layer model of networking.
- **OSPF:** A link-state routing protocol.
- **Packet:** Data unit at Layer 3.
- **PAT:** Many devices share one public IP via ports.
- **Ping:** Tests basic connectivity.
- **PoE:** Power over Ethernet — power and data on one cable (802.3af/at/bt).
- **Policing:** Dropping traffic above a rate. (Shaping buffers it instead.)
- **Port Number:** Identifies an app/service (Layer 4).
- **Preempt:** The HSRP option that lets a recovered higher-priority router reclaim the Active role.
- **Proposal/agreement:** The RSTP handshake that replaces waiting out timers on point-to-point links.
- **QoS:** Deciding which traffic goes first when a link is congested.
- **Rapid PVST+:** Cisco's default spanning-tree mode: 802.1w speed with one instance per VLAN.
- **Root Guard:** Stops a switch on a given port from becoming the STP root.
- **Router:** Connects different networks.
- **Routing Table:** A router's map of known networks.
- **SAE:** WPA3's replacement for the PSK handshake — blocks offline dictionary attacks by requiring a live exchange per guess.
- **Shaping:** Buffering traffic above a rate to send it later, smoothing bursts.
- **SLAAC:** Stateless Address Autoconfiguration — an IPv6 host building its own address from an RA prefix.
- **Sliding window:** TCP sending multiple unacknowledged segments up to the advertised window size.
- **SOHO:** Small Office / Home Office — where one box is router, switch, AP, firewall and DHCP server.
- **Solicited-node multicast:** FF02::1:FFxx:xxxx — the group NDP queries so only the target's NIC wakes up.
- **SPF (Dijkstra):** The algorithm each OSPF router runs on the LSDB to build its own shortest-path tree.

- **Spine-leaf:** Data center design where every leaf connects to every spine, giving equal latency between servers.
- **SSID:** A Wi-Fi network's name.
- **Split-MAC:** The lightweight architecture — real-time 802.11 work stays on the AP, management decisions move to the WLC.
- **STP:** Stops Layer 2 loops.
- **Stub area:** An OSPF area that blocks external LSAs and uses a default route instead.
- **Subnet:** A smaller piece of a network.
- **Subnet Mask:** Splits IP into network and host parts.
- **Switch:** Connects devices within a LAN using MACs.
- **TCP:** Reliable, ordered delivery.
- **Terraform:** A declarative, agentless infrastructure-as-code tool named in the current exam blueprint.
- **Trunk:** A link carrying many VLANs.
- **Trust boundary:** The point where the network decides whether to believe incoming QoS markings.
- **TTL:** Time To Live — decremented each routed hop; hits zero and the packet is dropped, preventing loops.
- **UDP:** Fast, no-guarantee delivery.
- **VLAN:** A virtual, separated LAN inside a switch.
- **VM (Virtual Machine):** A complete pretend computer, with its own OS, running as software on real hardware.
- **VPN:** An encrypted tunnel over the internet.
- **VRF:** Virtual Routing and Forwarding — several isolated routing tables on one router. (VLANs split L2; VRFs split L3.)
- **VRRP:** The open-standard FHRP, using Master and Backup roles.
- **Wildcard Mask:** The inverse of a subnet mask, used in ACLs/OSPF.

You Made It!

You just went through **everything** on the CCNA — from what a cable is, all the way to network automation. If some parts felt hard, that's normal. Come back, re-read, and **practice** (especially subnetting drills and the Lab Workbook).

Remember the big ideas:

- **Switches** work with **MAC** addresses inside a LAN (Layer 2).
- **Routers** work with **IP** addresses between networks (Layer 3).
- **Subnetting** is just cutting networks into smaller pieces — practice it daily.
- **Security** and **automation** are the future — learn them well.

You've got this. Good luck on your exam!

— End of Guide —